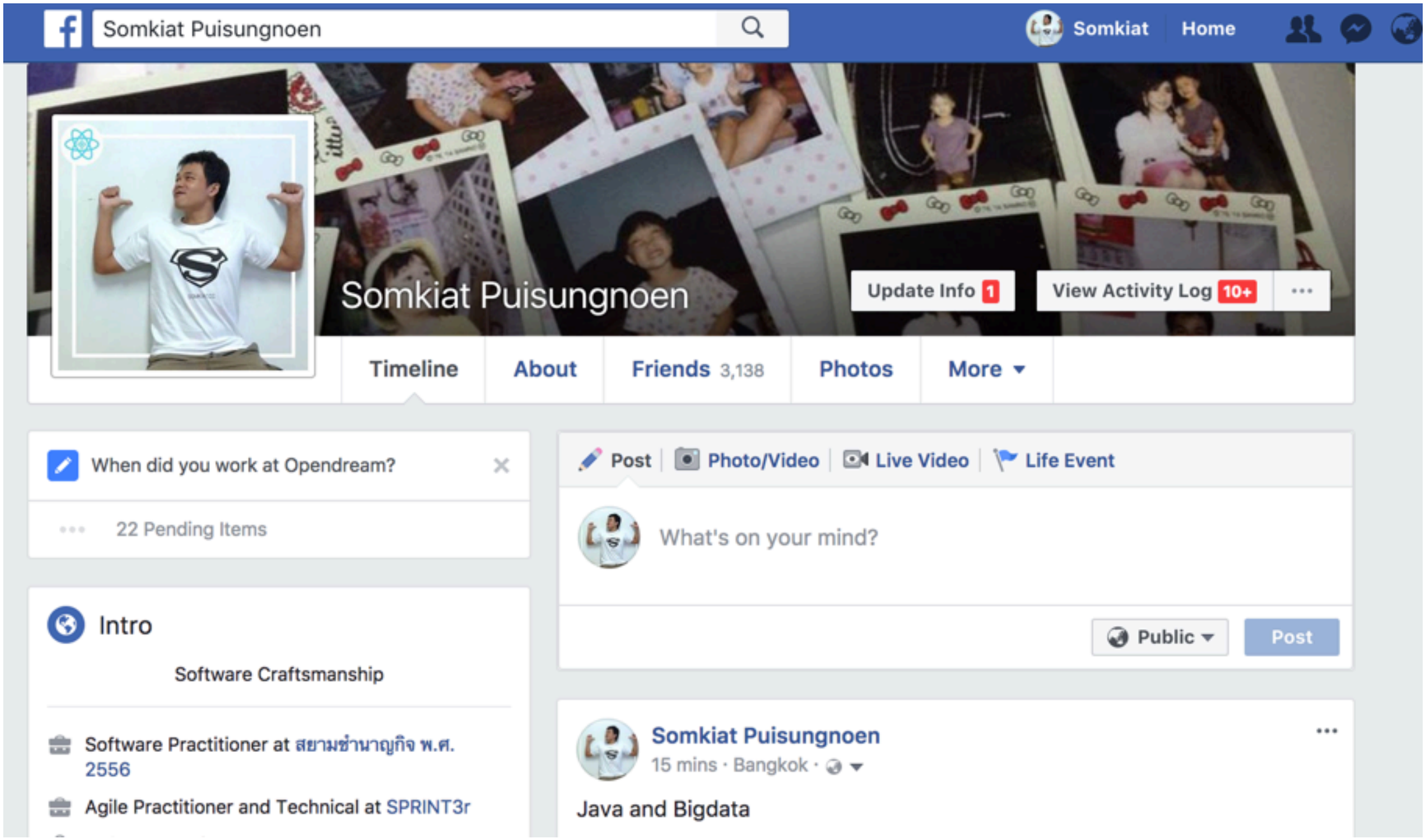
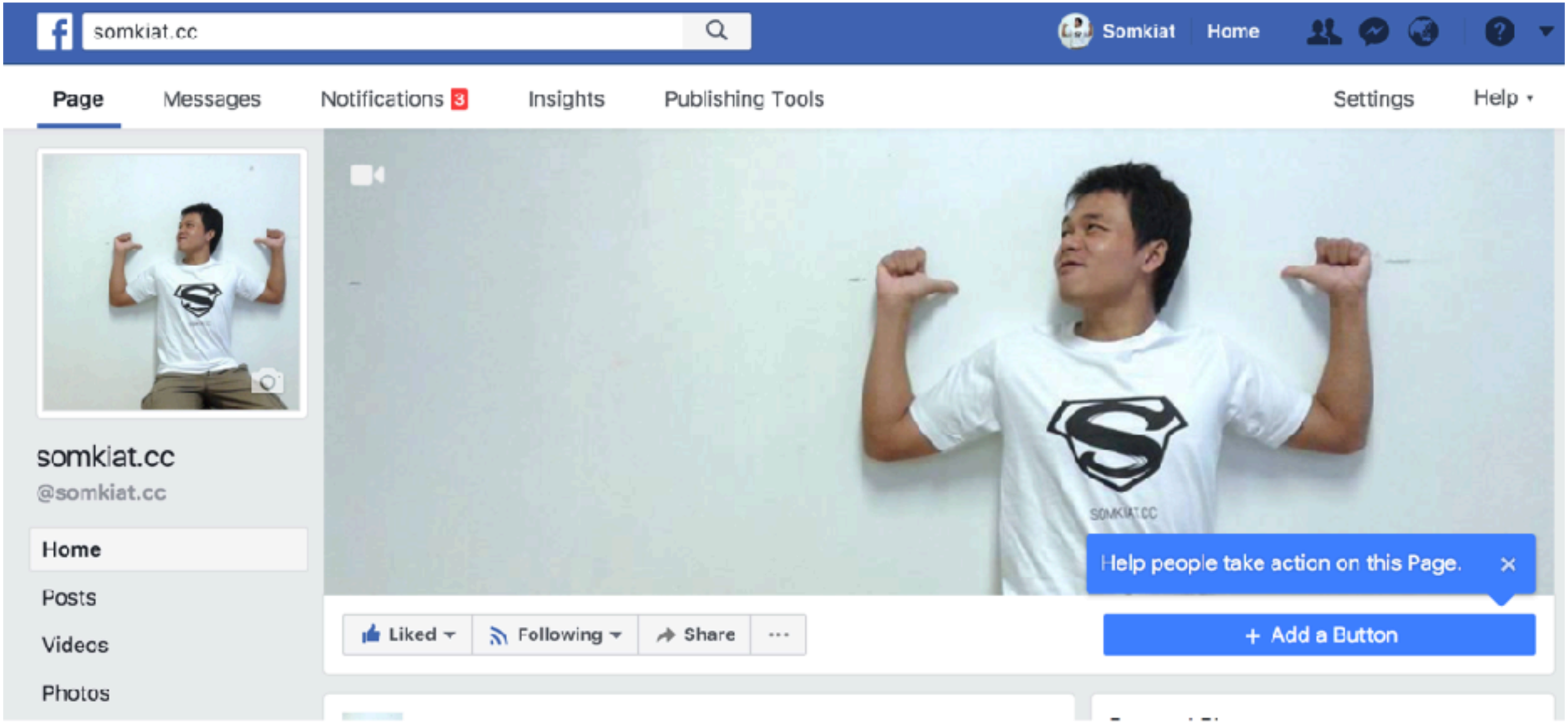
สยามชำนาญกิจ
SIAM CHAMNAN KIT

Fundamental
Java and Spring Boot





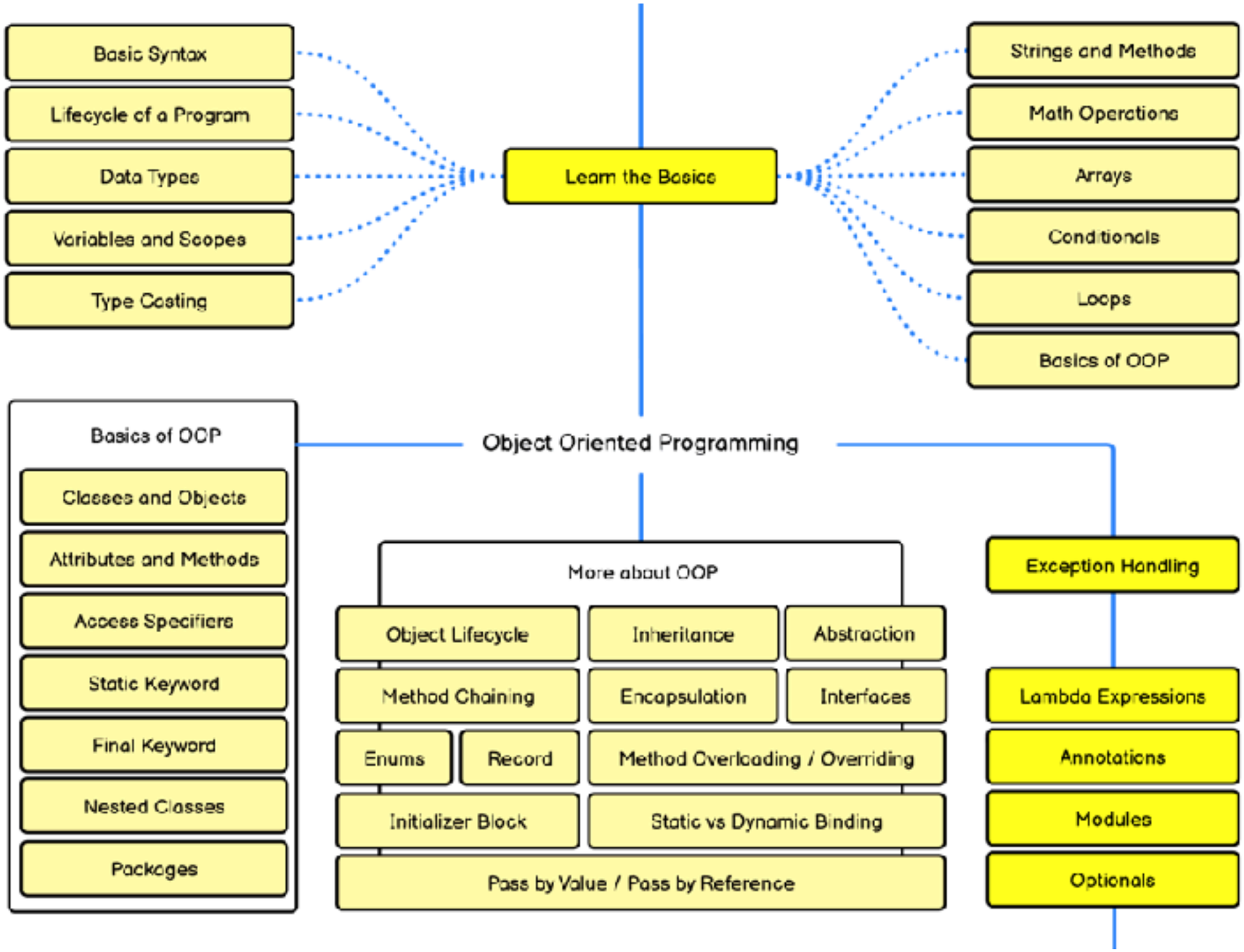
https://github.com/up1/course-java-spring-2026

Java Fundamental

Leaning Java

Modern Java (generic, optional and lambda)
Error handling
Object-Oriented Concepts with Java
Data structure with Java collection framework
Threading model
Testing with JUnit and Mockito

Java Developer Roadmap



https://roadmap.sh/java

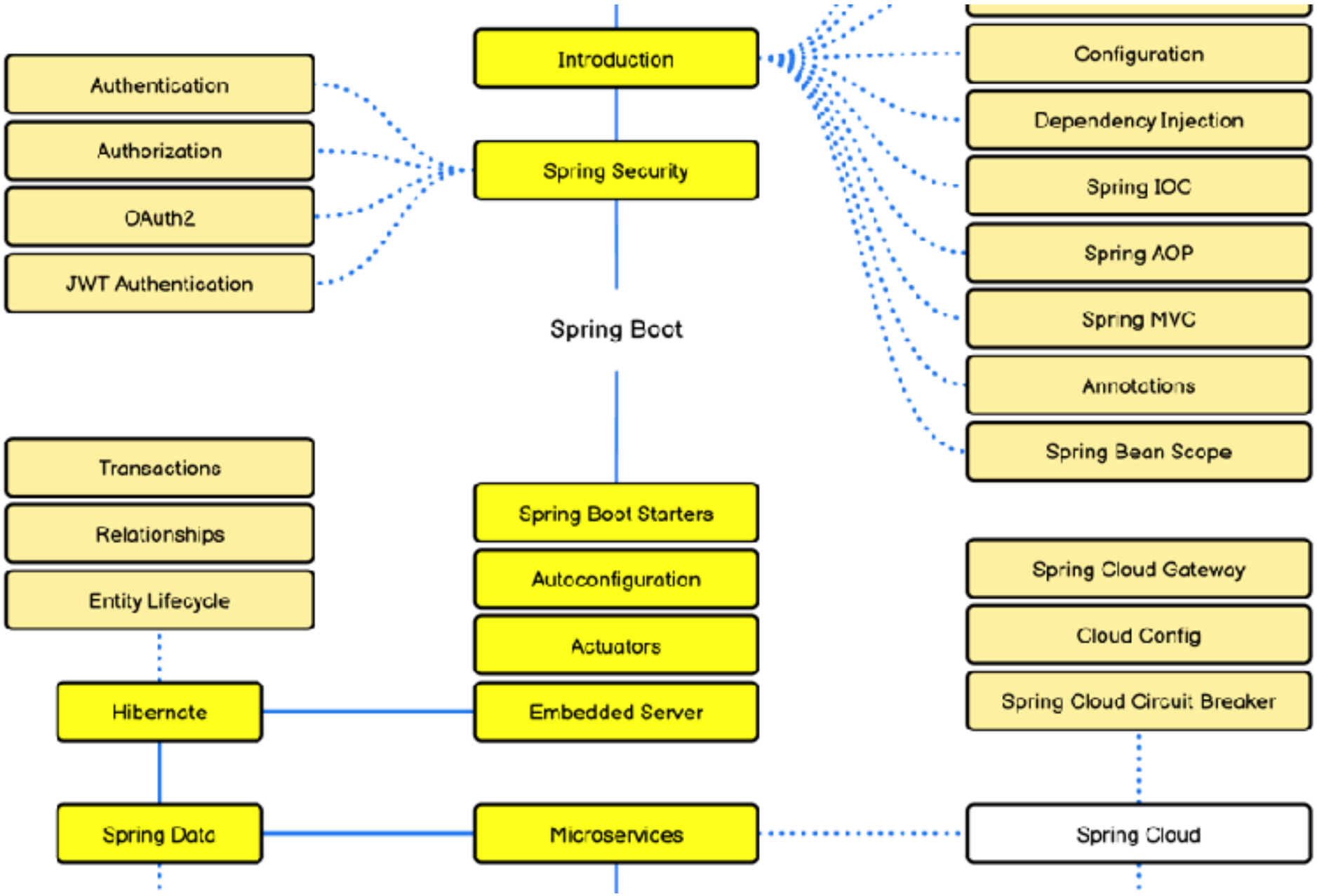
Basic of REST

REST (REpresentational State Transfer)
6 REST constraints
REST vs RESTful API
Design principles

Spring Boot

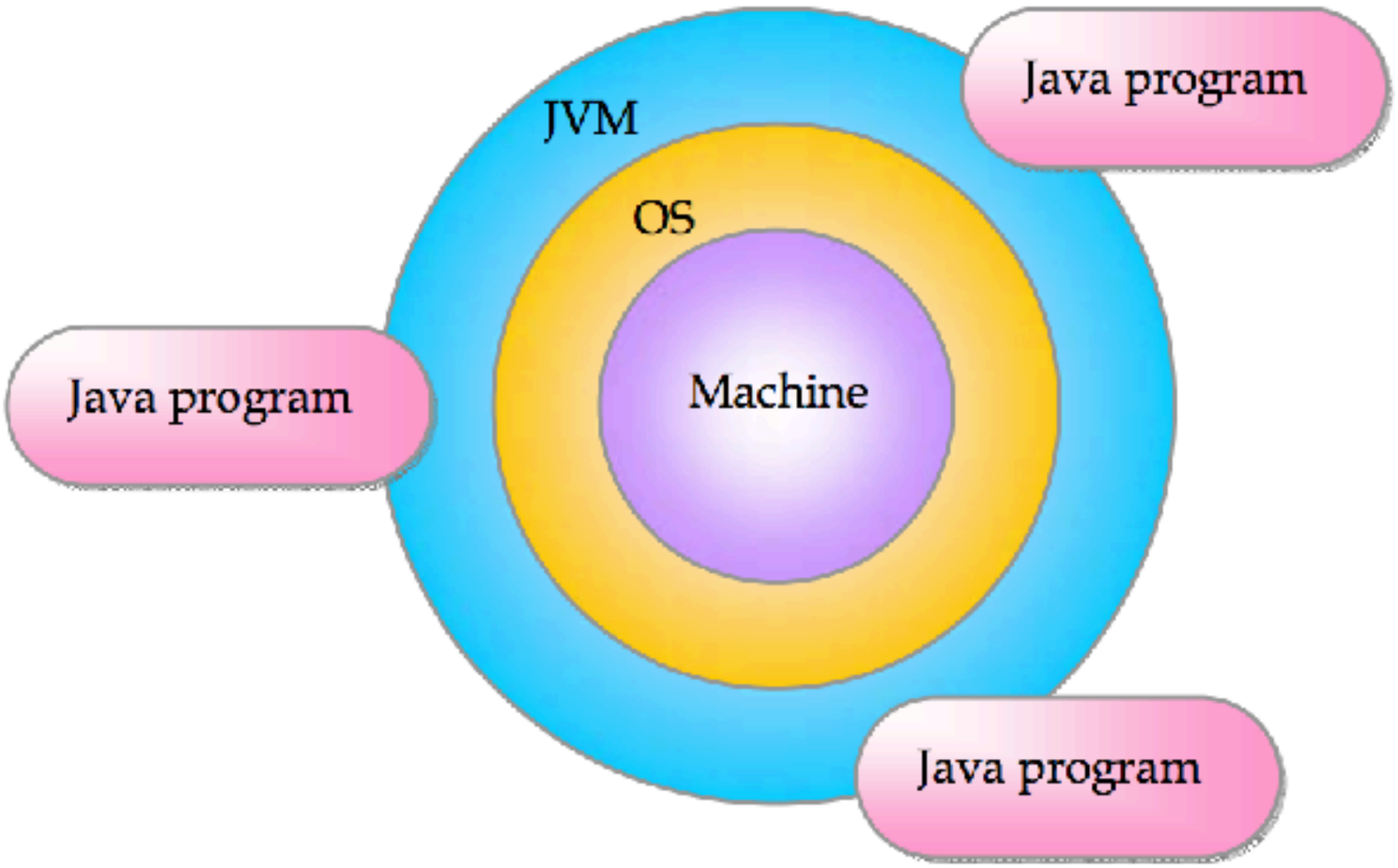
Goals of Spring Boot
Better project structure of Spring Boot
Develop RESTful APIs
Error handling
Working with database with Spring Data
Manage connections and transactions
Testing with Spring Boot
Observable service (log, trace, metric)

Spring Boot Roadmap



https://roadmap.sh/spring-boot

Write Once Run Anywhere ?

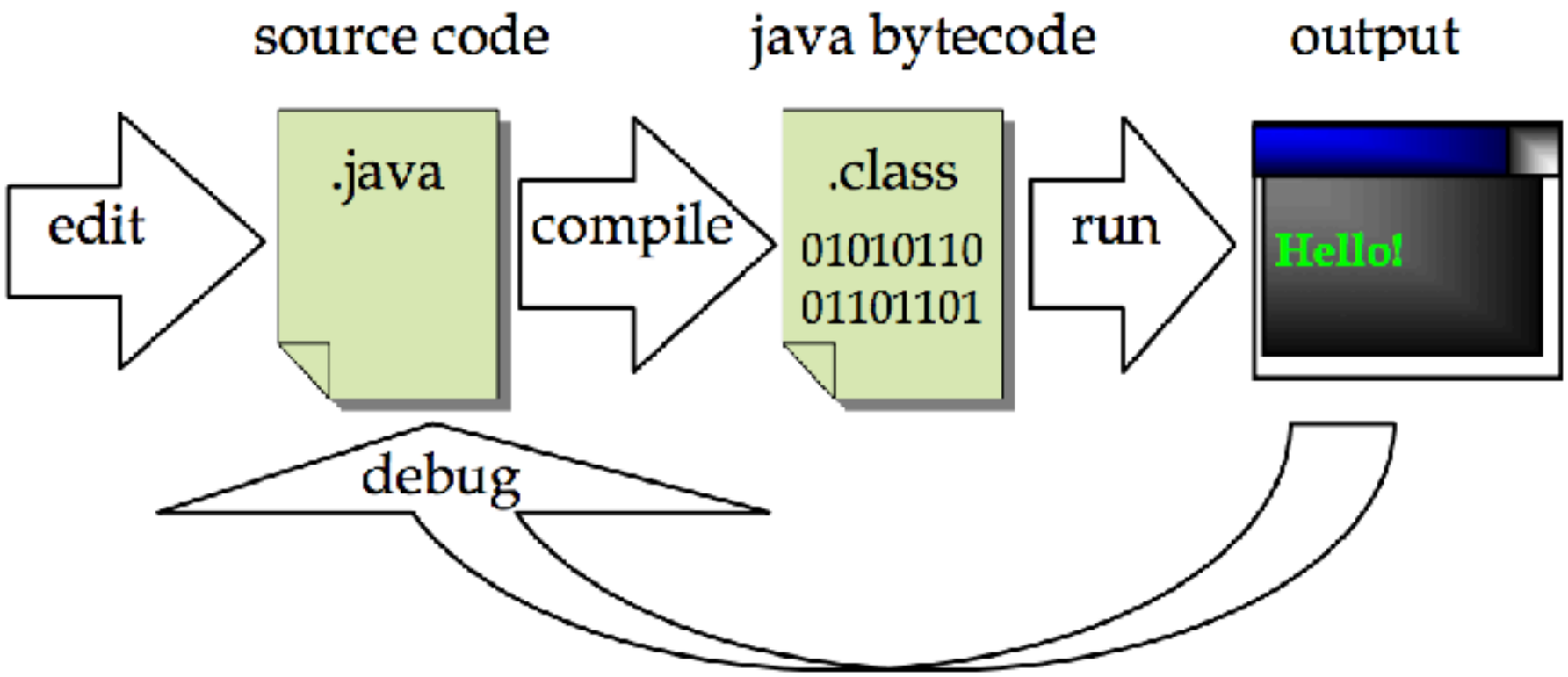


Structure of Java program

Create file HelloWorld.java

```
1 public class HelloWorld {
2
3     // Show message in a console
4     public static void main(String[] args) {
5         System.out.println("Hello World");
6
7     }
8
9 }
```

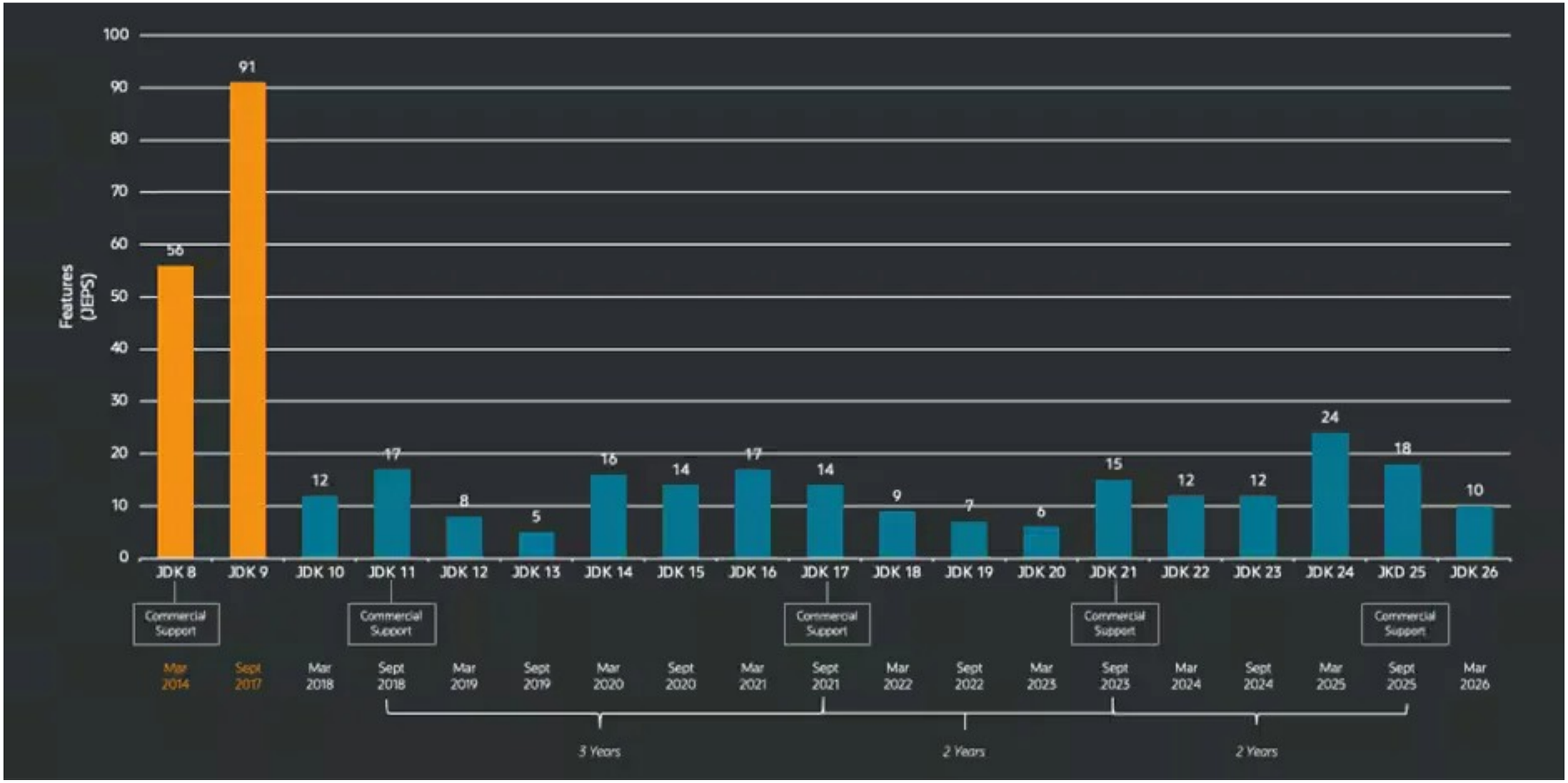
Java Development Workflow



JDK

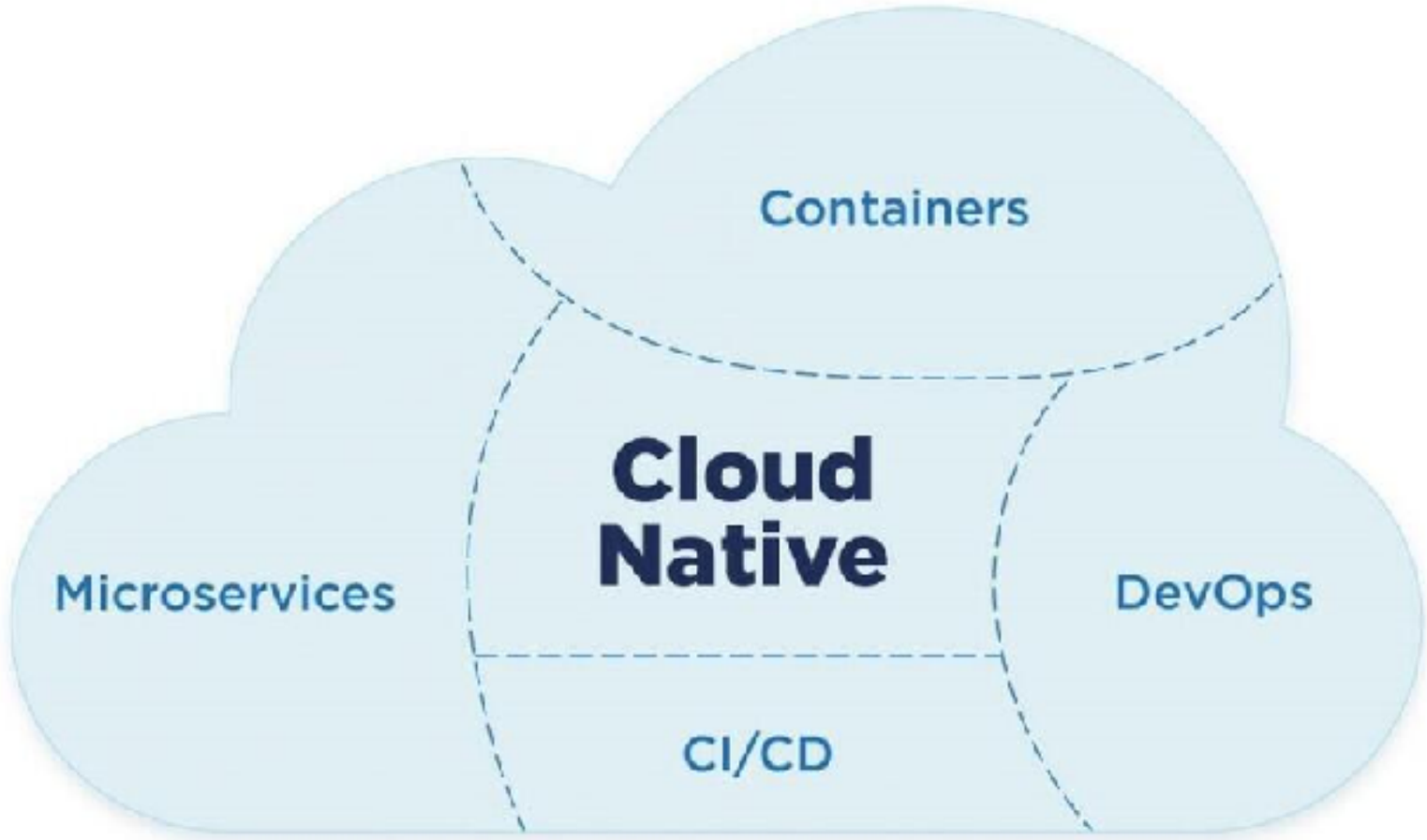
JRE

History of Java from 8 to 26



https://www.infoq.com/news/2026/03/java26-released/

Cloud Native Architecture



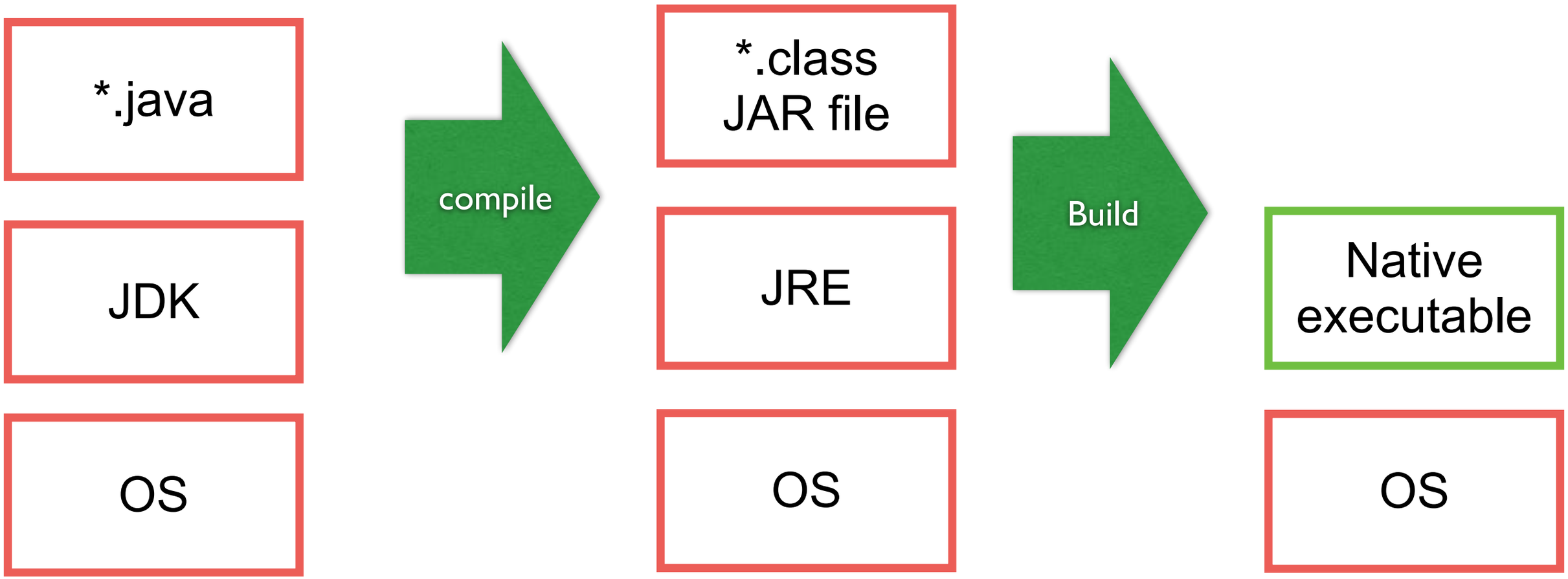
Java in Cloud Native App

Improve performance and resource usages

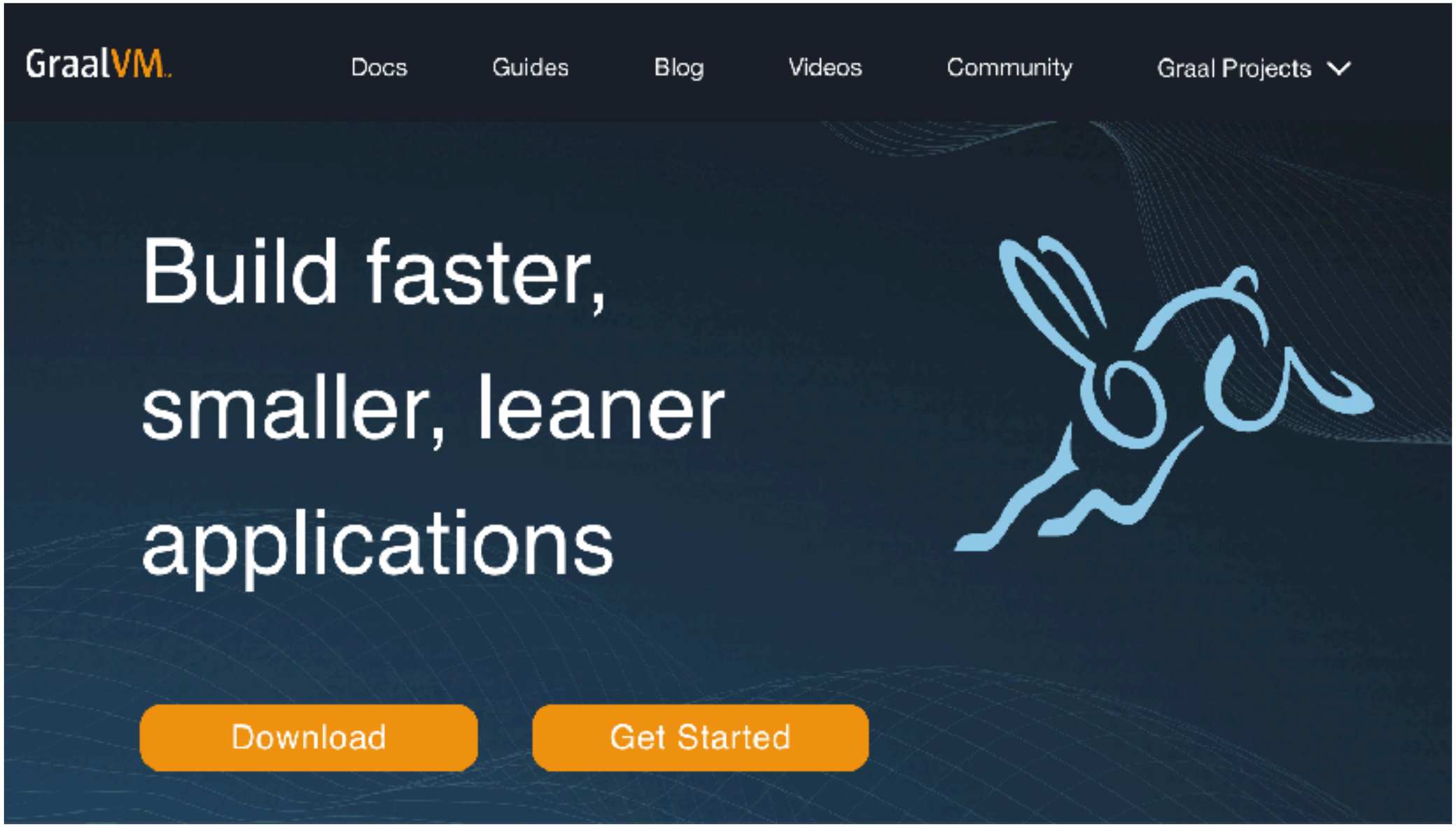
QUARKUS

GraalVM

Java in Cloud Native



Working with GraalVM



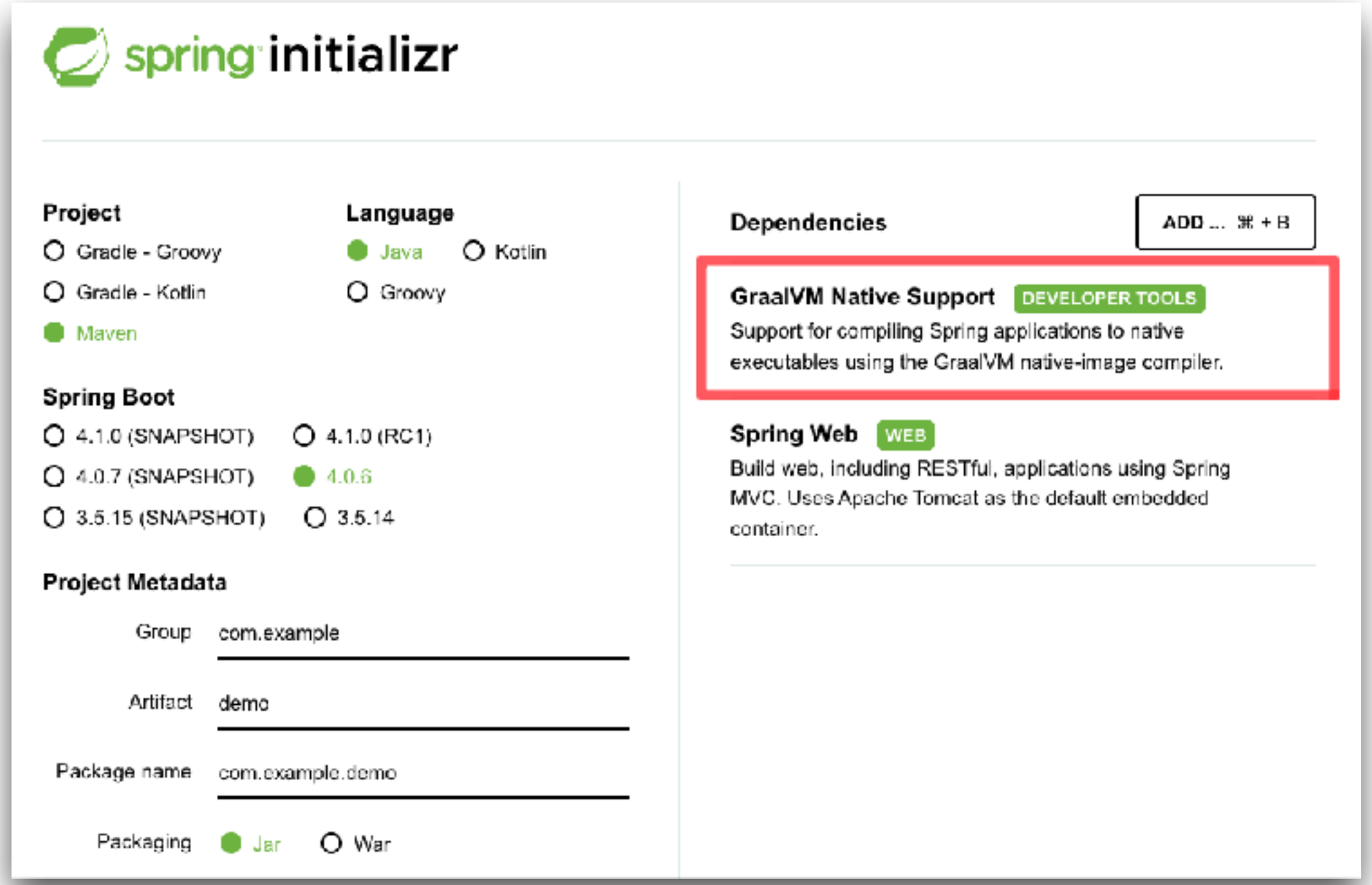
https://www.graalvm.org/

Native Image of GraalVM

Use less resources (compare with JRE)
Start in milliseconds
Deliver peak performance immediately (no warmup)
Lightweight package
Reduce attack surface

https://www.graalvm.org/latest/reference-manual/native-image/

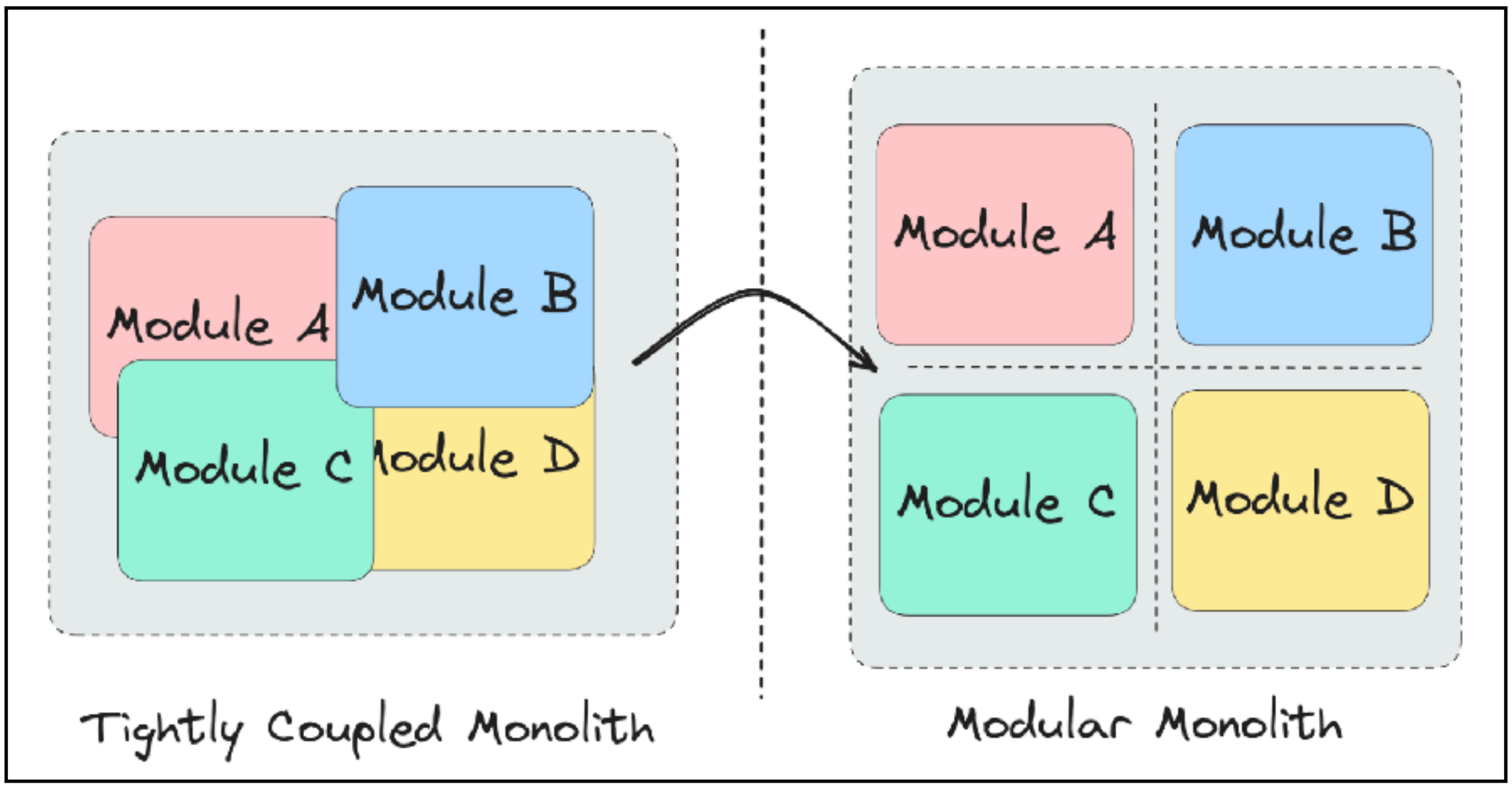
GraalVM in Spring Boot



https://start.spring.io/

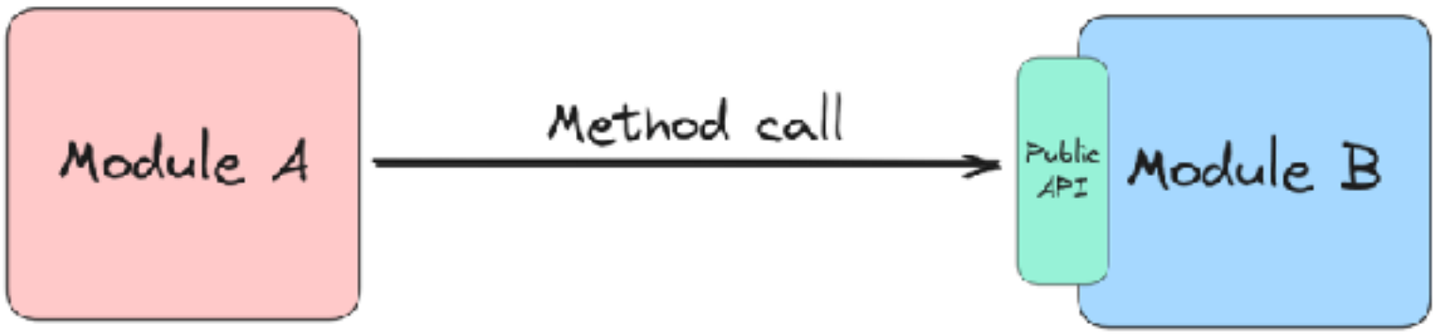
Cloud Native Architecture Design

Modular Monolith



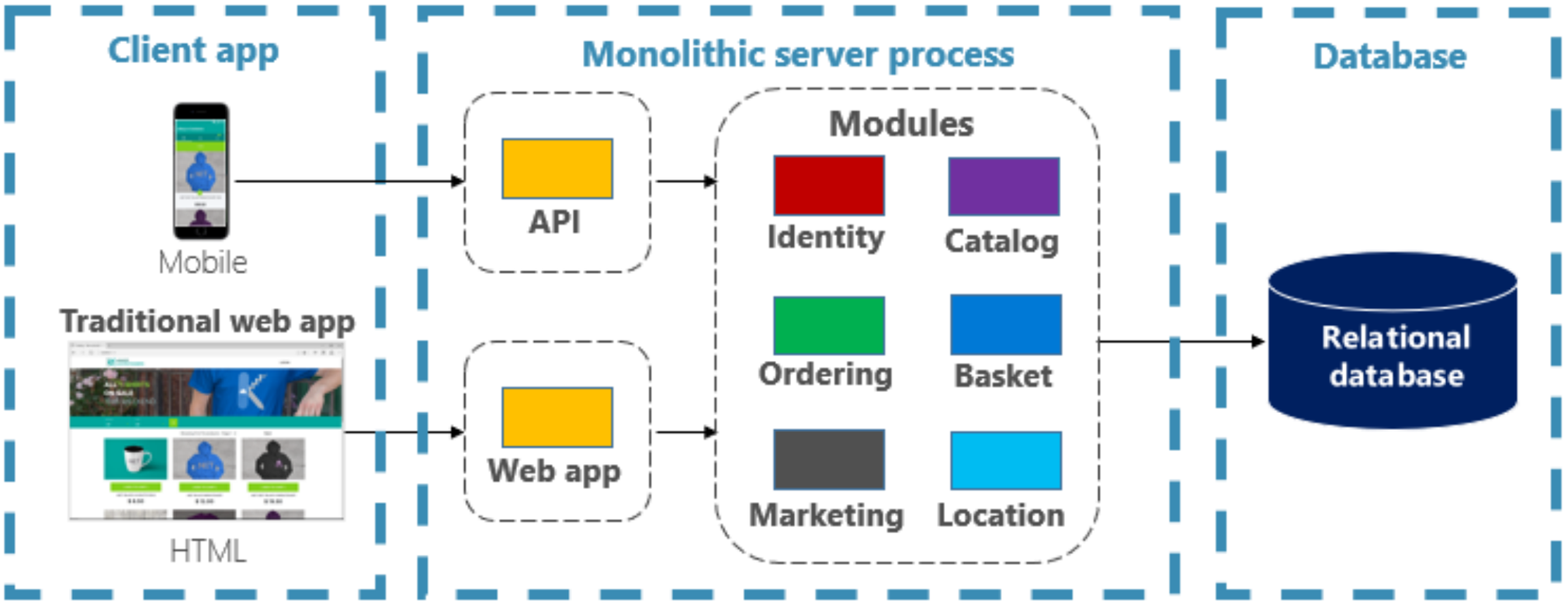
https://milanjovanovic.tech/blog/modular-monolith-communication-patterns

Communication ?



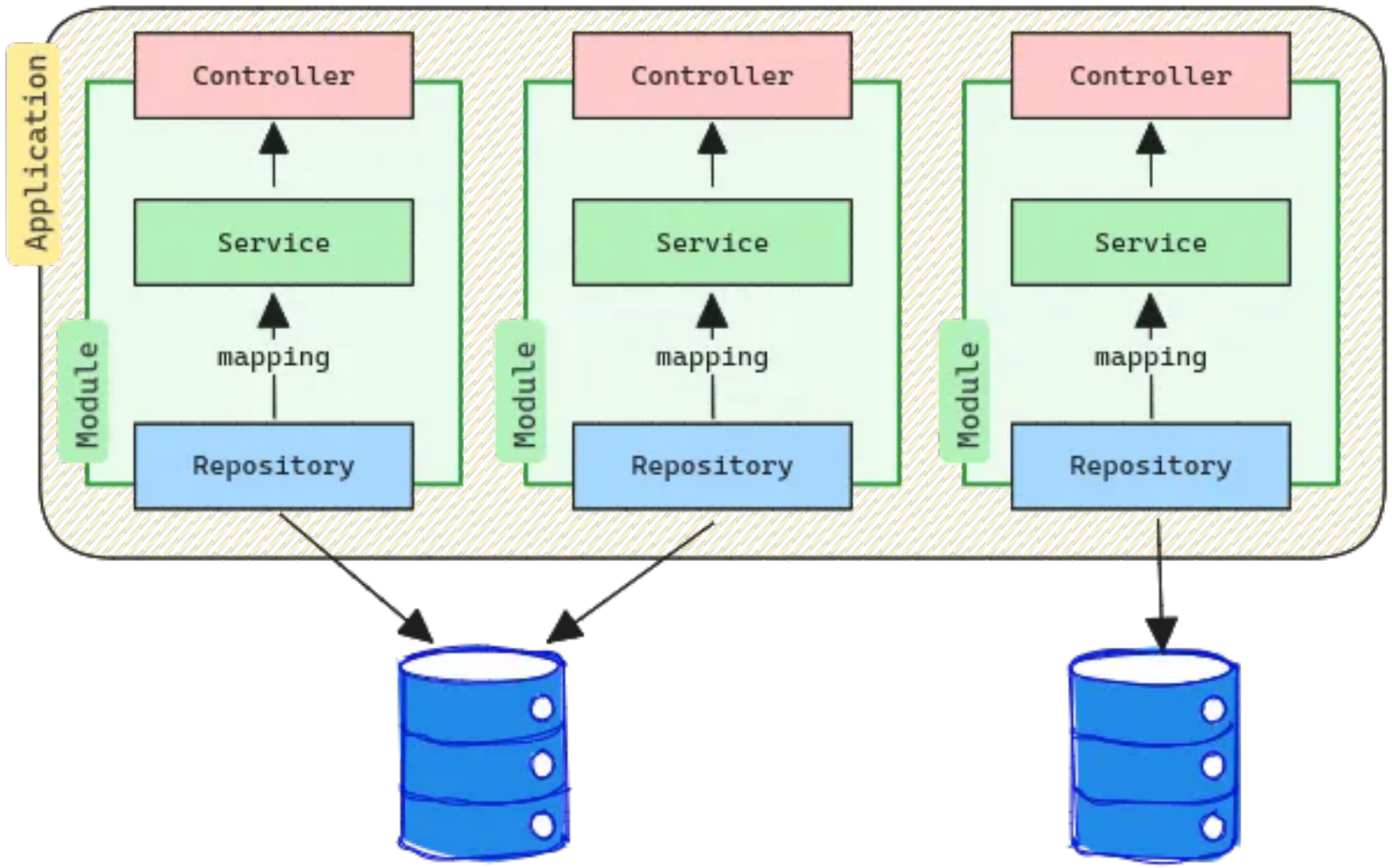
https://milanjovanovic.tech/blog/modular-monolith-communication-patterns

Modular Monolith



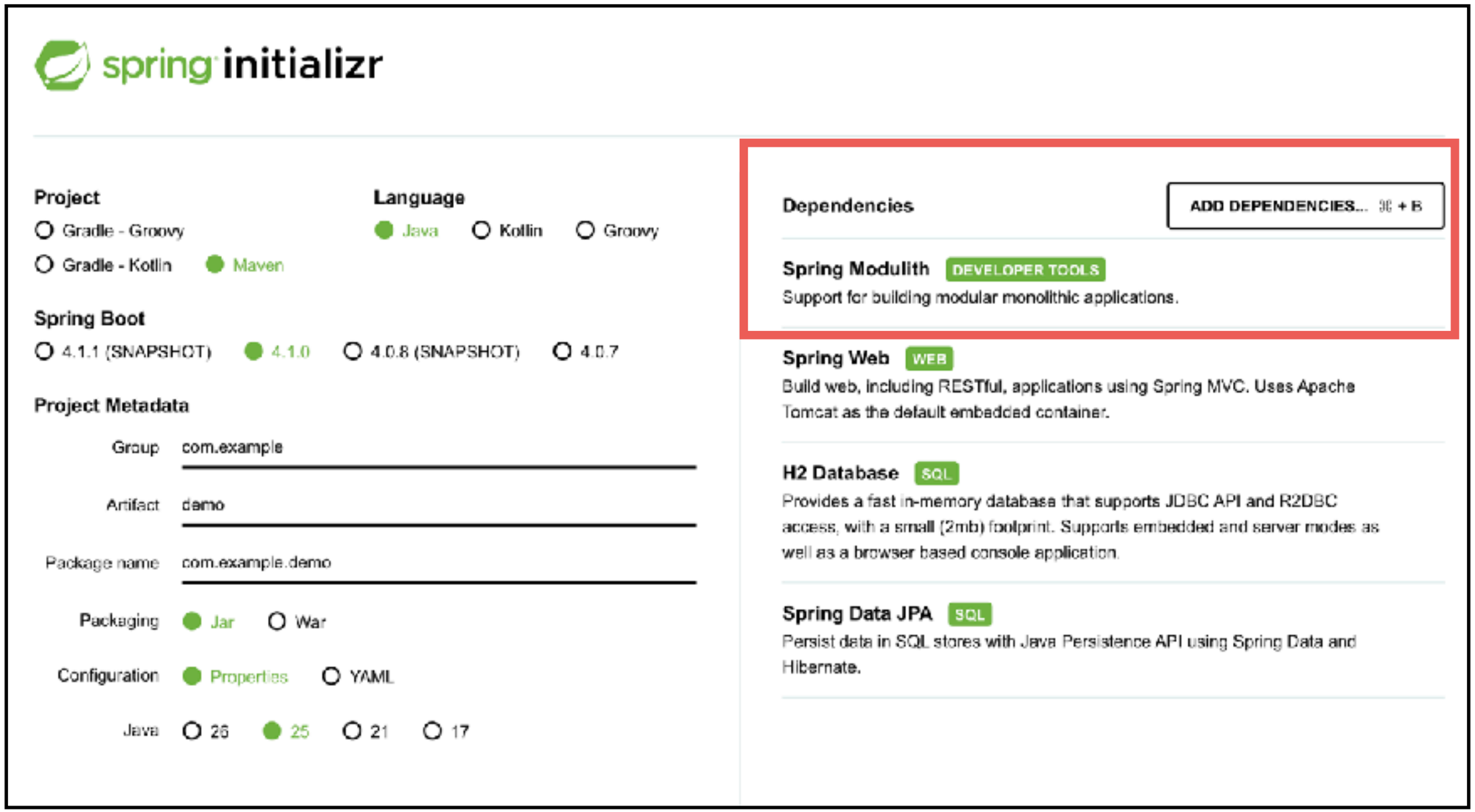
https://learn.microsoft.com/th-th/dotnet/architecture/cloud-native/introduction

Spring Modulith

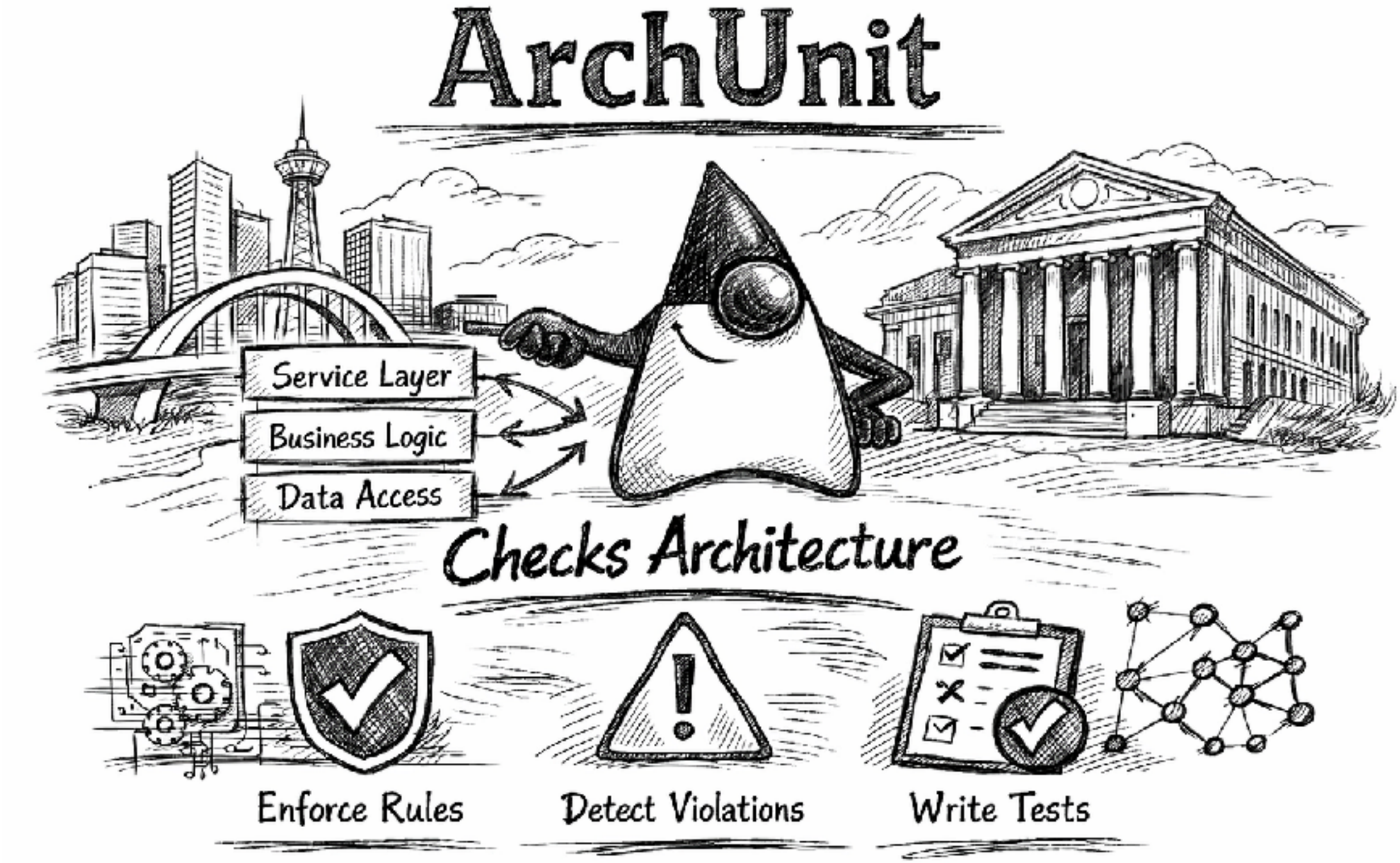


https://spring.io/projects/spring-modulith

Spring Modulith

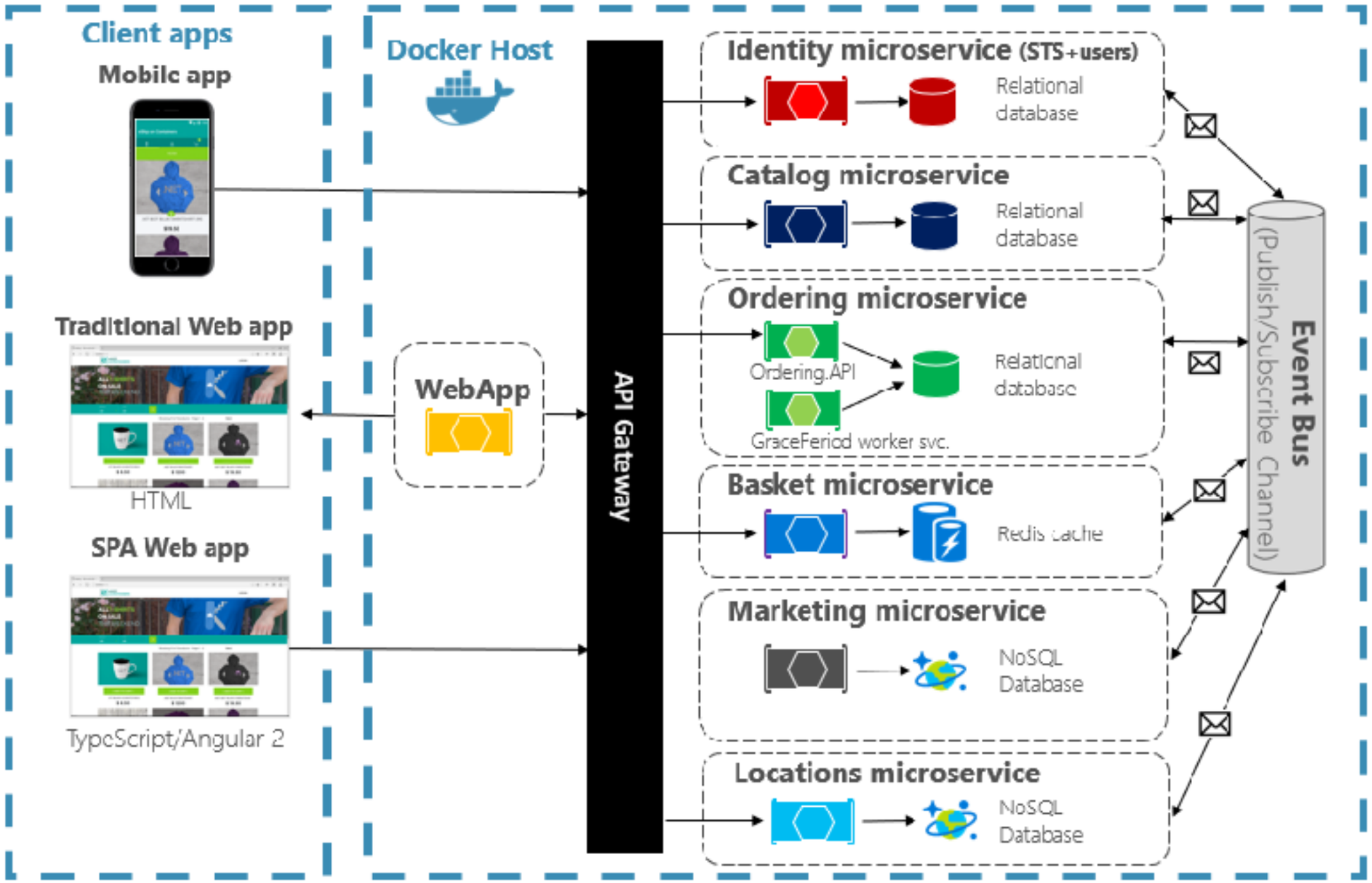


https://start.spring.io/



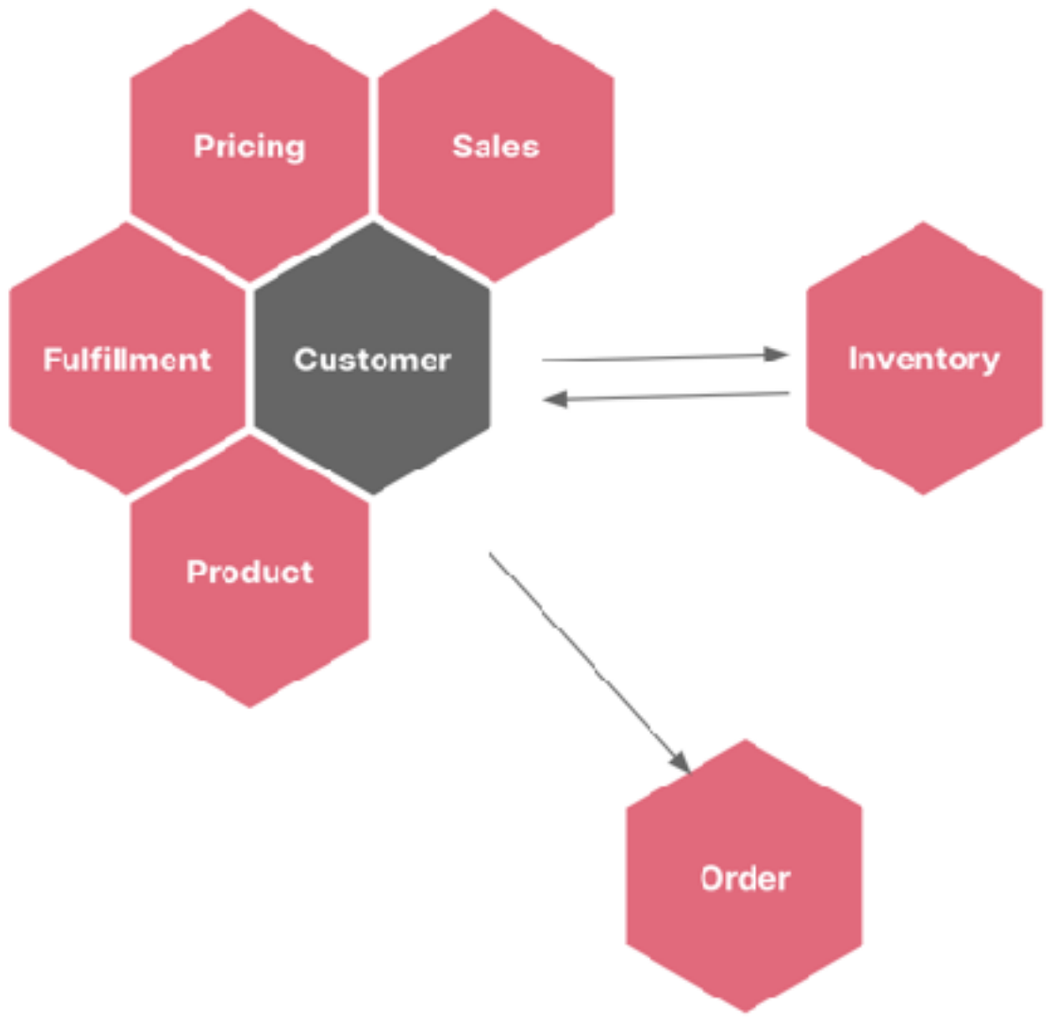
https://www.archunit.org/

Microservices



https://learn.microsoft.com/th-th/dotnet/architecture/cloud-native/introduction

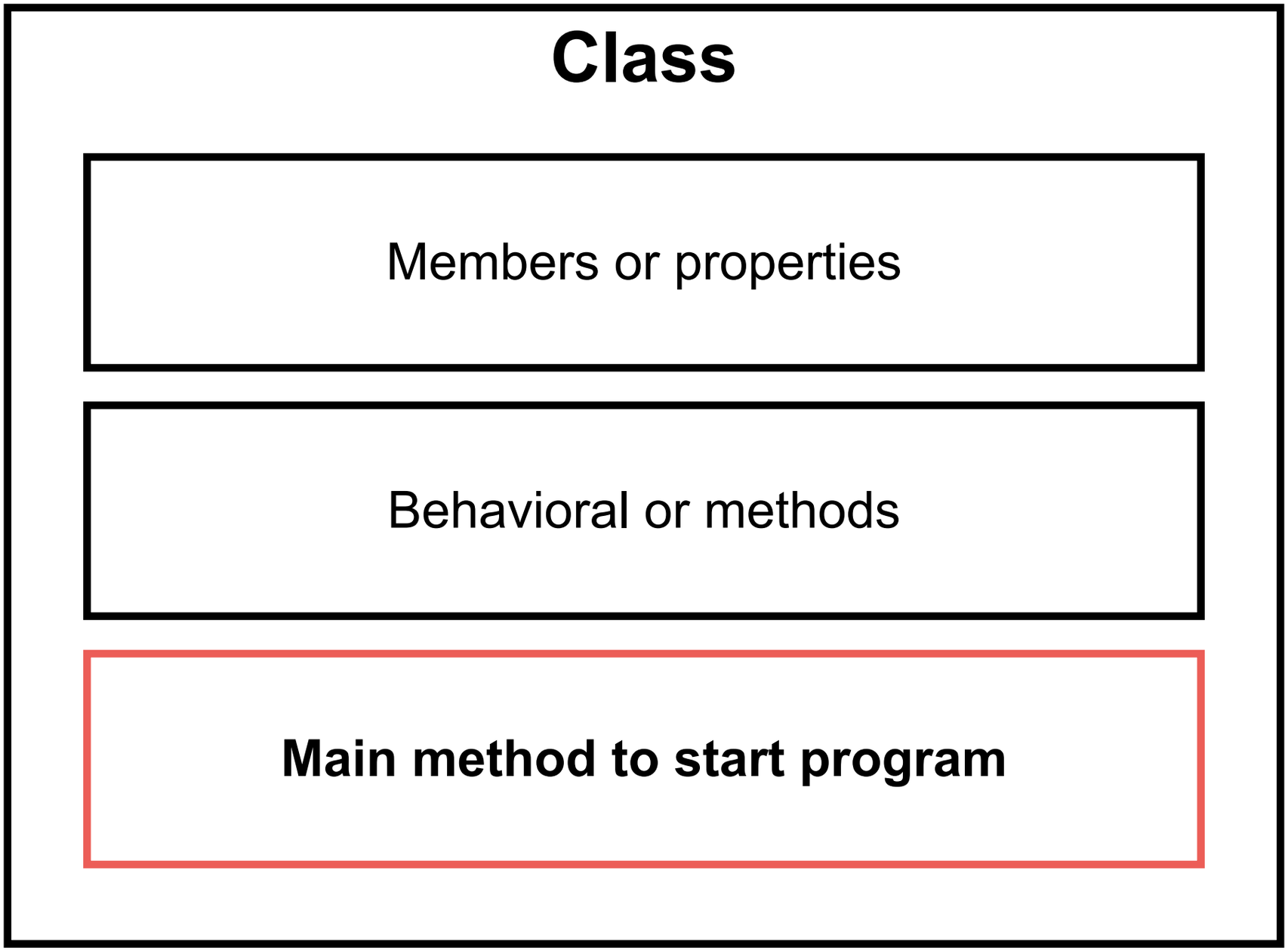
Split from module to service



https://www.thoughtworks.com/en-us/insights/blog/microservices/modular-monolith-better-way-build-software

Object-Oriented Programming

Structure of Java program



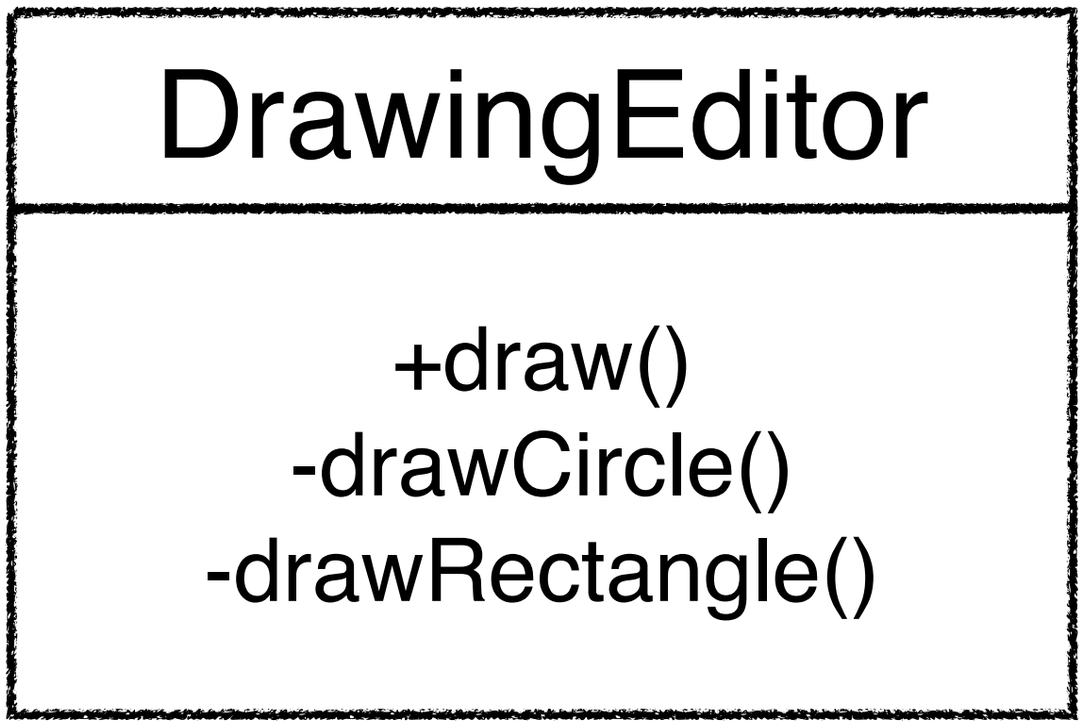
Thinking before create class

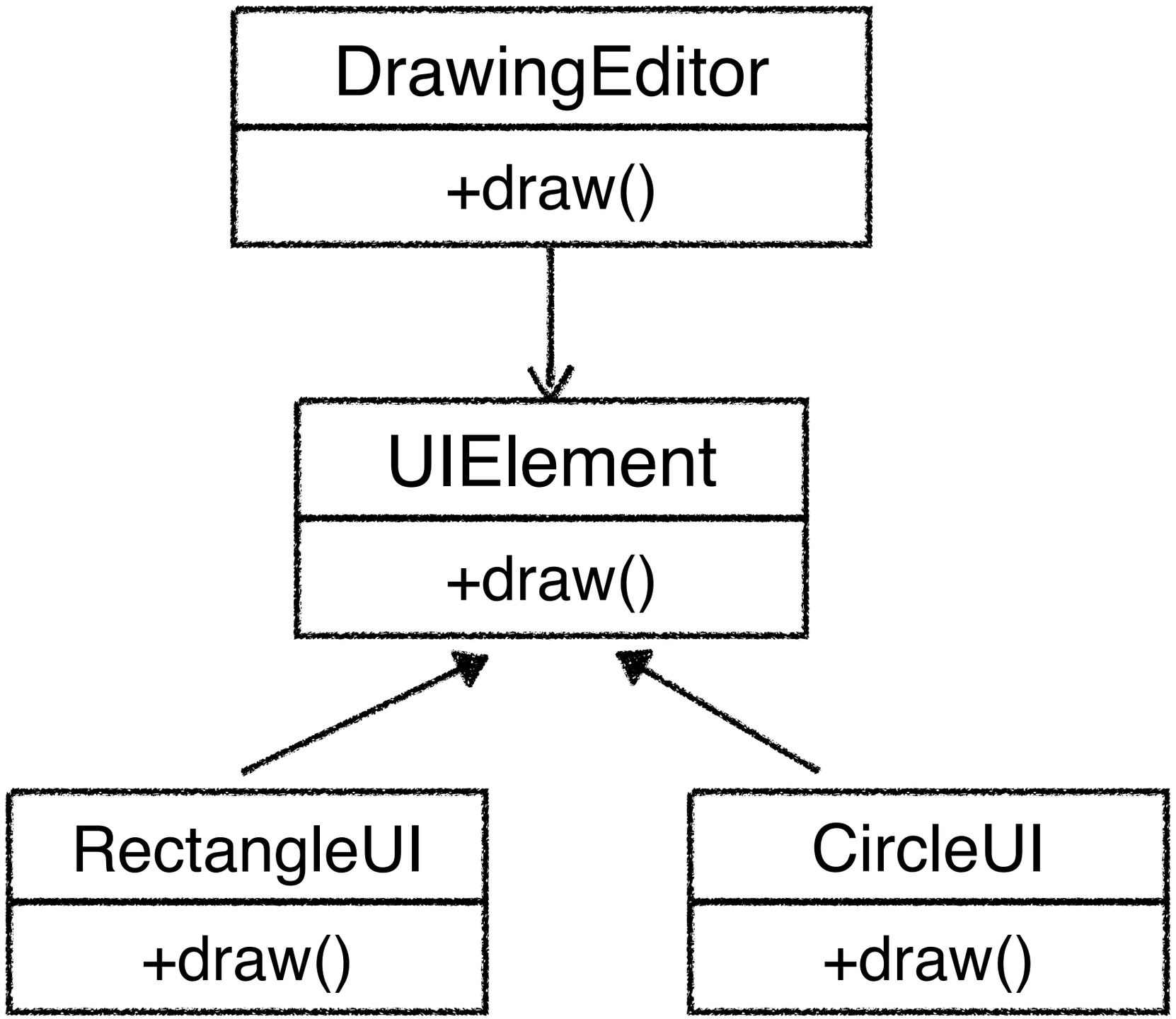
Thinking before create class

Single Responsibility Principle

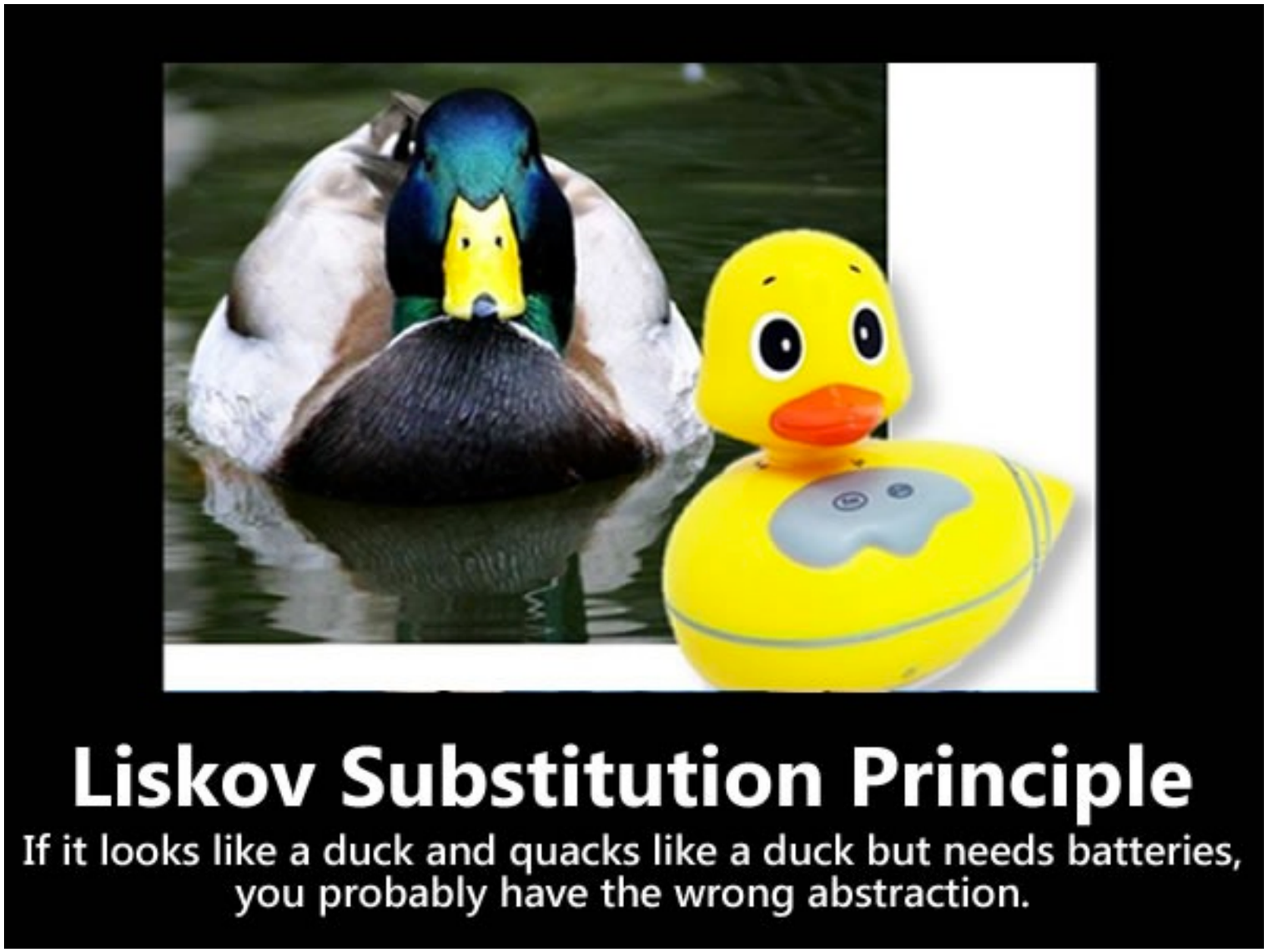
Open-Closed Principle

Example





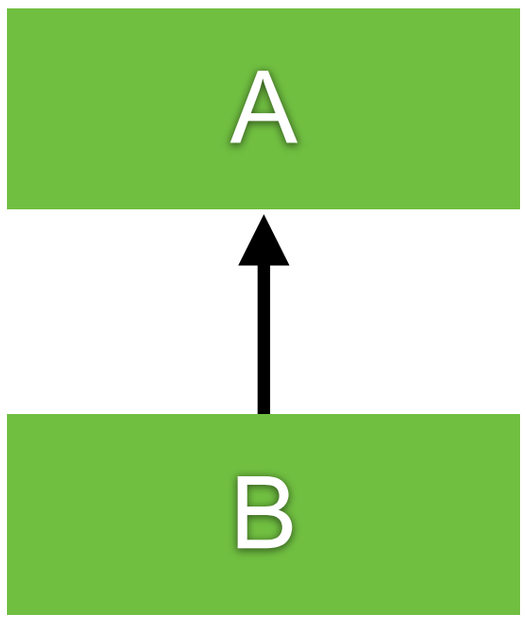
Liskov Substitution Principle



Inheritance

Ability to derive a new class from existing class

superclass / base class



subclass / derived class

Additional properties and behaviors

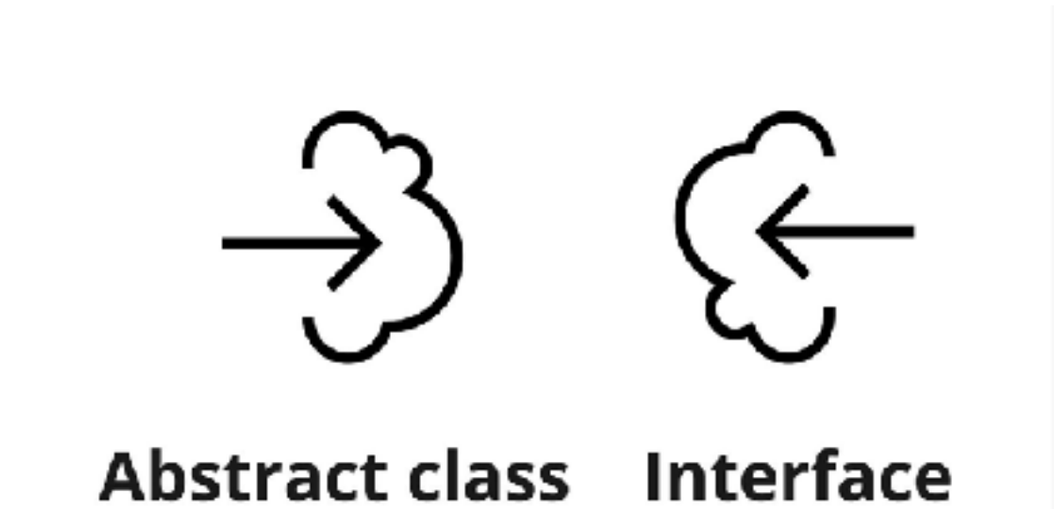
How to create ?

Extends from base class only one class

```
class BaseClass {

}

class SubClass extends BaseClass {

}
```

More OOP with Java

Abstract classes
Interfaces



Abstract classes

Placeholder for a method that fully defined in a subclass

Class have abstract modifier

Method can have abstract modifier

Abstract method can’t be private

Abstract method can’t have body

“Class that contains an abstract method is called ***Abstract class”***

Classes

“Class that contains an abstract method is called ***Abstract class”***

“Class that not contains an abstract method is called ***Concrete class”***

Abstract classes

```
abstract class Employee {

    private int id;

    public abstract double getSalary();

    public boolean sameSalary(Employee other) {
        return this.getSalary() == other.getSalary();
    }

}
```

Pitfall of abstract class

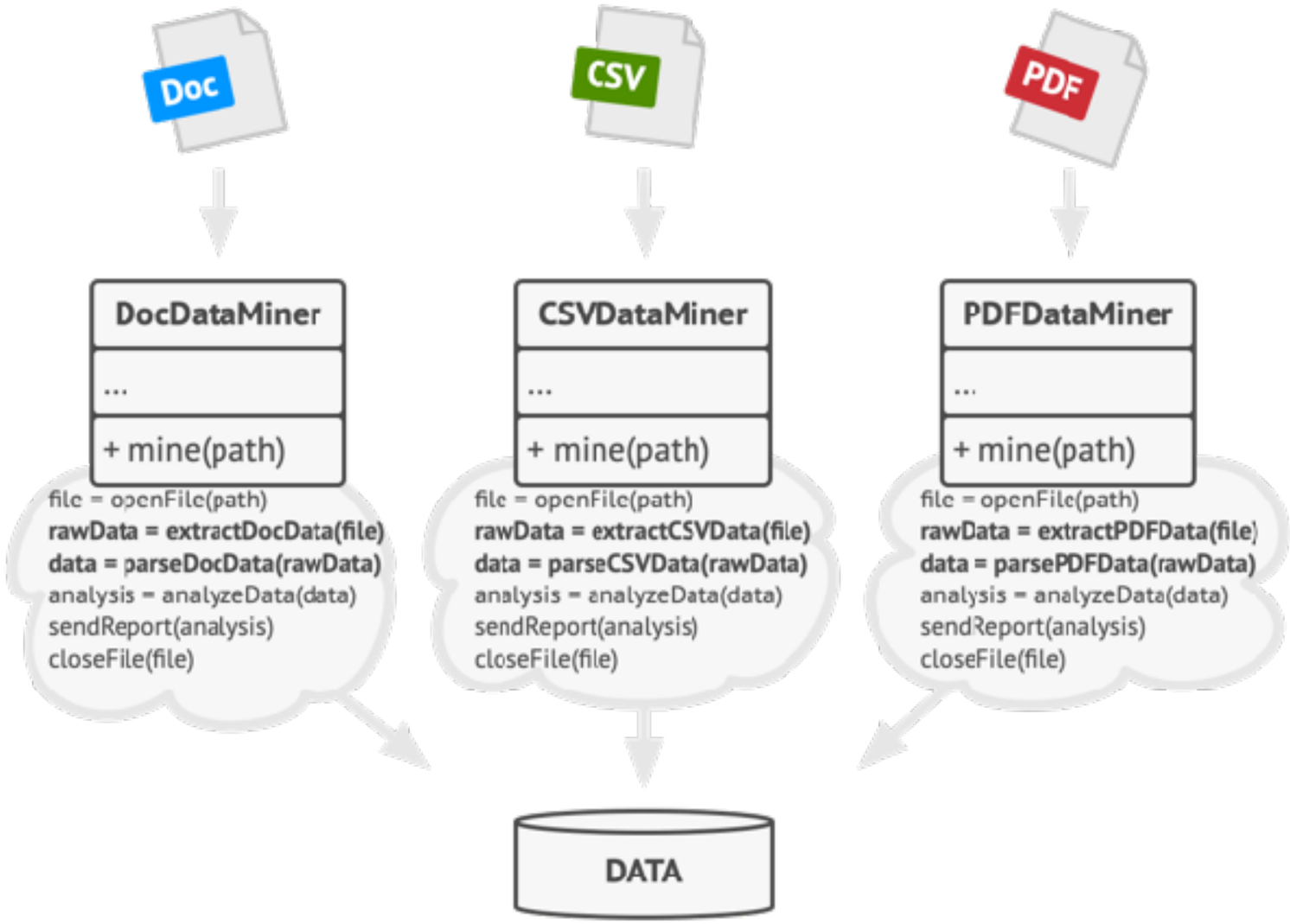
You can’t create instances of an abstract class
Abstract class can only use to derive more classes

Purpose of abstract class

Create a function as base class
But can be extended by sub class

```
abstract class Report {
    public abstract void createHeader();
    public abstract void createBody();
    public abstract void createFooter();

    public void generate() {
        createHeader();
        createBody();
        createFooter();
    }
}
```

Template Method Pattern



https://refactoring.guru/design-patterns/template-method

Interfaces

Like an extreme abstract class
All of methods in an interface are abstract by default
Not a class
No instance variables

```
interface Animal {
    void eat();
    void sleep();
}
```

Using interface from class (1)

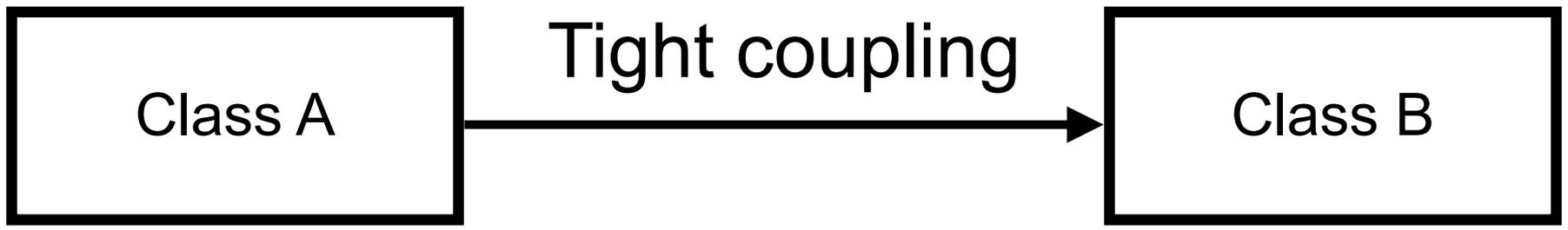
```
class Cat implements Animal {

    @Override
    public void eat() {
    }

    @Override
    public void sleep() {
    }

}
```

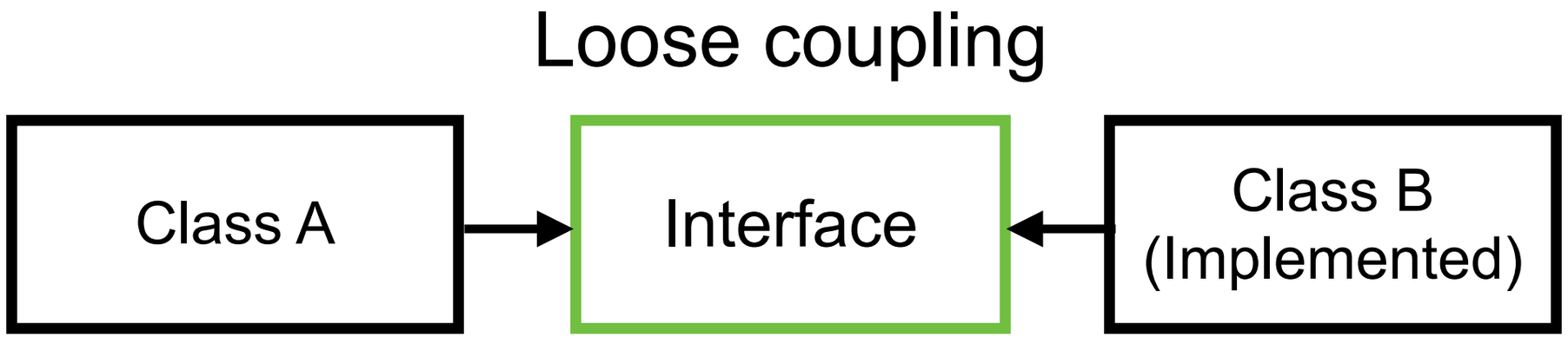
Using interface from class (2)

```
class Cat implements Animal {

    @Override
    public void eat() {
    }


    @Override
    public void sleep() {
    }
}
```

Need to implements all method from interface

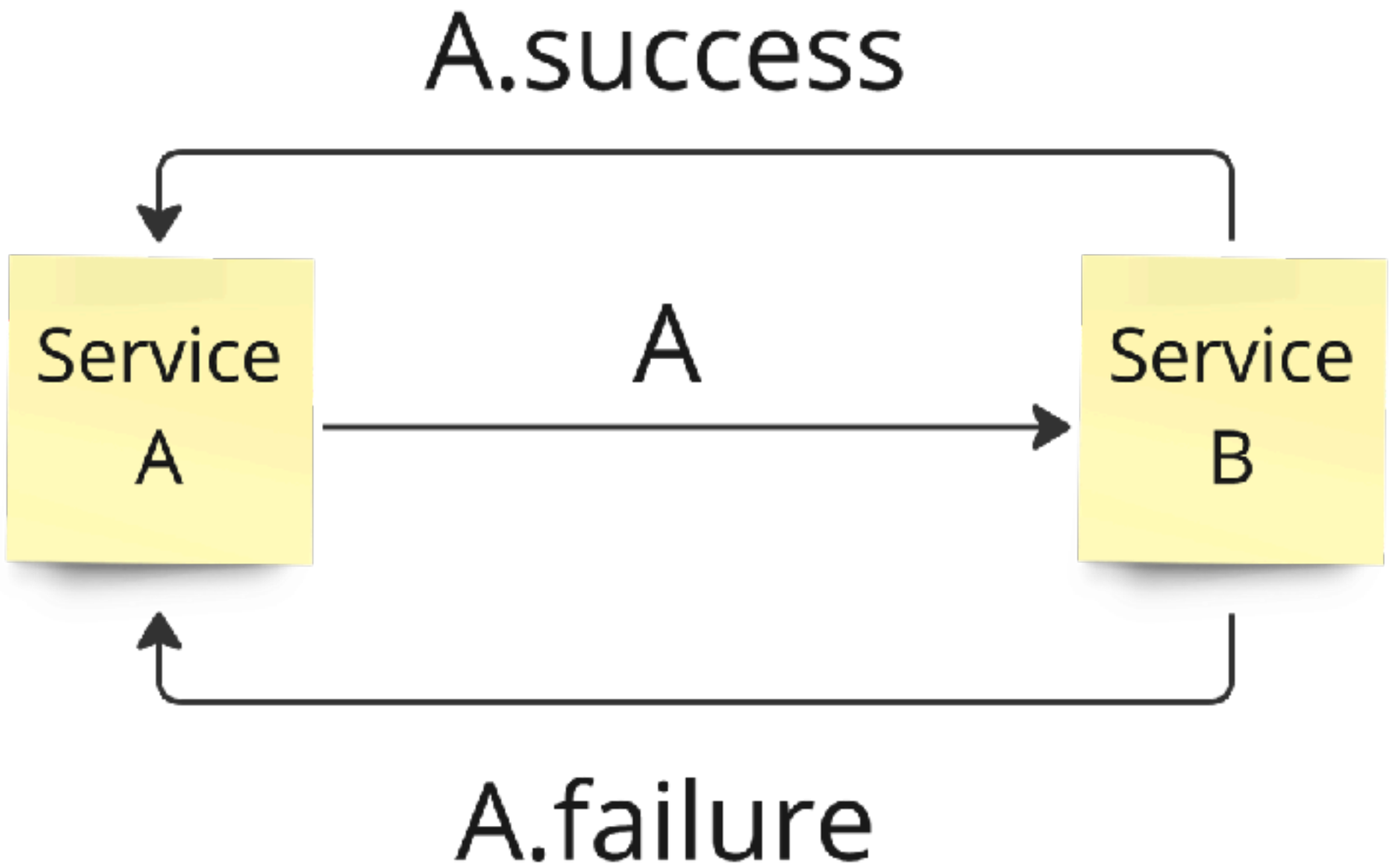
When to use interface ?



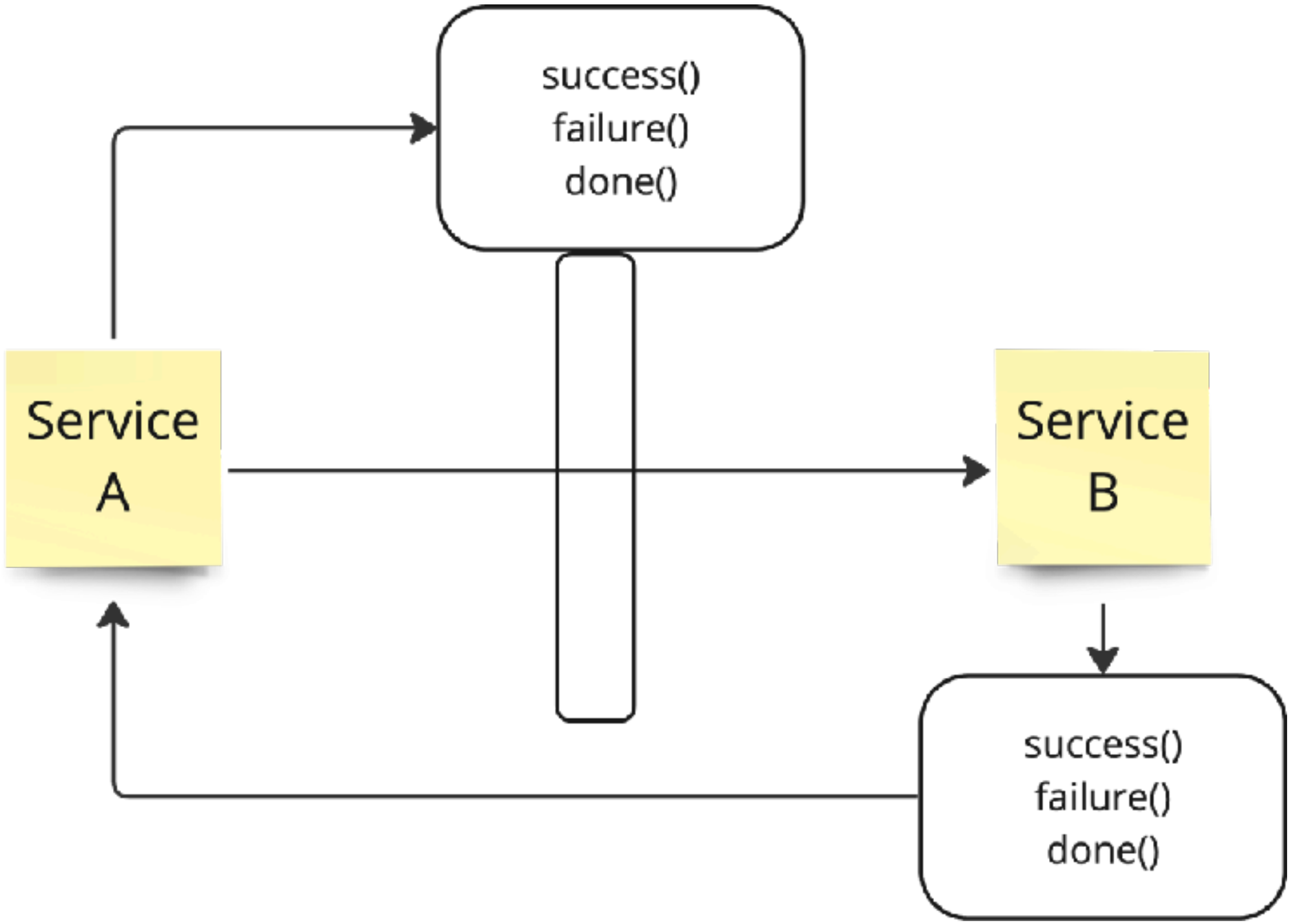


Callback with Interface

Call back with class (1)

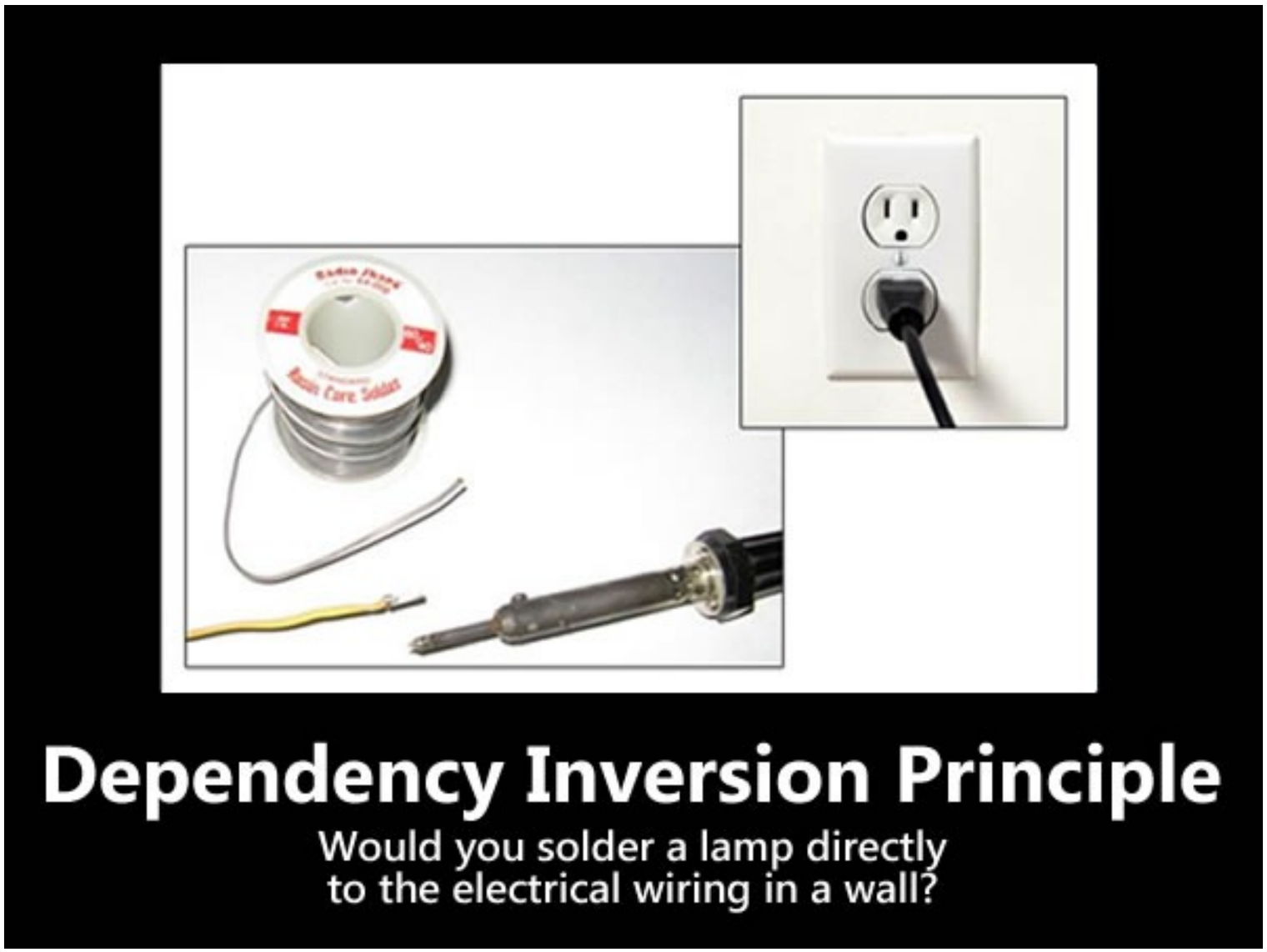


Call back with interface (2)

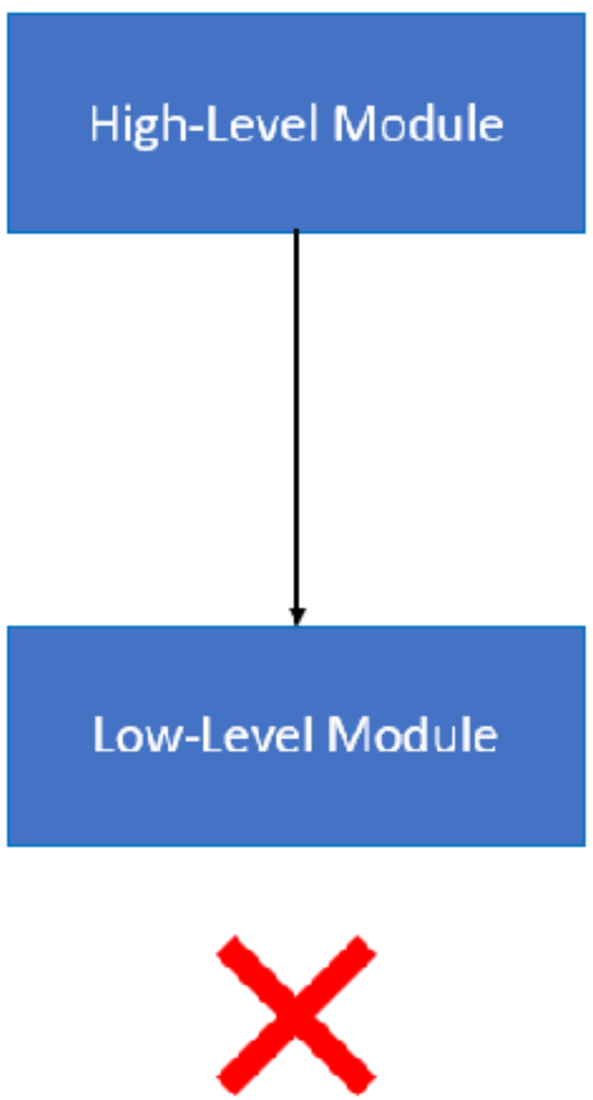


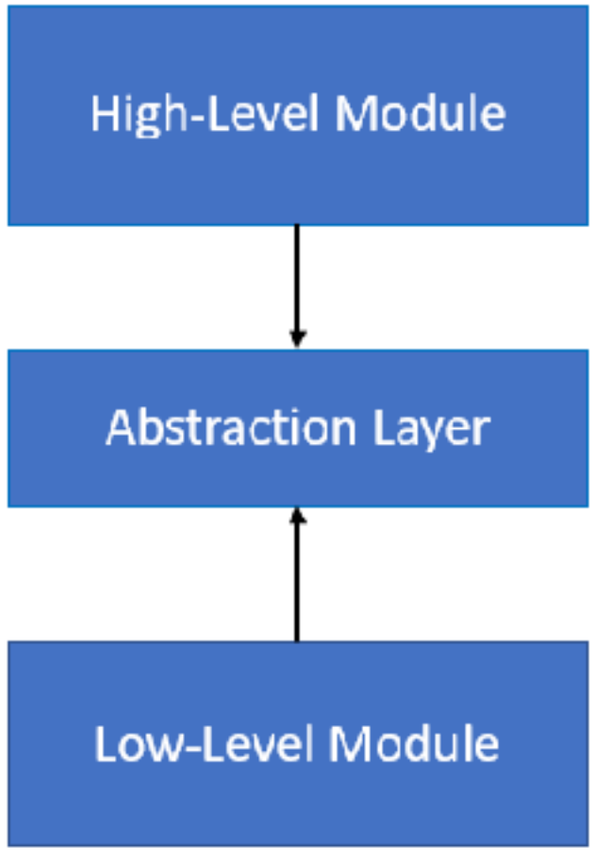
Interface Segregation Principle

Dependency Inversion Principle



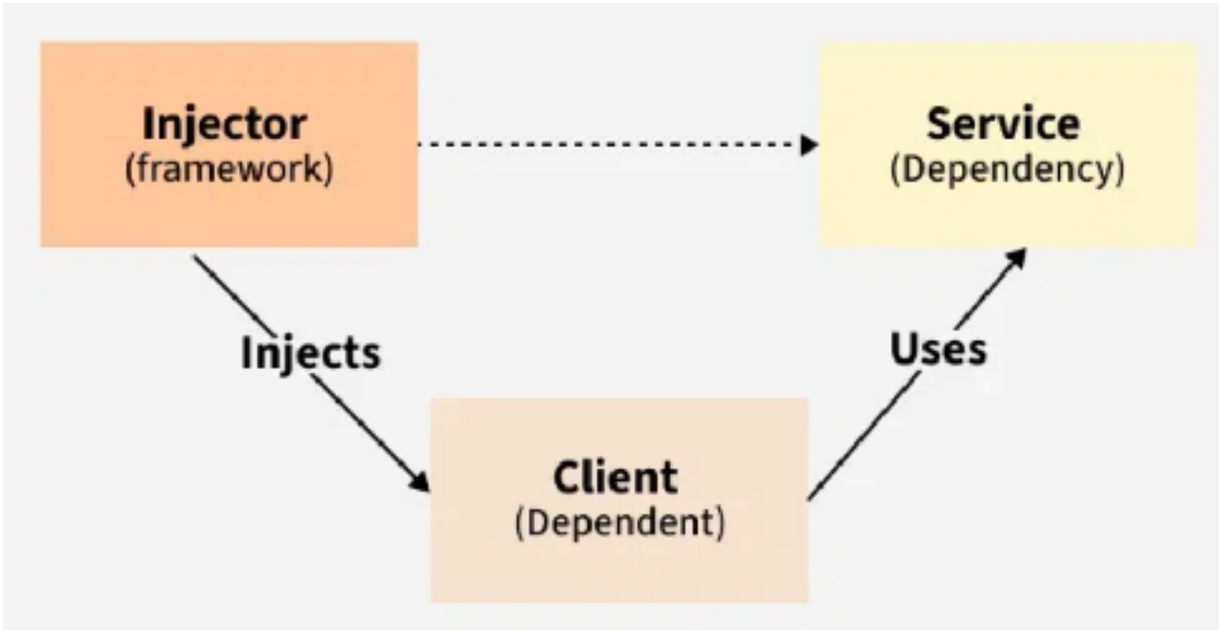
Depend on abstraction





Dependency Injection

Property injection
Constructor injection
Method injection
Interface injection



Modern Java Features

https://github.com/up1/course-java-spring-2026/wiki/Workshop-with-Modern-Java-Features

Modern Java Features

Generic
Seal class
Virtual thread
Stream API with Collection framework
Lambda
Record

Java Generic

Problem: Duplicate Code and Logic in Java

Use Java Generics to Solve

PROBLEM: Duplicate Code and Logic

When we handle multiple types (e.g., User, Product, Order), we end up writing the same CRUD code over and over again.

UserService.java

```
public class UserService {
    private List<User> users =
        new ArrayList<>();

public void add(User user) {
    users.add(user);
}

public User getById(Long id) {
    for (User u : users) {
        if (u.getId().equals(id))
            return u;
    }
    return null;
}

public List<User> getAll() {
    return users;
}

public void delete(Long id) {
    users.removeIf(u ->
        u.getId().equals(id));
    }
}
```

ProductService.java

```
public class ProductService {
    private List<Product> products=
        new ArrayList<>();

public void add(Product p) {
    products.add(p);
}

public Product getById(Long id) {
    for (Product p : products) {
        if (p.getId().equals(id))
            return p;
    }
    return null;
}

public List<Product> getAll() {
    return products;
}

public void delete(long id) {
    products.removeIf(p ->
        p.getId().equals(id));
    }
}
```

OrderService.java

```
public class OrderService {
    private List<Order> orders =
        new ArrayList<>();

public void add(Order o) {
    orders.add(o);
}

public Order getById(long id) {
    for (Order o : orders) {
        if (o.getId().equals(id))
            return o;
    }
    return null;
}

public List<Order> getAll() {
    return orders;
}

public void delete(Long id) {
    orders.removeIf(o ->
        o.getId().equals(id));
    }
}
```

What's the problem?

- Same CRUD logic is written 3 times (or more as the project grows)
- Hard to maintain – fix a bug in one place, easy to miss others
- More code = more risk
- Violates DRY (Don't Repeat Yourself) principle

SOLUTION: Use Java Generics

Create a generic service that works with any type T.
No more duplicate code!

GenericService.java

```
public interface Identifiable<ID> {
    ID getId();
}

public class GenericService<T extends Identifiable<ID>, ID> {
    private final List<T> items = new ArrayList<>();

    public void add(T item) {
        items.add(item);
    }

    public T getById(ID id) {
        for (T item : items) {
            if (item.getId().equals(id))
                return item;
        }
        return null;
    }

    public List<T> getAll() {
        return new ArrayList<>(items);
    }

    public void delete(ID id) {
        items.removeIf(i -> i.getId().equals(id));
    }
}
```

Benefits

- ✓ No duplicate code
- ✓ Easy to maintain
- ✓ Type-safe
- ✓ Reusable
- ✓ Follows DRY principle

Usage

```
GenericService<User, Long> userService = new GenericService<>();
GenericService<Product, Long> productService = new GenericService<>();
GenericService<Order, Long> orderService = new GenericService<>();

// Works for all types!
userService.add(new User(1L, "Alice"));
productService.add(new Product(10L, "Laptop"));
orderService.add(new Order(100L, "Order#100"));

User u = userService.getById(1L);
Product p = productService.getById(10L);
Order o = orderService.getById(100L);
```

★ One generic solution. Many types. Zero duplication.

Key Takeaway

Use Java Generics to write flexible, reusable, and type-safe code.
Less code. Fewer bugs. Easier maintenance.

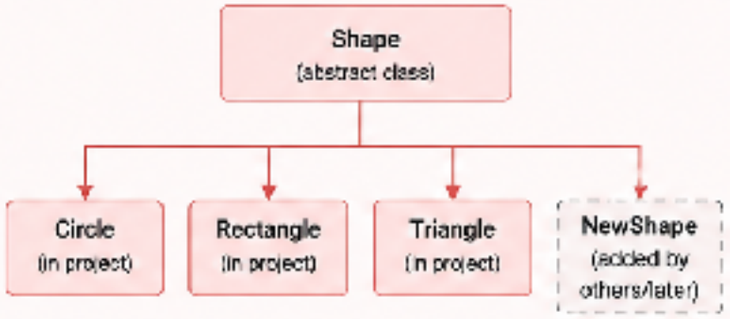
Seal classes

Problem: Inheritance Hierarchy in Java

Use Sealed Classes to Solve

PROBLEM

Uncontrolled inheritance leads to fragile code, missing cases, and runtime errors



- ✕ Any developer can add new subclasses anywhere
- ✕ Compiler can't verify exhaustiveness
- ✕ Switch / if-else can miss cases → runtime bugs
- ✕ Harder to maintain and refactor
- ✕ Violates closed design principle

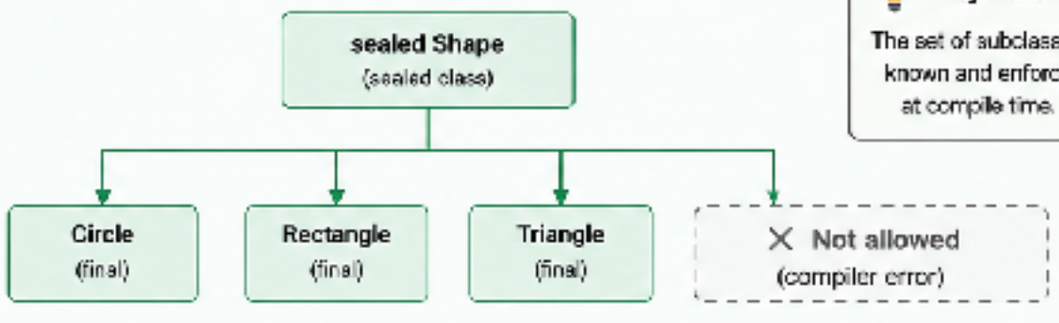
```
// Example: Missing case leads to bug
static double area(Shape s) {
    return switch (s) {
        case Circle c -> Math.PI * c.radius() * c.radius();
        case Rectangle r -> r.width() * r.height();
        // Forgot Triangle and NewShape!
    };
}
```

At runtime: IllegalStateException or wrong result

SOLUTION: Use Sealed Classes (Java 17+)

Define a closed set of subclasses.
The compiler helps you handle all cases.

Key Benefit
The set of subclasses is known and enforced at compile time.



- ✓ Only permitted subclasses can exist (same package/module)
- ✓ Compiler checks exhaustiveness in switch / if-else
- ✓ Safer refactoring and easier evolution
- ✓ Follows closed design principle
- ✓ Better documentation of domain model

```
// Sealed class and permitted subclasses
public sealed abstract class Shape
      permits Circle, Rectangle, Triangle { }
public final class Circle extends Shape {
    public double radius() { ... }
}
public final class Rectangle extends Shape { ... }
public final class Triangle extends Shape { ... }
```

```
// Exhaustive switch - compiler checked
static double area(Shape s) {
    return switch (s) {
        case Circle c -> Math.PI * c.radius() * c.radius();
        case Rectangle r -> r.width() * r.height();
        case Triangle t -> 0.5 * t.base() * t.height();
    }; // Compiler ensures all cases are handled
}
```

Compile-time safety, fewer bugs, clearer design

Best for domain models with a fixed set of types (e.g., payments, results, events, shapes, states).
Use sealed classes + records to build robust, expressive, and maintainable Java code.

Java Collection Framework

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/doc-files/coll-overview.html

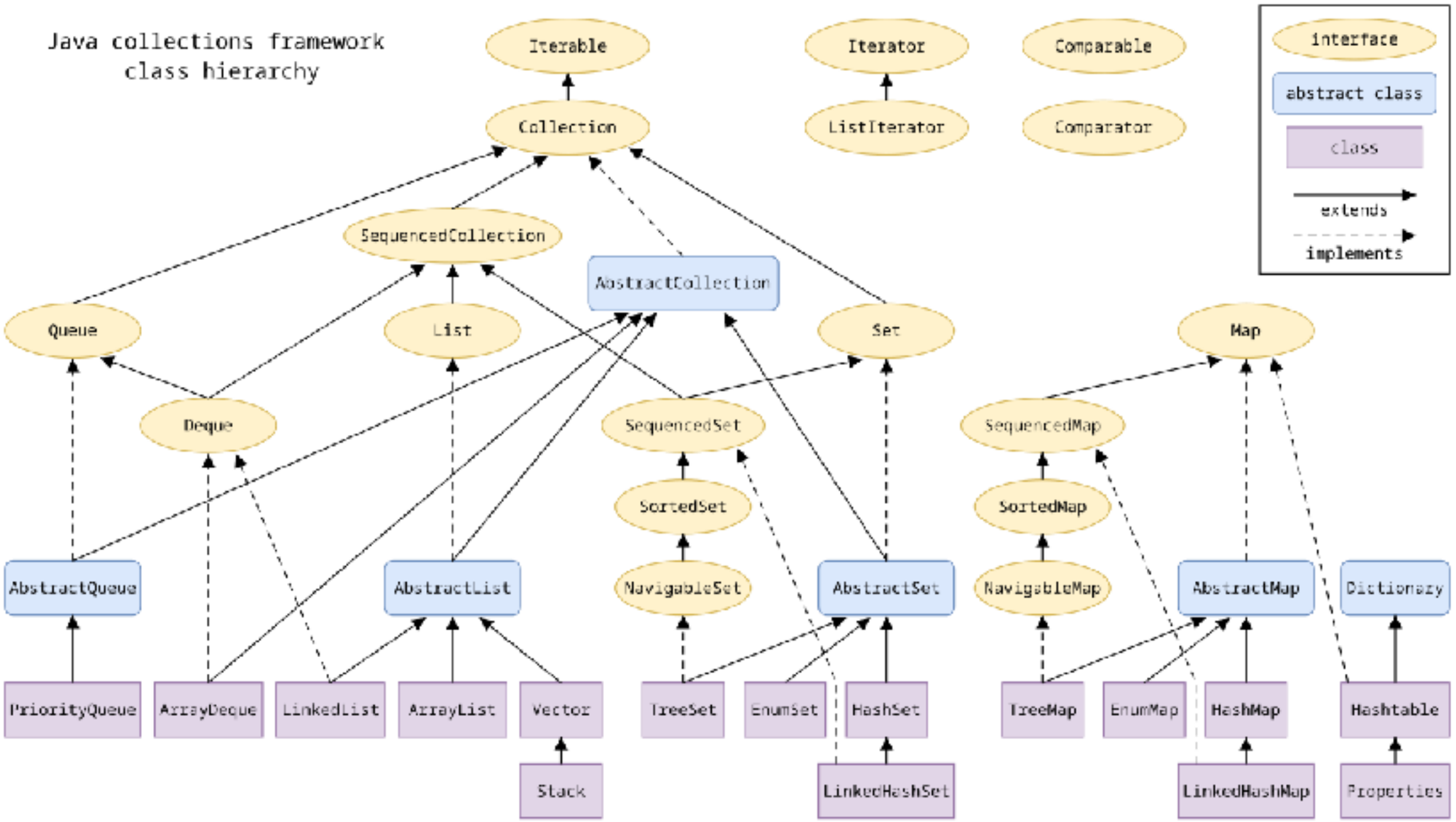
Java Collection Framework

General purpose implementations

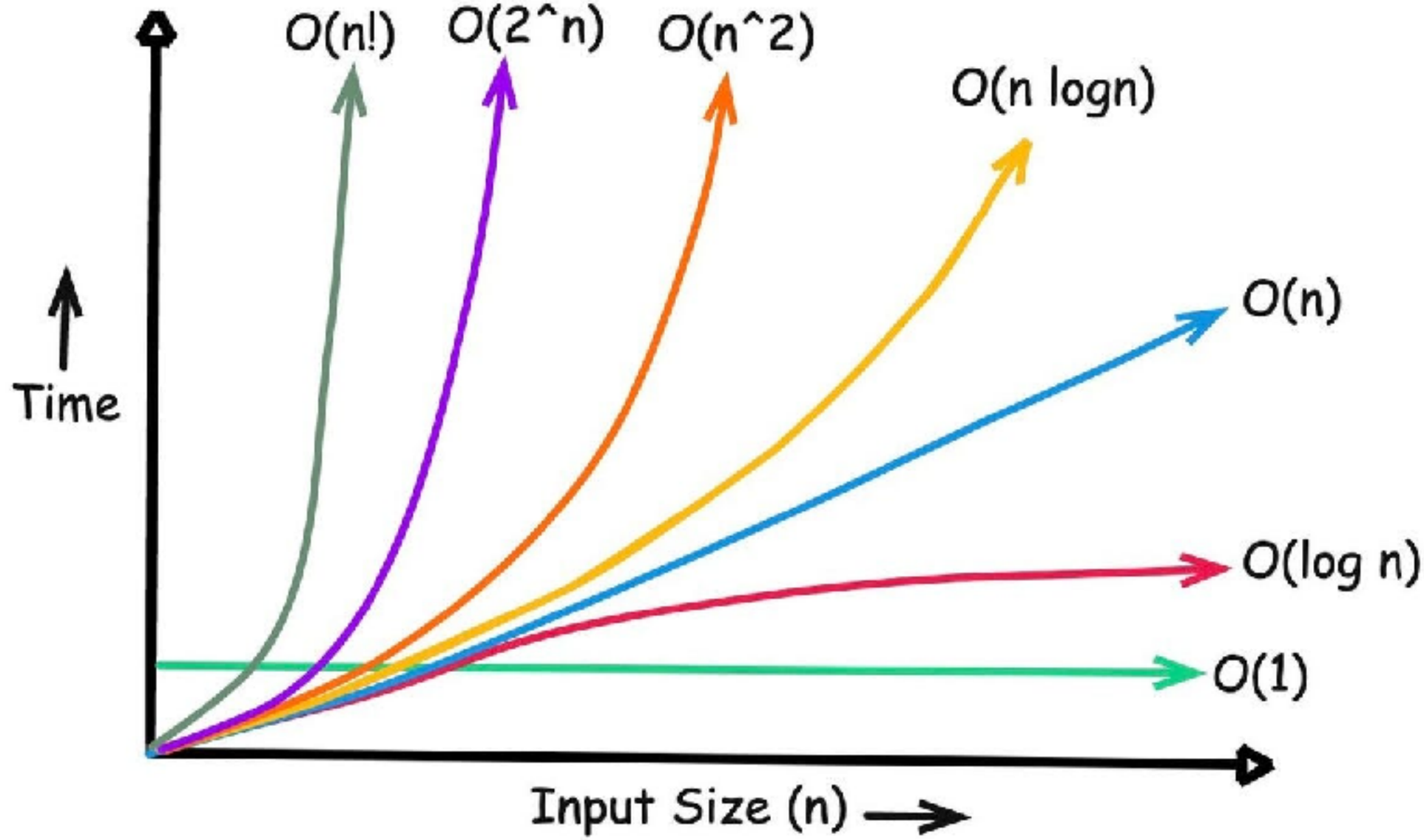
Interface	Hash Table	Resizable Array	Balanced Tree	Linked List	Hash Table + Linked List
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Queue, Deque		ArrayDeque		LinkedList	
Map	HashMap		TreeMap		LinkedHashMap

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/doc-files/coll-overview.html

Java Collection Framework



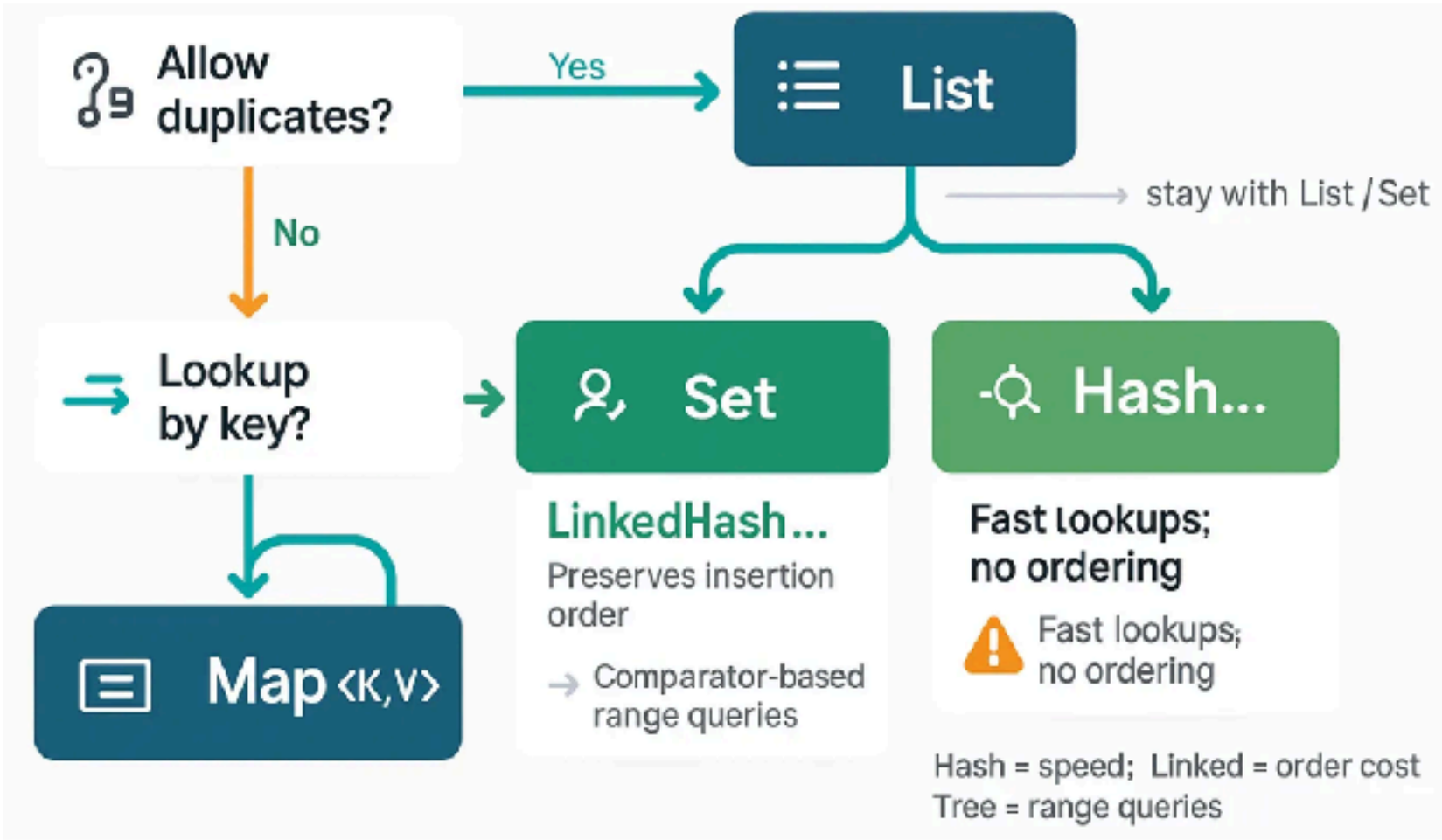
Big O Notation



Working with use cases

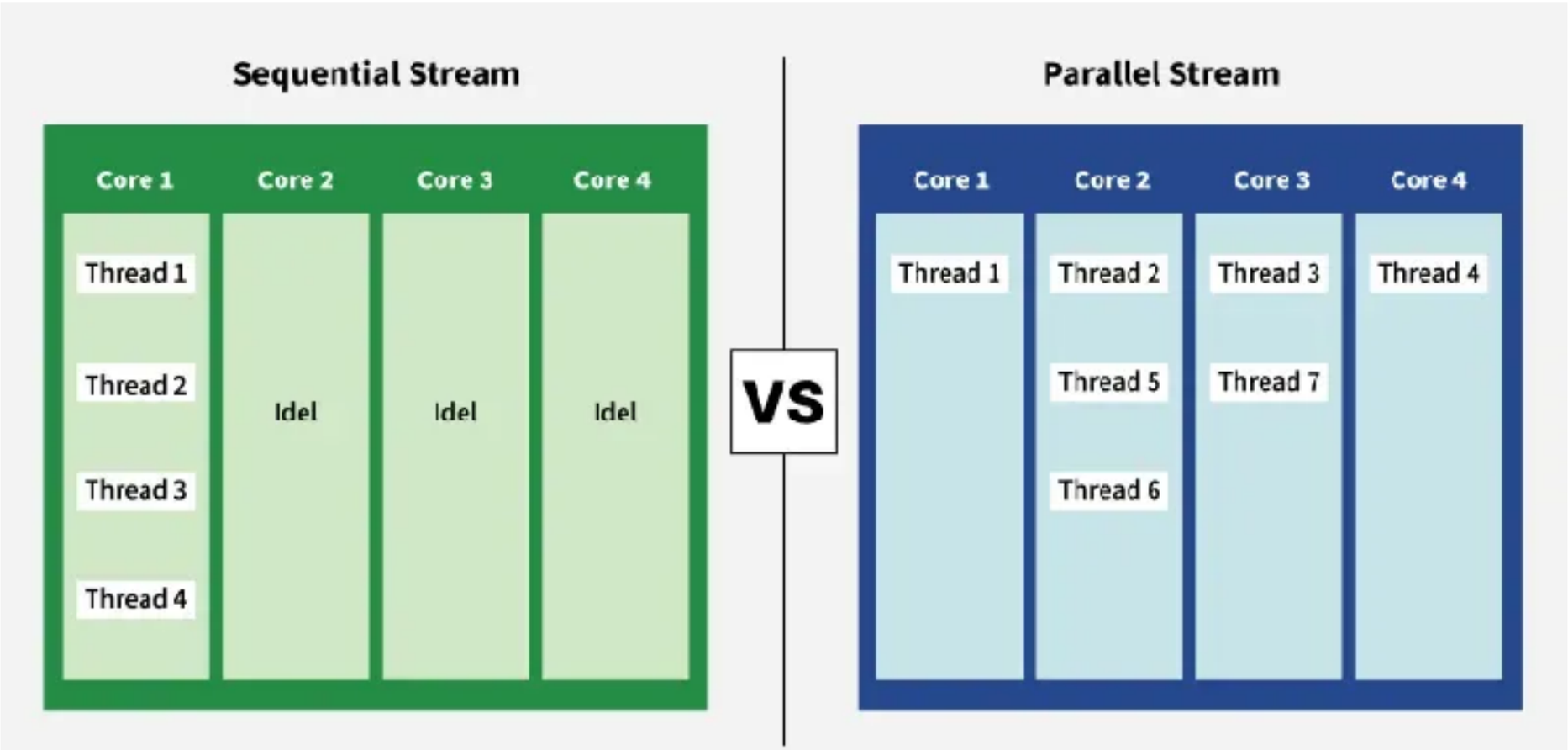
List vs Map

Lookup optimization



Java Stream

Sequencial vs Parallel



https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/stream/Stream.html

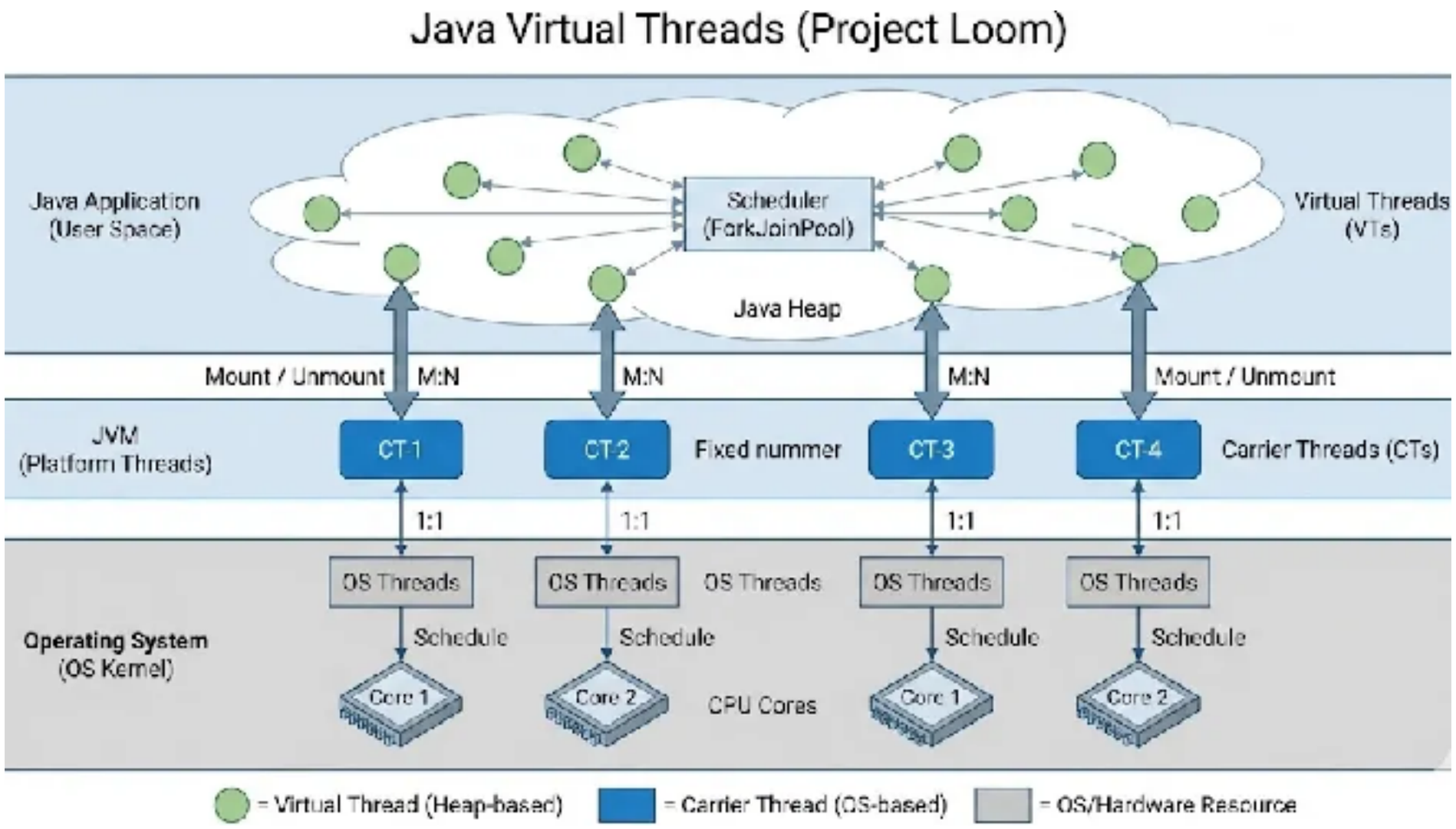
Virtual Thread

https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html

Java Virtual Threads (Project Loom)

Java 21

Lightweight, JVM-managed thread



https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html

Java Virtual Threads (Project Loom)

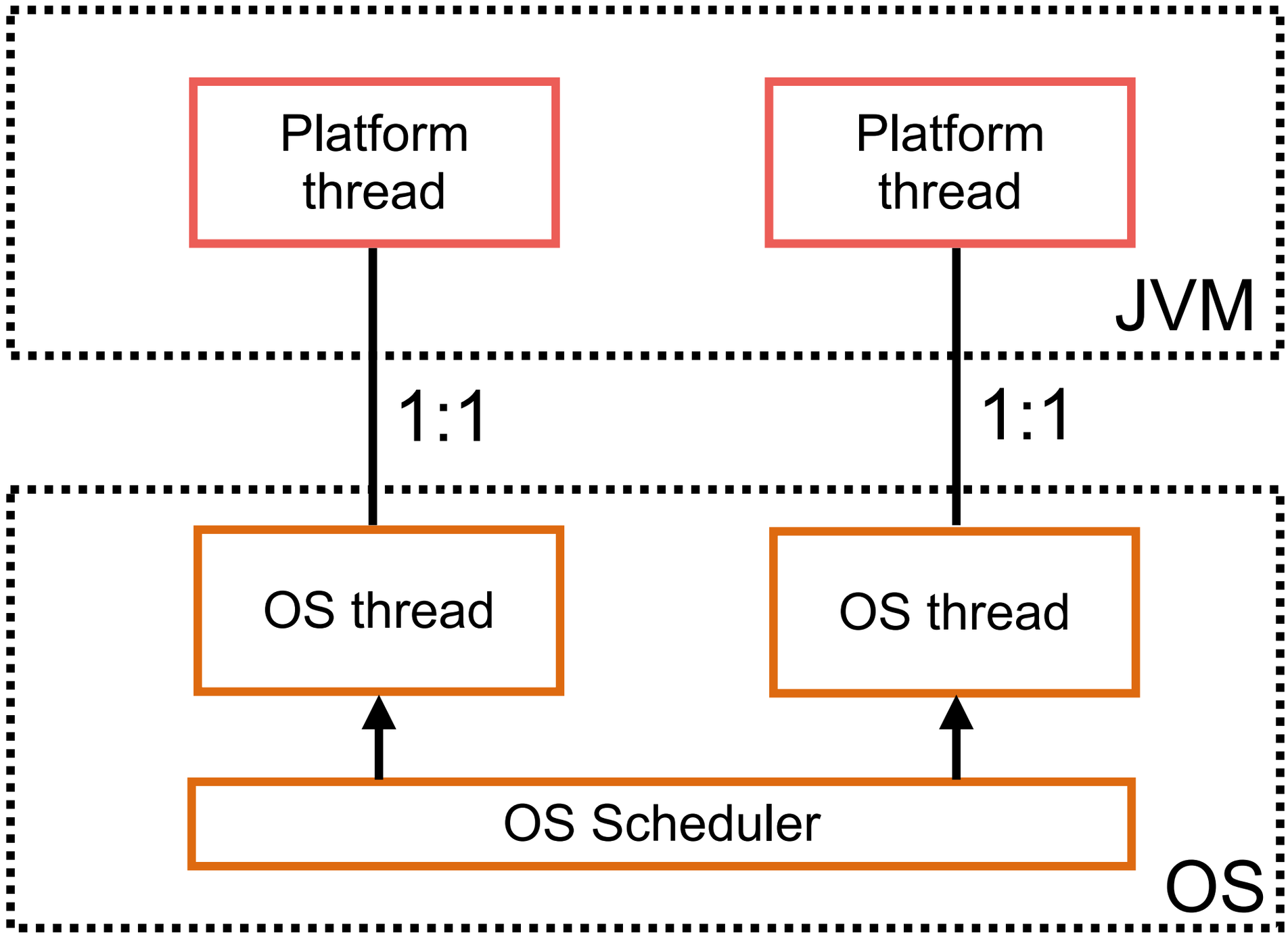
Java 21
Lightweight, JVM-managed thread
Parked and resumed without blocking OS threads
No thread pool management
Efficient context switching
Utilized resources

https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html

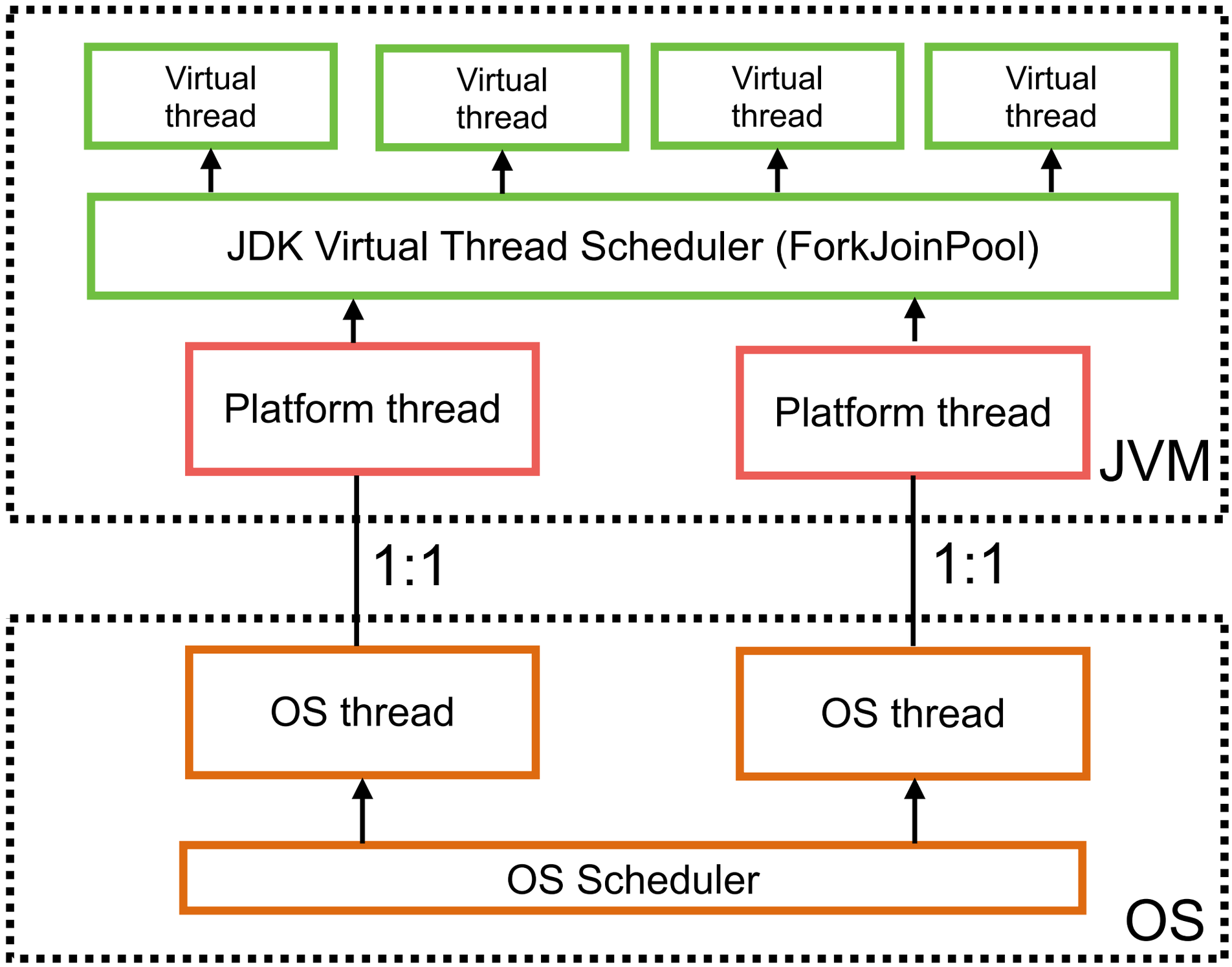
When to use Virtual Thread ?

I/O bound application
High-concurrency workload
Simplify asynchronous programming

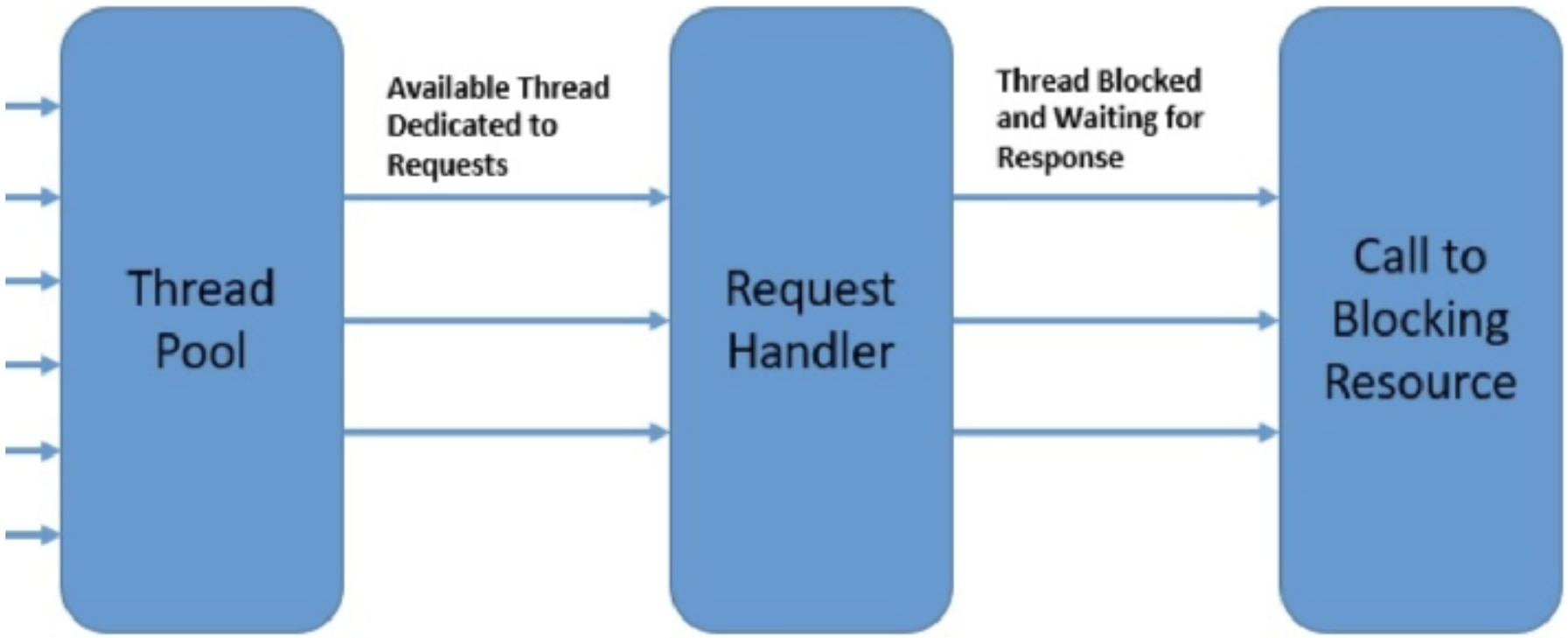
Java Platform Thread



Virtual Thread



Spring Boot with Virtual Thread !!

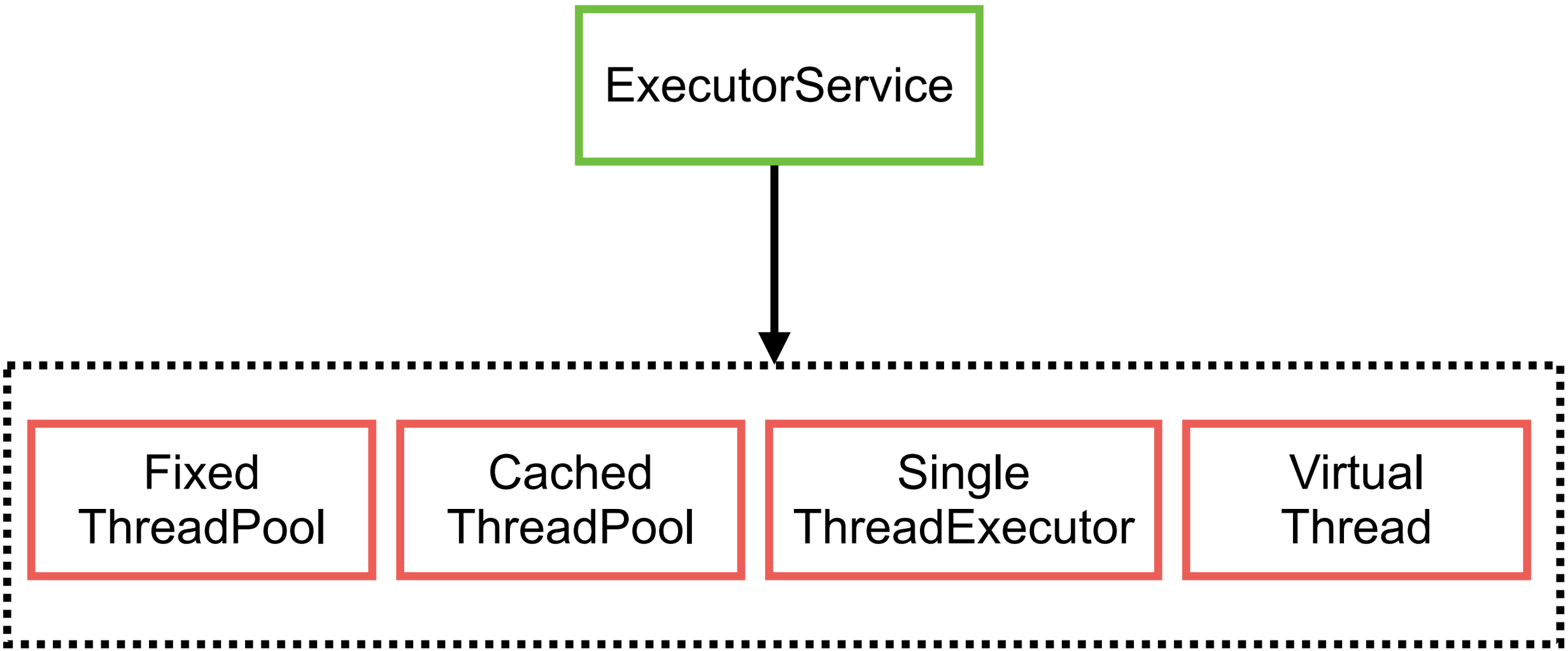


https://www.somkiat.cc/virtual-thread-in-spring-boot-3/

Platform vs Virtual Thread

Feature	Platform thread	Virtual thread (Java21+)
Mapping	1:1 with OS threads	M:N Many virtual threads to few OS threads
Creation cost	High memory and CPU	Low
Memory footprint	1 MB per thread (Stack size)	< 1 KB (Stored on heap)
Use case	CPU-intensive tasks	I/O intensive tasks (blocking call)
Scalability	Limit by OS resources	Scalable to millions
Pooling ?	Required to prevent resource exhaustion	Create a new one per task

High level concurrency framework



Executor Service

Thread pooling
Task management (runnable, callable)
Life cycle control
Working with try-with-resources (AutoClosable)

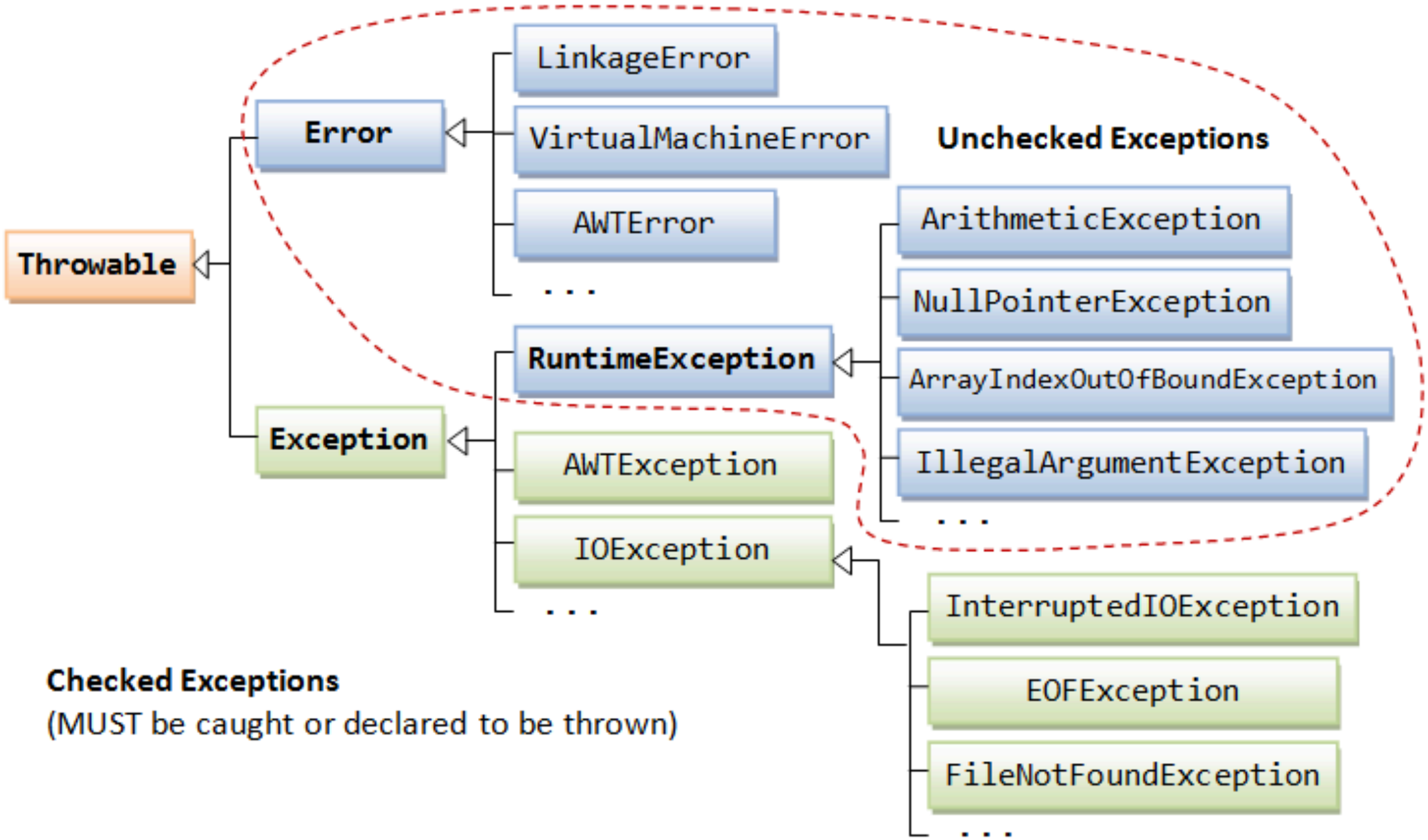
https://docs.oracle.com/en/java/javase/25/docs/api//java.base/java/util/concurrent/ExecutorService.html

Exception Handling

Types of Exception

- Compile-time
- Runtime
- Logical

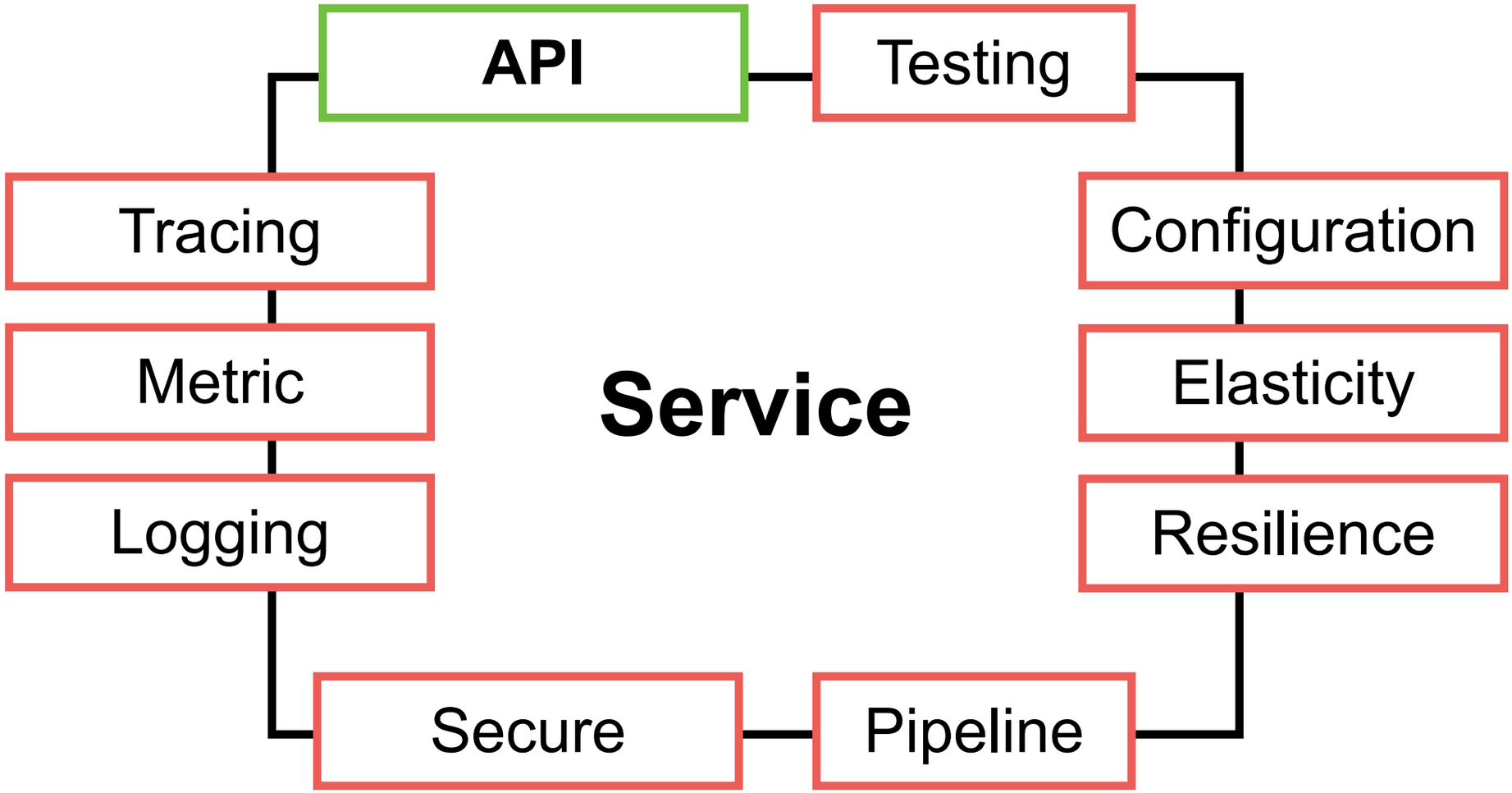
Exception Classes



Checked Exceptions
(MUST be caught or declared to be thrown)

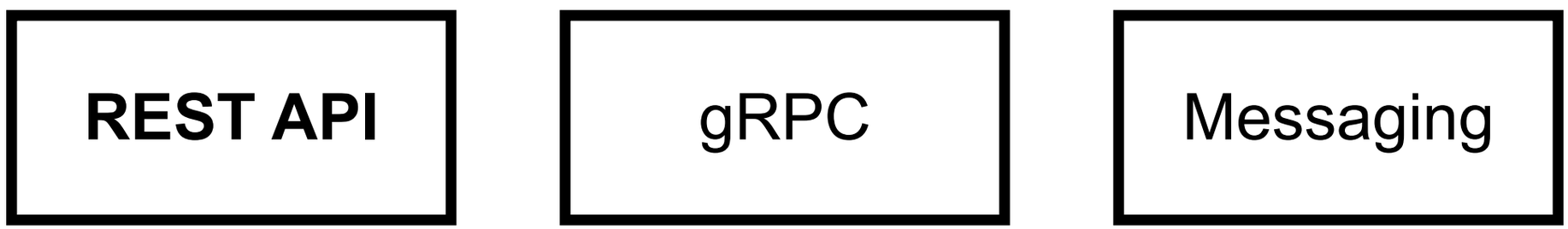
REST API with Spring Boot

Properties of Service



APIs

Application Programming Interface



REST

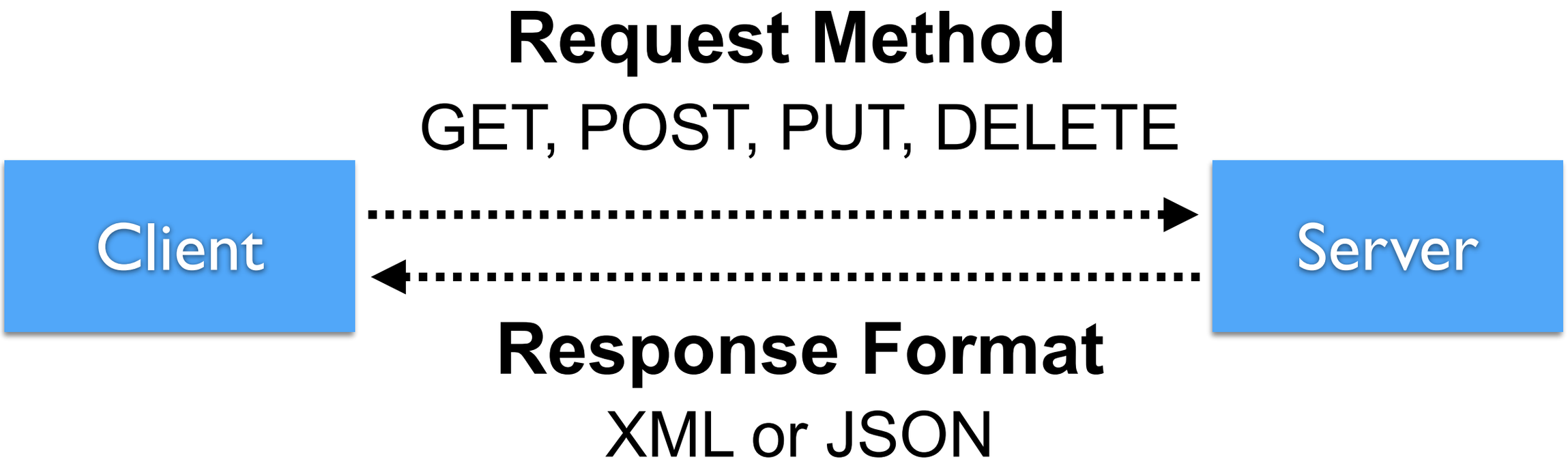
REpresentation **S**tate **T**ransfer

The style of **software architecture** behind RESTful services

Defined in 2000 by Roy Fielding

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

REST Request & Response



HTTP Methods meaning

Method	Meaning
GET	Read data
POST	Create/Insert new data
PUT/PATCH	Update data or insert if a new id
DELETE	Delete data

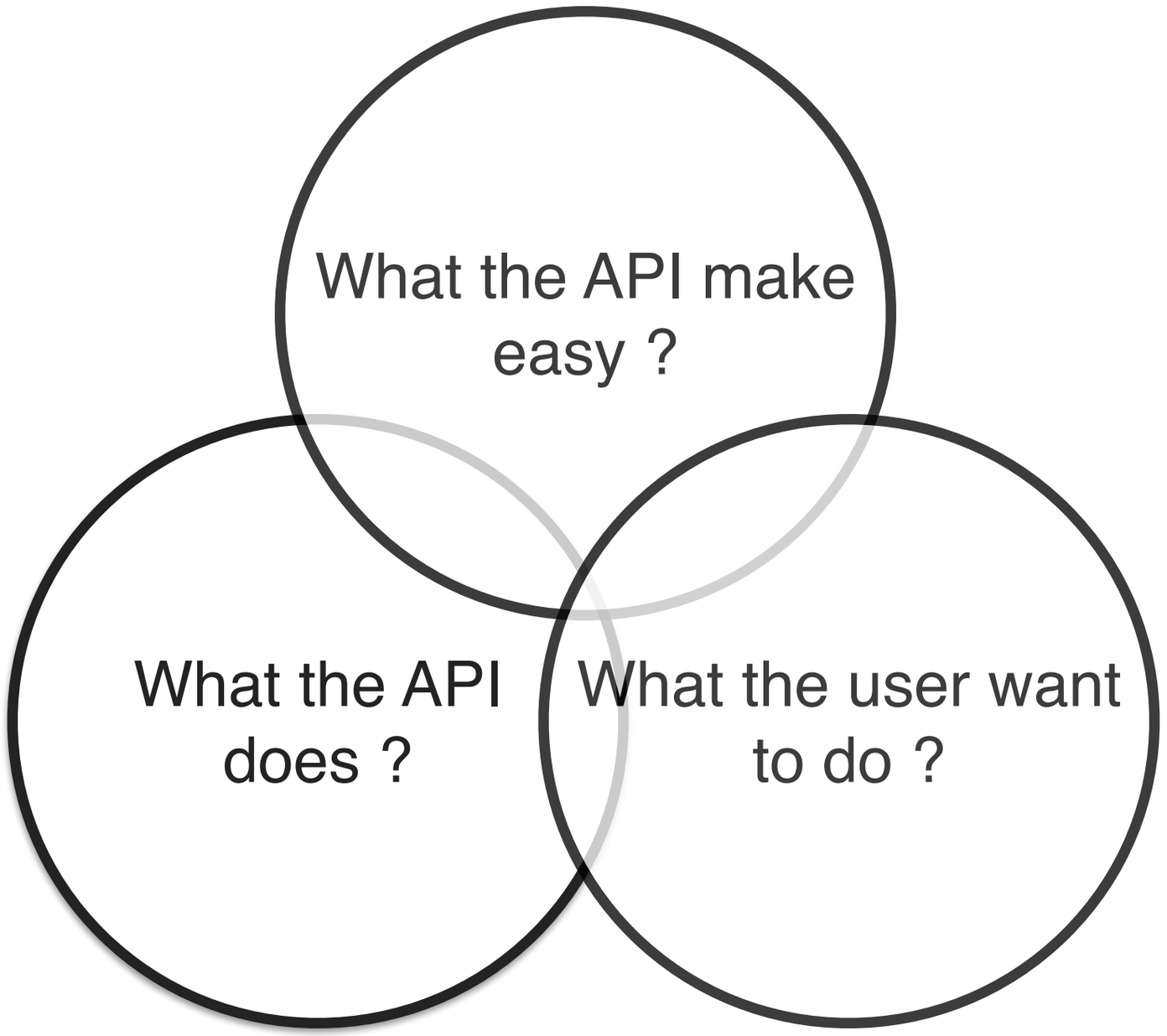
Response format ?

Status Code

Code	Description
1xx	Information
2xx	Successfu
3xx	Redirection
4xx	Client error
5xx	Server error

https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

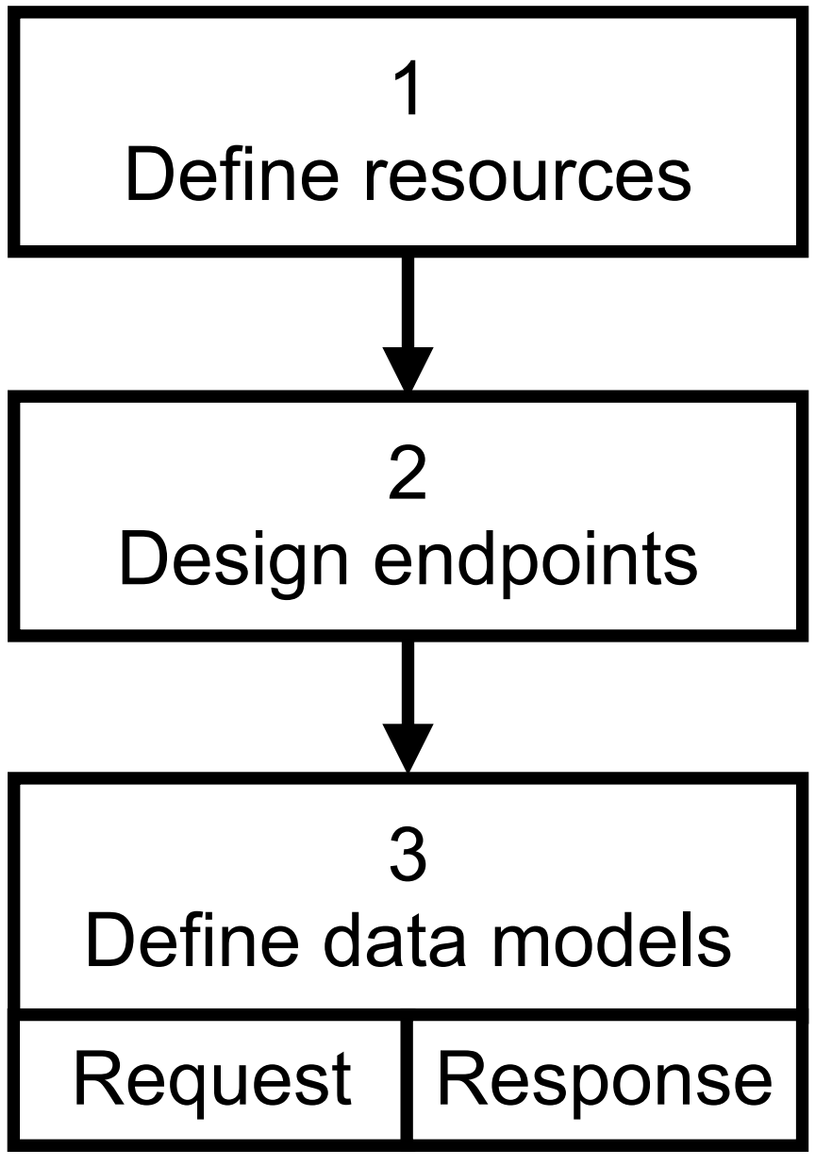
Good APIs ?



Principles of API Design

Consistency	Statelessness	Resource-oriented design
Use standard HTTP methods	Versioning	

Steps to Design APIs



More !!

Authentication
Authorization

Pagination and
filtering

Rate Limiting

Pagination and
filtering

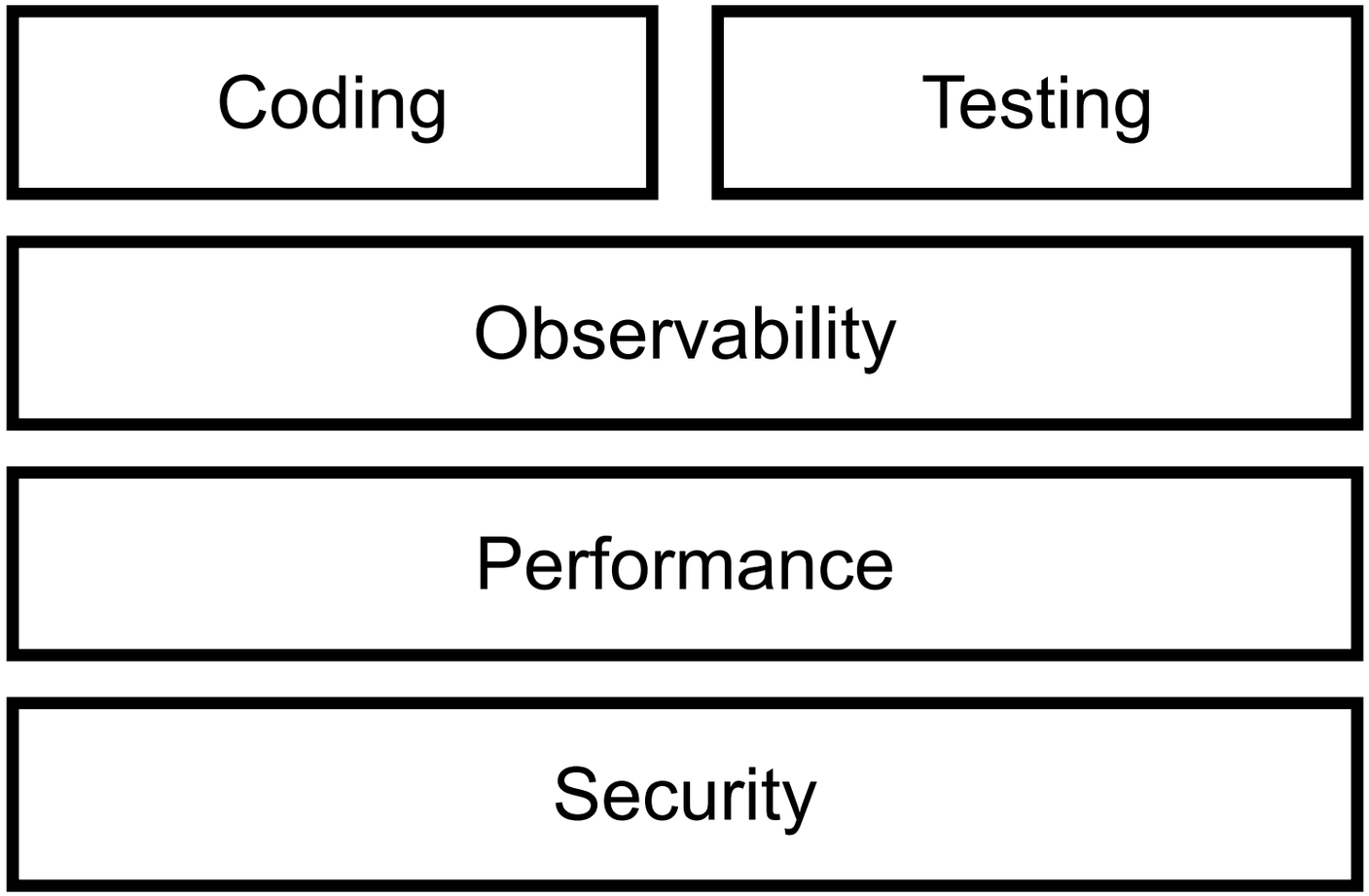
Error handling

Documentation

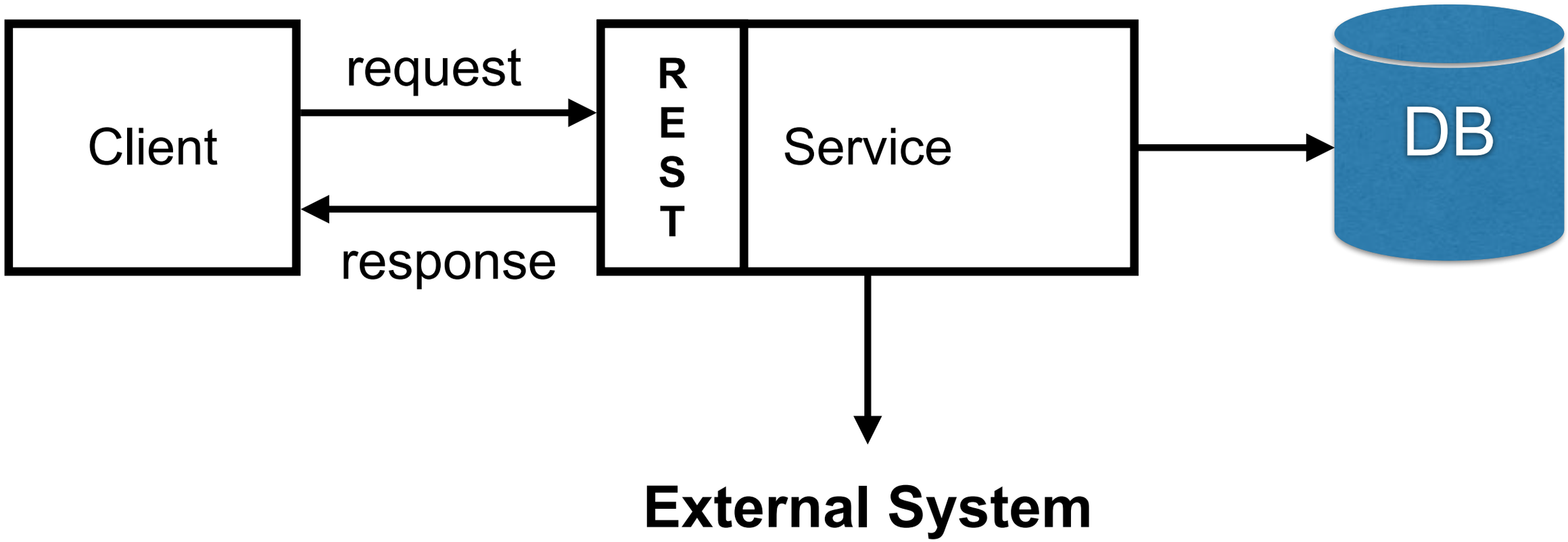
Testing

Monitoring and
Observability

Implementation



Structure of Service



Spring Boot Structure (1)

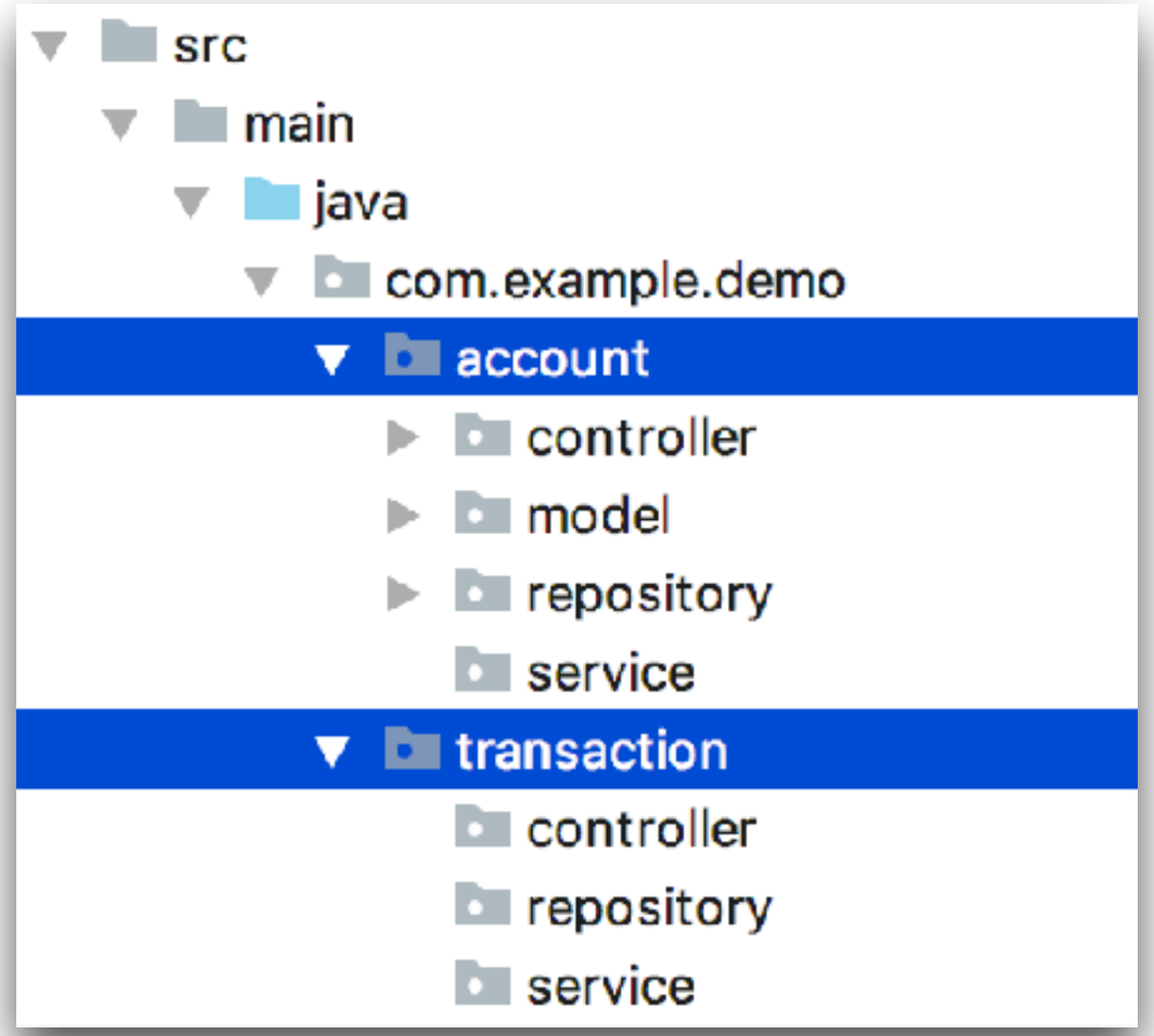
Separate by function/domain/feature

```
feature1
    - controller
    - service
    - repository
feature2
    - controller
    - service
    - repositor
```

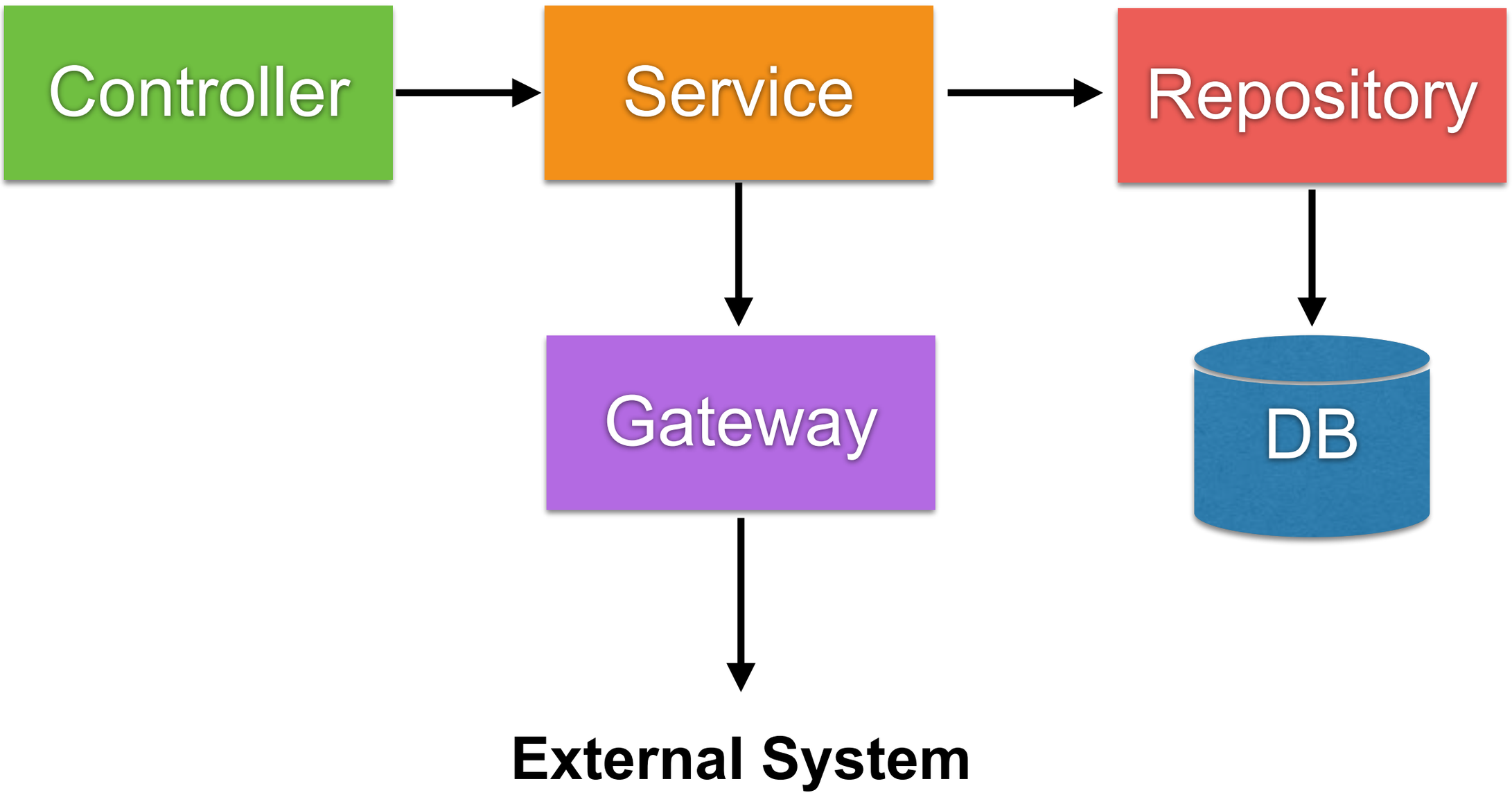
https://docs.spring.io/spring-boot/reference/using/structuring-your-code.html

Spring Boot Structure (2)

Separate by function/domain/feature



Structure of Spring Boot project



Controller

Request and Response
Validation input
Delegate to others classes such as service and repository

Service

Control the flow of main business logics
Manage database transaction
Don’t reuse service

Repository

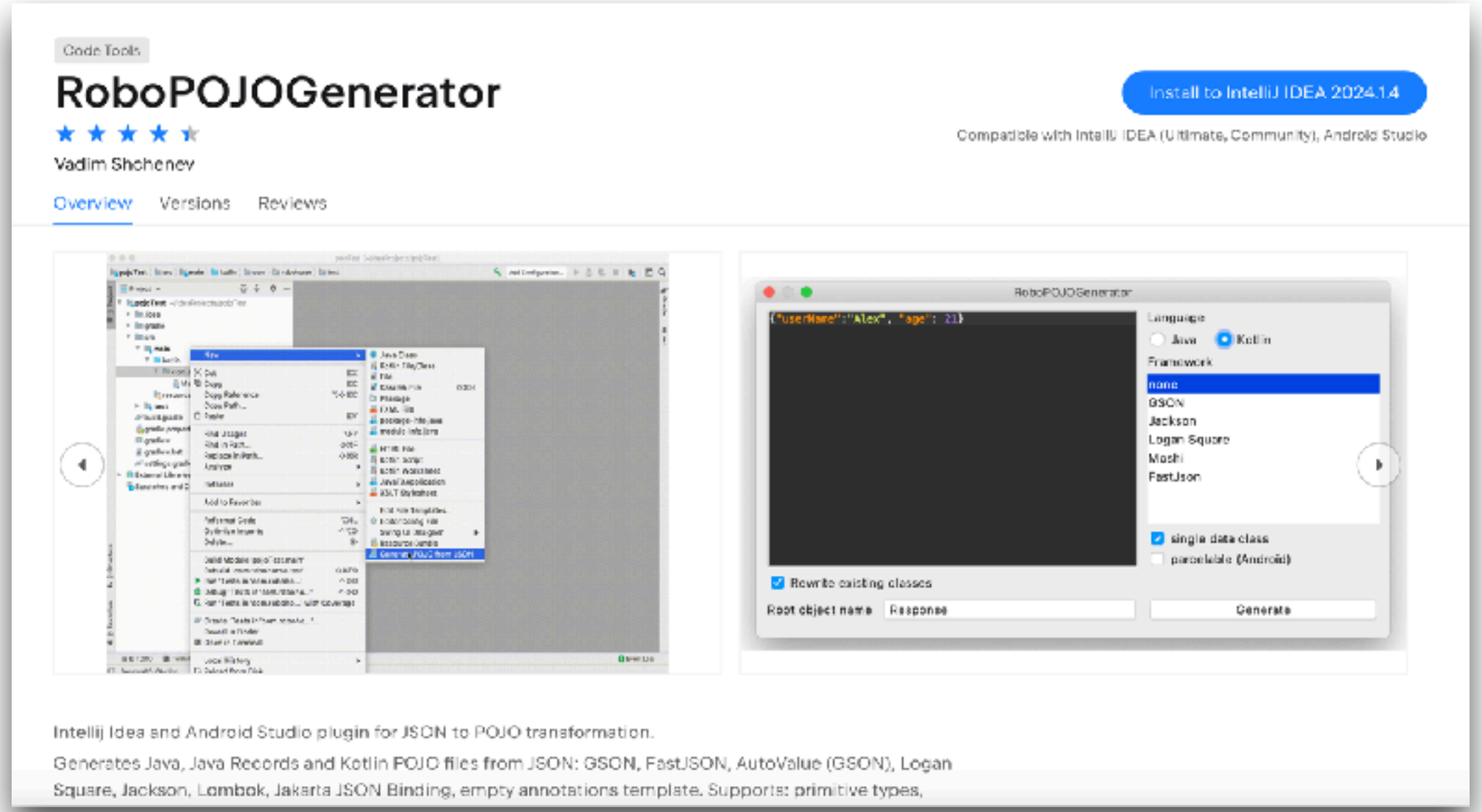
Manage data in data store such as RDBMS and NoSQL

Gateway

Call external service via network such as WebServices and RESTful APIs

Convert JSON to POJO

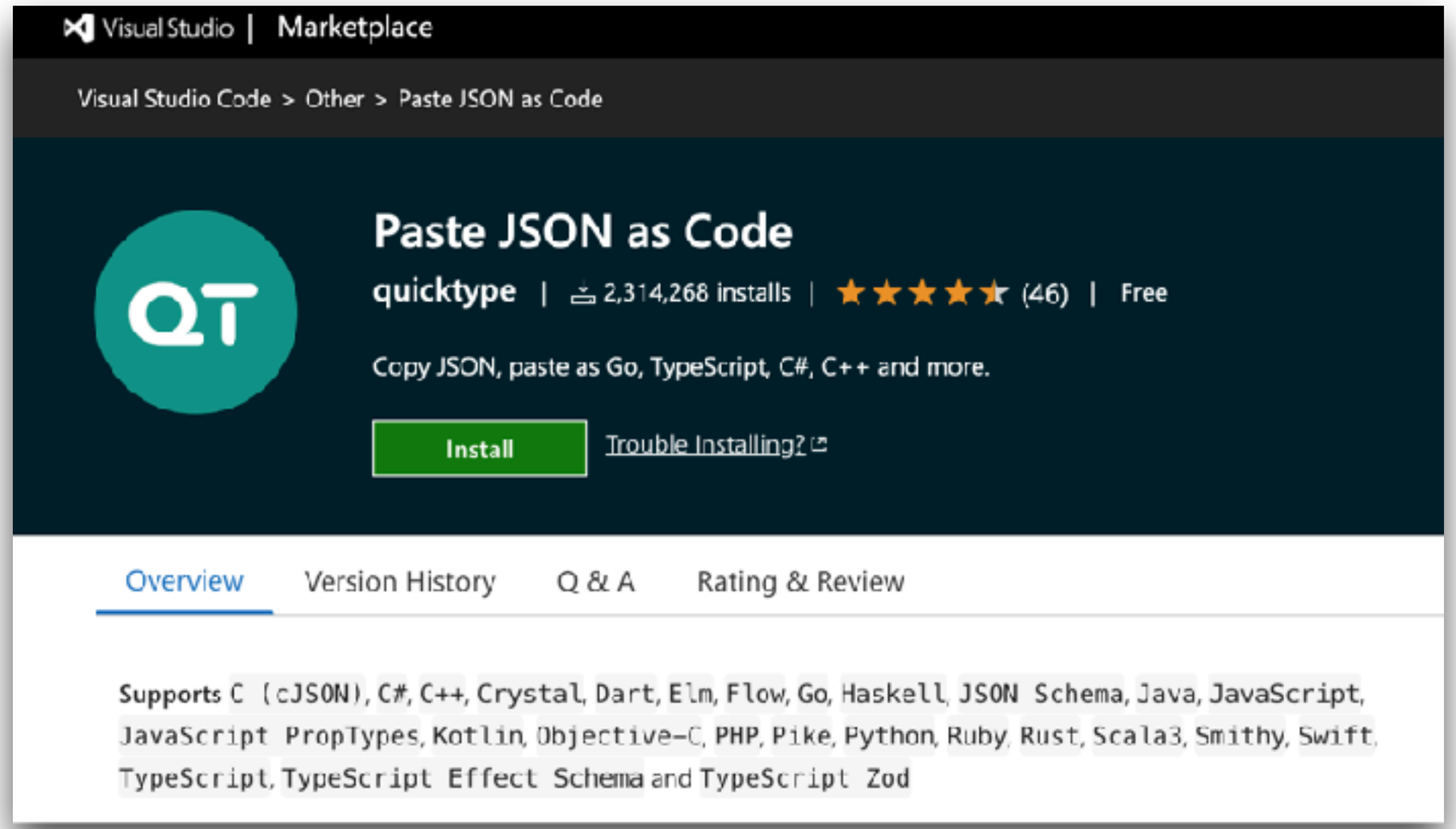
IntelliJ IDEA



https://plugins.jetbrains.com/plugin/8634-robopojogenerator

Convert JSON to POJO

VS Code

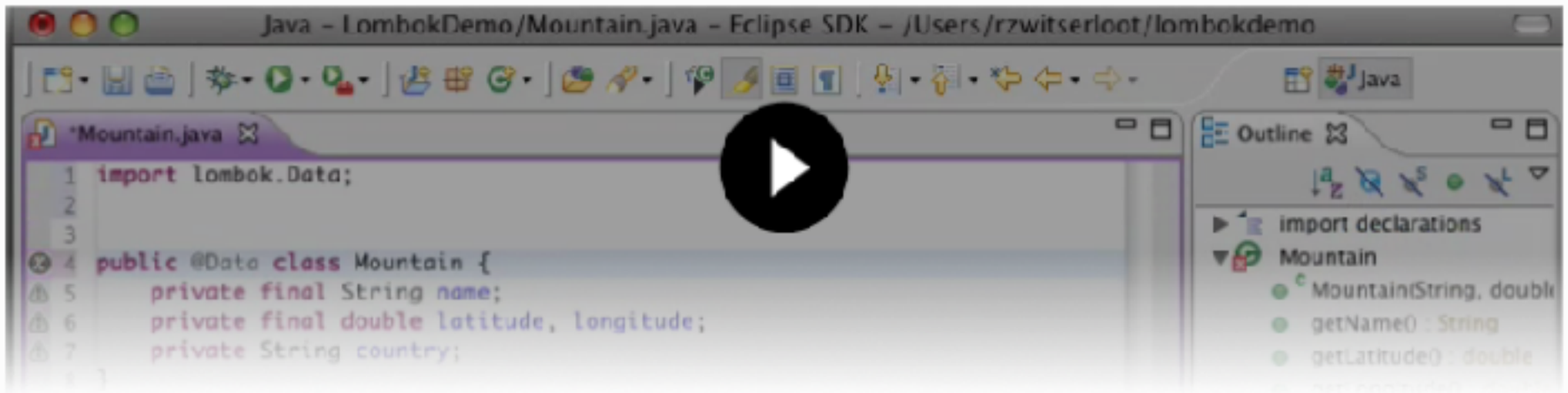


https://marketplace.visualstudio.com/items?itemName=quicktype.quicktype

Using Lombok

Project Lombok

Project Lombok is a Java library that automatically plugs into your editor and build tools, spicing up your java.
Never write another getter or equals method again, with one annotation your class has a fully featured builder,
Automate your logging variables, and much more.

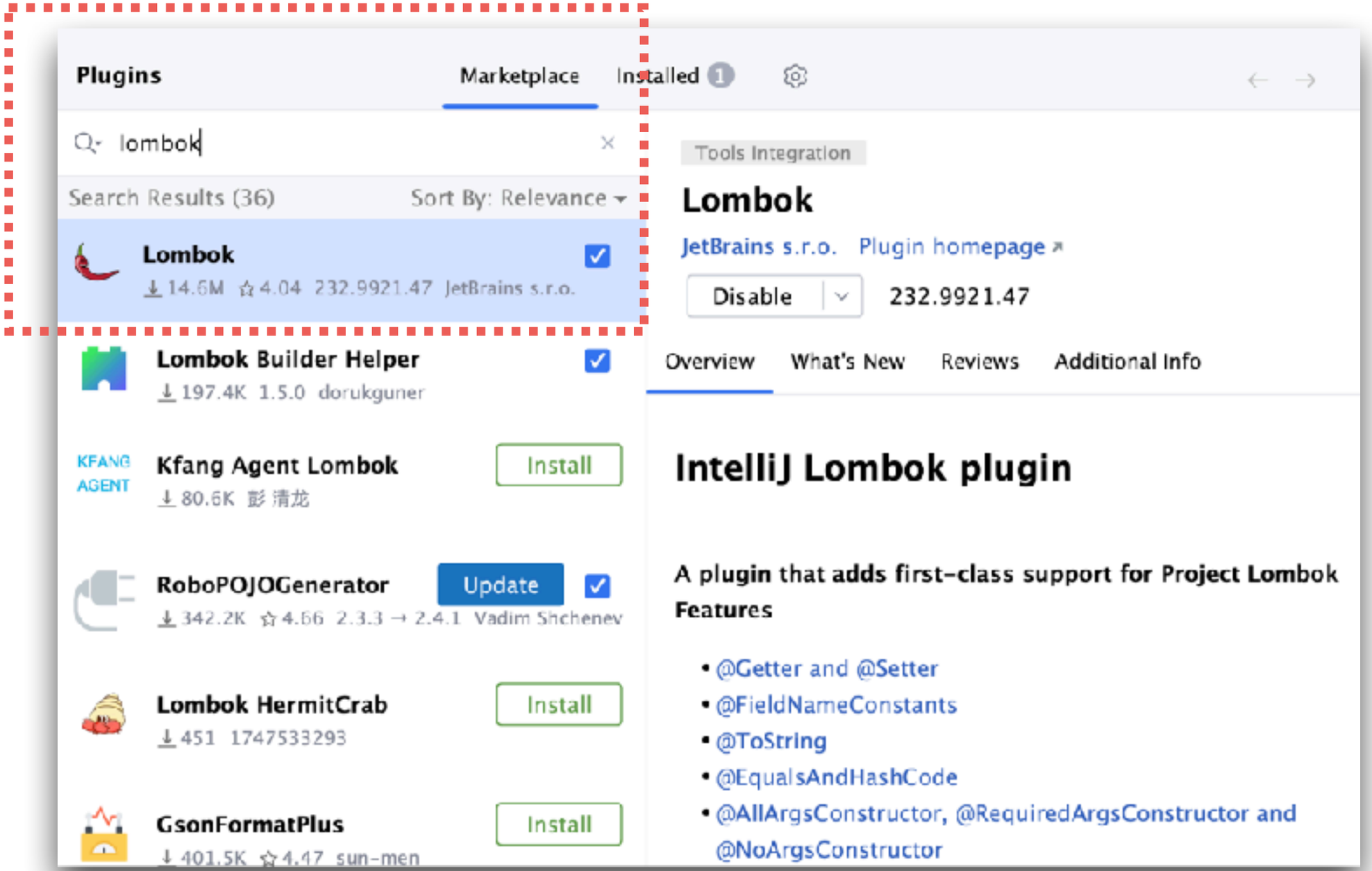


Show me a text and images based explanation and tutorial instead!

Project Lombok is powered by:

https://projectlombok.org/

IntelliJ IDEA plug-in



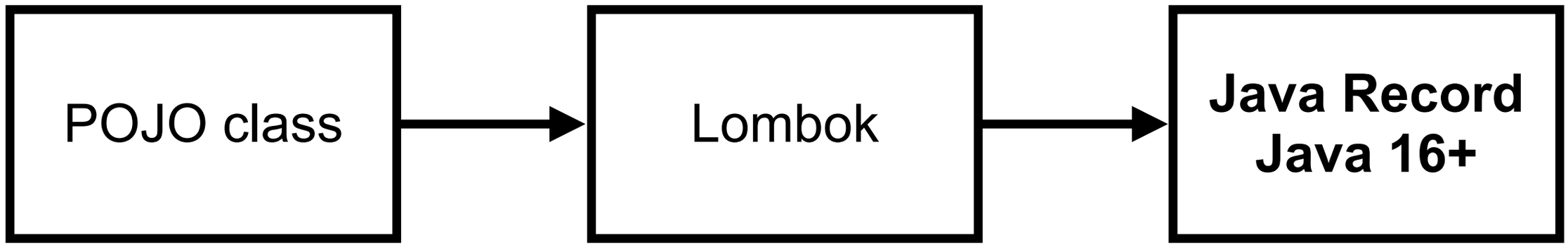
https://plugins.jetbrains.com/plugin/6317-lombok

Use Lombok

```java
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class UserRequest {
    private String email;
    private String password;
}
```

POJO to Record

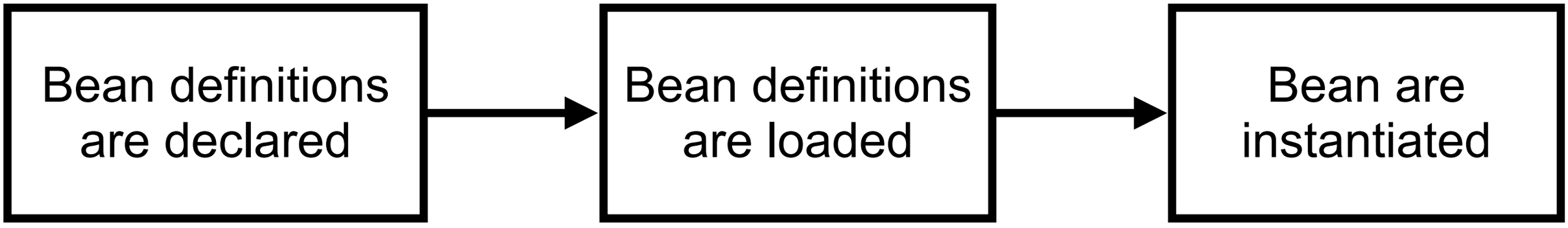
Reduce boilerplate code !!



https://docs.oracle.com/en/java/javase/21/language/records.html

Workshop with Spring Boot

Spring Bean Life Cycle ?



@Bean
@Component
@RestController
@Service
@Repository
@Configuration

Bean Scopes ?

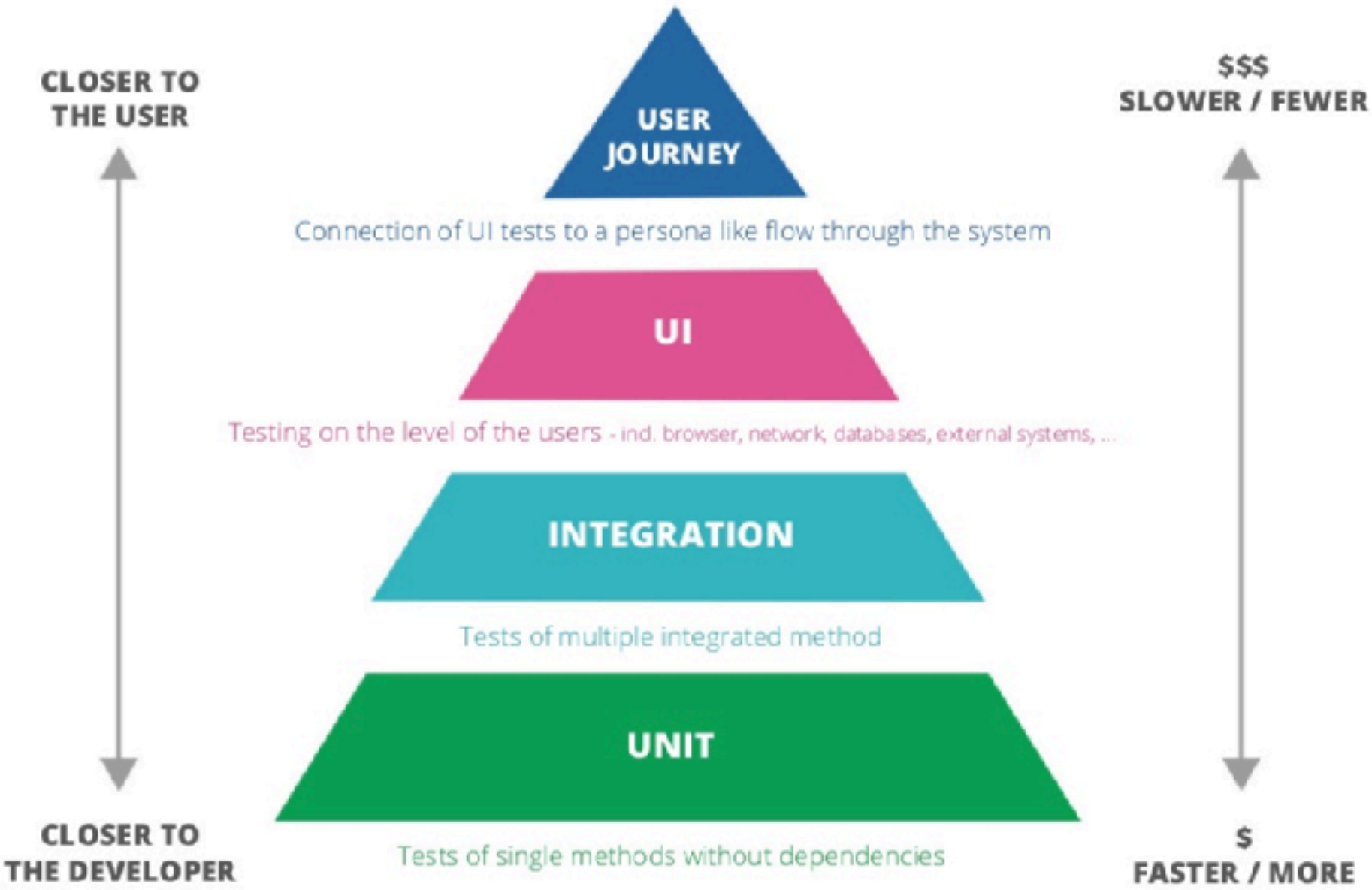
Spring Bean Scopes ?

Scope	Description	Environment
Singleton	One shared instance per spring container (default)	Core/Any
Prototype	A new instance created every time it is requested	Core/Any

https://docs.spring.io/spring-framework/reference/core/beans/factory-scopes.html

How many number of beans ?

Testing with Spring Boot



Spring Boot Testing

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
</dependency>
```

https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing

Testing in Spring Boot ?

@SpringBootTest
@WebMVCTest
@JsonTest
@DataJpaTest
@RestClientTest

https://docs.spring.io/spring-boot/reference/testing/spring-boot-applications.html

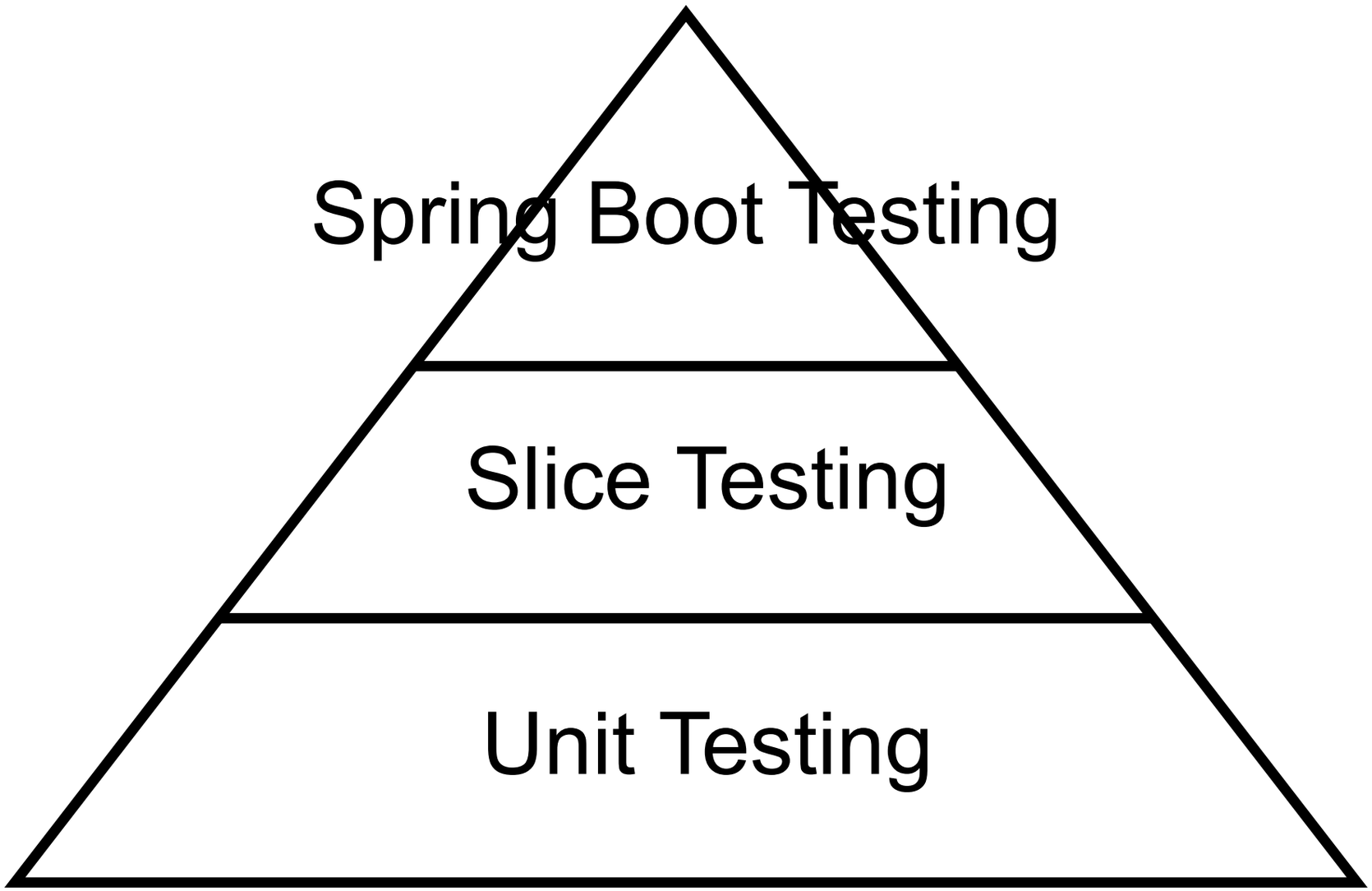
Testing in Spring Boot

@SpringBootTest

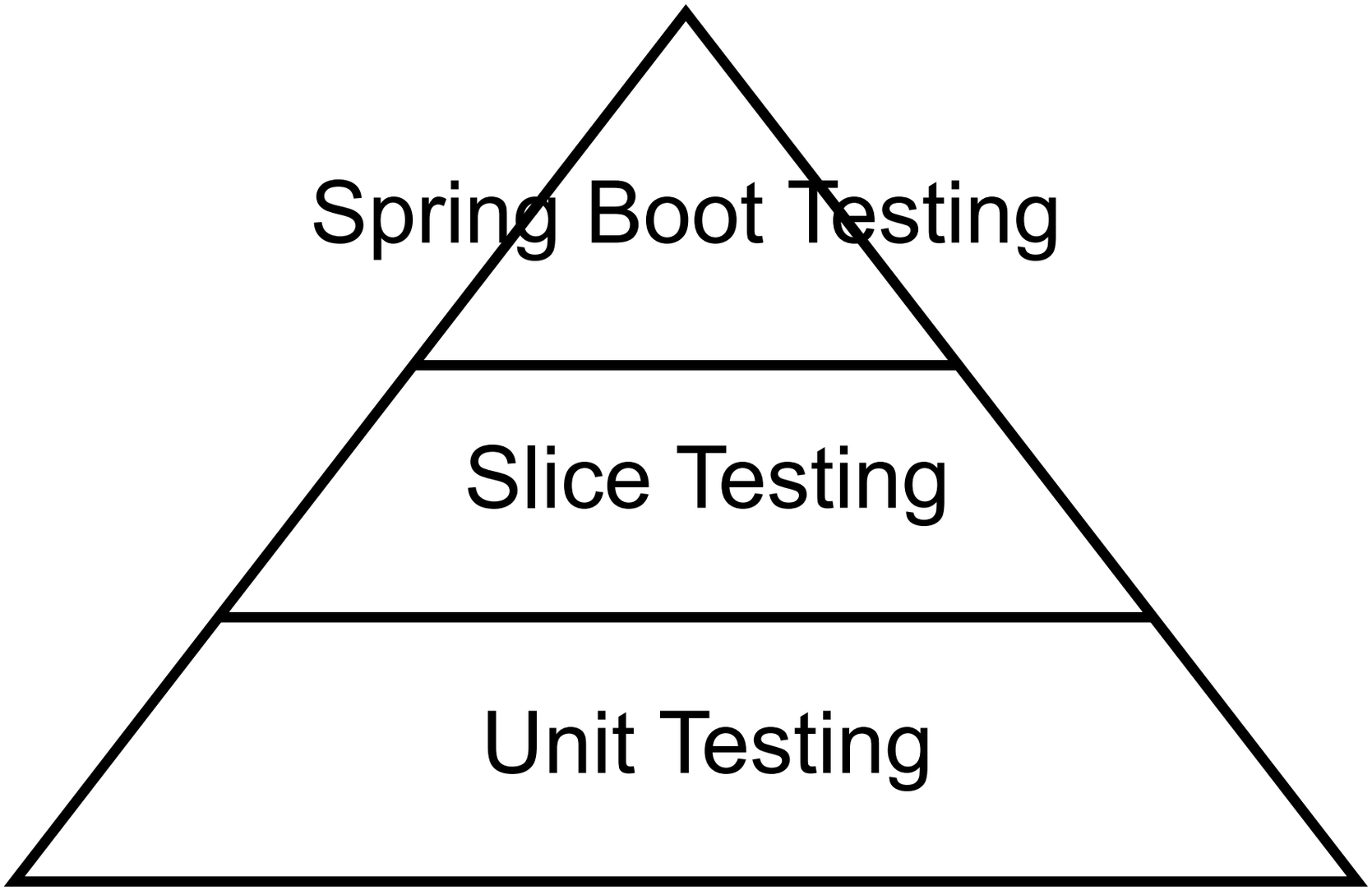
@WebMVCTest
@JsonTest
@DataJpaTest
@RestClientTest

Slice testing

Testing in Spring Boot



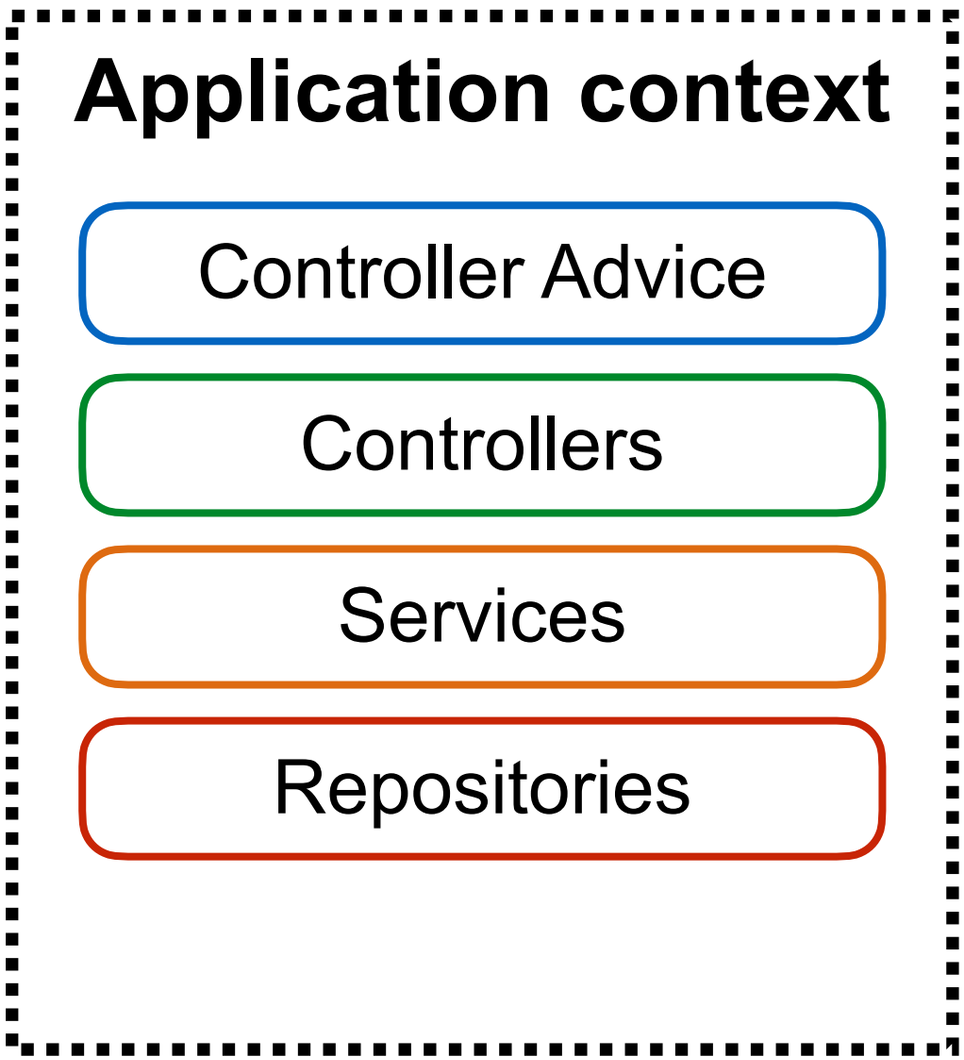
Testing in Spring Boot



Controller testing

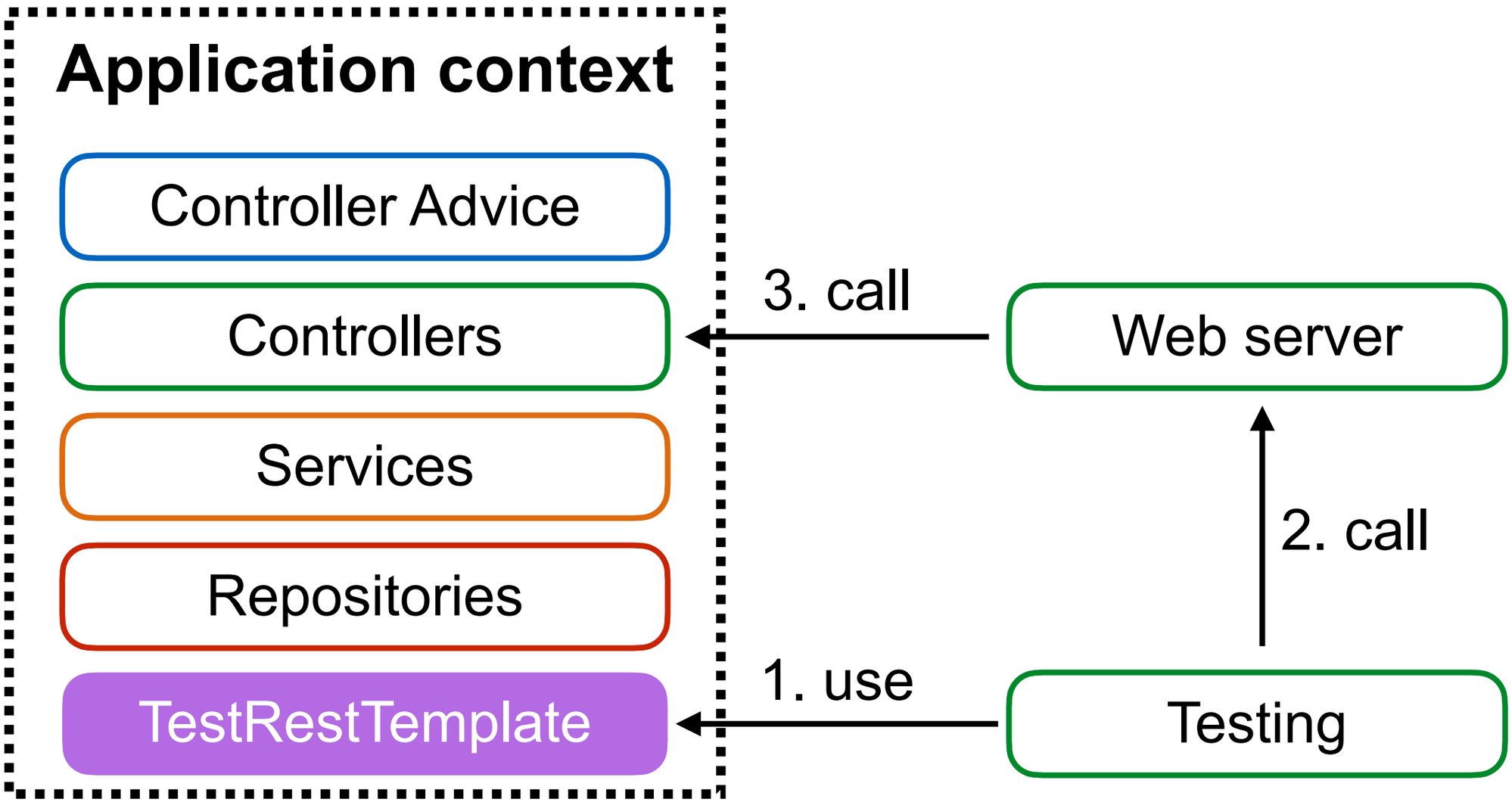
1. Spring Boot Testing
2. Slice Testing with MockMvc
3. Unit Testing

1. SpringBootTest



https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#features.testing

1. SpringBootTest



SpringBootTest #1

```java
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment
        = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class HelloControllerTest {

    @Autowired
    private TestRestTemplate testRestTemplate;

    @Test
    public void sayHi() {
        // Action :: Call controller
        Hello actualResult
                = testRestTemplate.getForObject("/hello/somkiat",
                                                Hello.class);

        // Assertion :: Check result with expected result
        assertEquals("Hello, somkiat", actualResult.getMessage());

    }
}
```

SpringBootTest #2

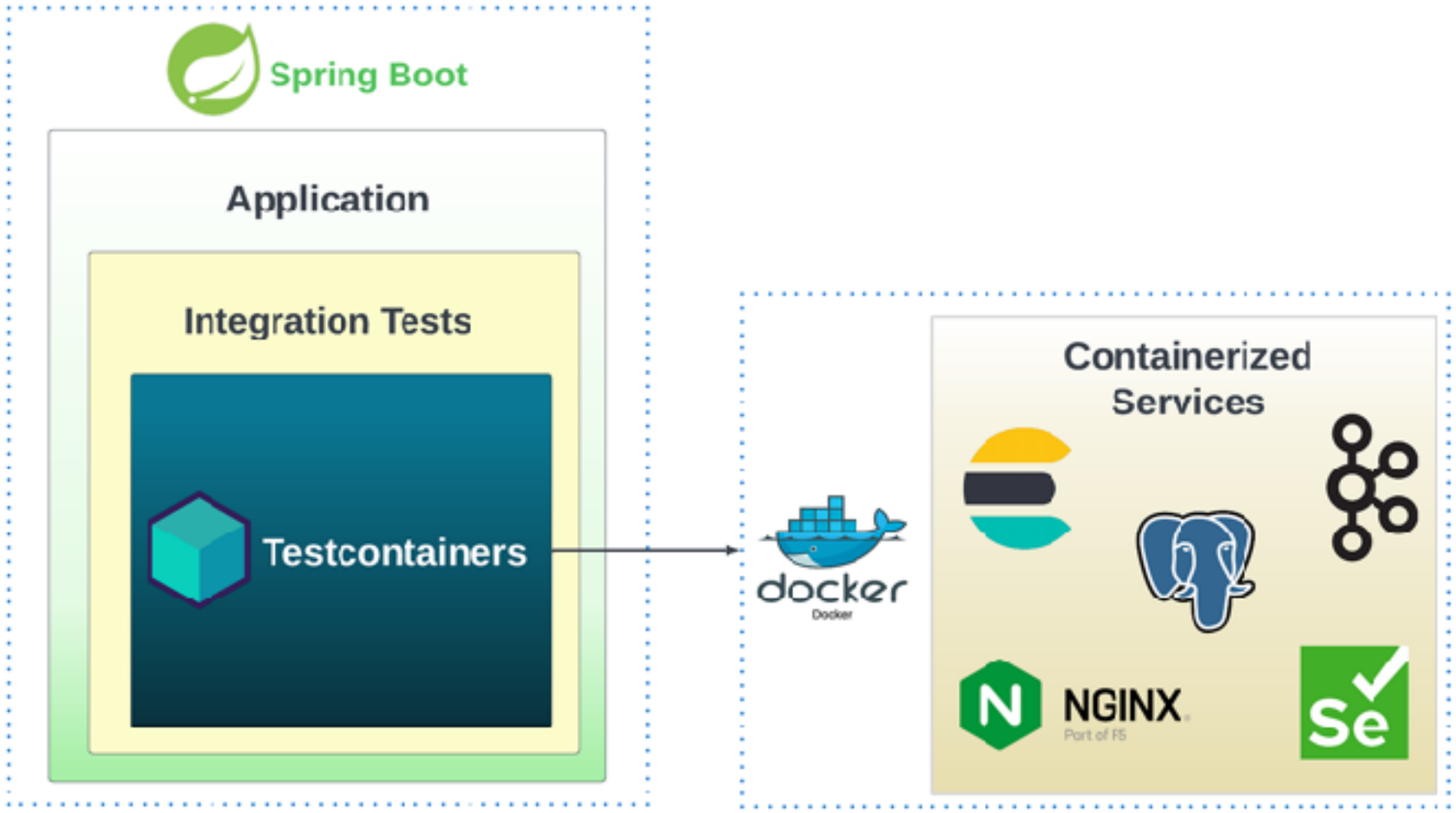
```java
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment
        = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class HelloControllerTest {

    @Autowired
    private TestRestTemplate testRestTemplate;

    @Test
    public void sayHi() {
        // Action :: Call controller
        Hello actualResult
                = testRestTemplate.getForObject("/hello/somkiat",
                                                Hello.class);

        // Assertion :: Check result with expected result
        assertEquals("Hello, somkiat", actualResult.getMessage());

    }
}
```

SpringBootTest #3

```java
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment
        = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class HelloControllerTest {

    @Autowired
    private TestRestTemplate testRestTemplate;

    @Test
    public void sayHi() {
        // Action :: Call controller
        Hello actualResult
                = testRestTemplate.getForObject("/hello/somkiat",
                                                Hello.class);

        // Assertion :: Check result with expected result
        assertEquals("Hello, somkiat", actualResult.getMessage());

    }
}
```

Testing in Spring Boot

Manage dependencies with TestContainers



https://docs.spring.io/spring-boot/reference/testing/testcontainers.html

Error handling

Working with Error Responses

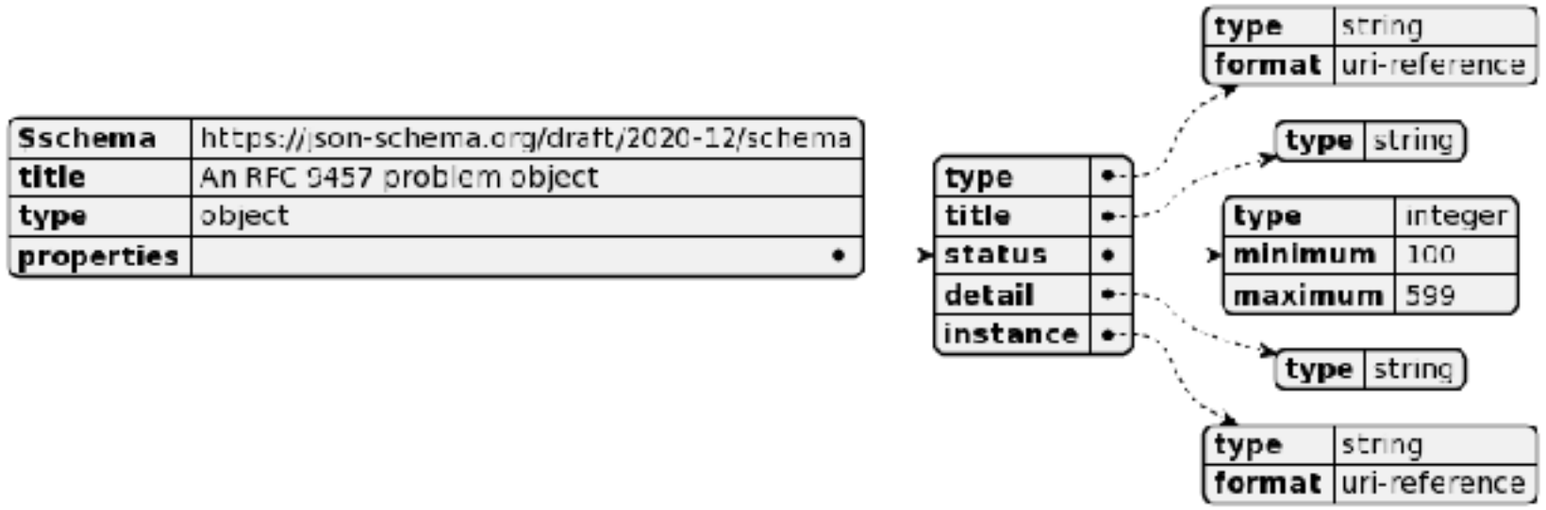
ErrorResponse

ProblemDetail

Customization

https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-rest-exceptions.html

ProblemDetail




```
{
  "type":"https://problems-registry.smartbear.com/missing-body-property",
  "status":400,
  "title":"Missing body property",
  "detail":"The request is missing an expected body property.",
  "code":"400-09",
  "instance":"/logs/regisrations/d24b2953-ce05-488e-bf31-67de50d3d085",
  "errors":[
    {
        "detail":"The body property {name} is required",
        "pointer":"/name"
    }
  ]
}
```

https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/http/ProblemDetail.html

ProblemDetail

Enable ProblemDetail for error by default

application.properties

```
spring.mvc.problemdetails.enabled=true
```

Example in ControllerAdvice

```
@RestControllerAdvice
class HelloControllerAdvice {

    @ExceptionHandler(InvalidInputException.class)
    public ProblemDetail handleInvalidInputException(
            InvalidInputException e, WebRequest request) {

        ProblemDetail problemDetail
                = ProblemDetail
                .forStatusAndDetail(HttpStatus.BAD_REQUEST, e.getMessage());
        problemDetail.setInstance(URI.create("demo-instance"));
        problemDetail.setTitle("demo");

        return problemDetail;
    }
}
```

Error handling

```
@Service
public class UserService {

    private AccountRepository accountRepository;

    @Autowired
    public UserService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }
```

```
public Account getAccount(int id) {
    Optional<Account> account = accountRepository.findById(id);
    if(account.isPresent()) {
        return account.get();
    }
    throw new MyAccountNotFoundException(
            String.format("Account id=[%d] not found", id));
}
```

MyAccountNotFoundException

```
public class MyAccountNotFoundException
                extends RuntimeException {

    public MyAccountNotFoundException(String message) {
        super(message);
    }

}
```

Response Status

404 = Not Found

Status	Description
400	Request body doesn’t meet API spec
401	Authentication/Authorization fail
403	User can’t perform the operation
404	Resource does not exist
405	Unsupported operation
500	Error on server

Handling error in Spring Boot

```java
@RestControllerAdvice
public class AccountControllerHandler {

    @ExceptionHandler(MyAccountNotFoundException.class)
    public ResponseEntity<ExceptionResponse> accountNotFound(
            MyAccountNotFoundException exception) {

        ExceptionResponse response =
                new ExceptionResponse(exception.getMessage(),
                                      "More detail");

        return new ResponseEntity<ExceptionResponse>(response,
                                                     HttpStatus.NOT_FOUND);

    }

}
```

1

Handling error in Spring Boot

```
@RestControllerAdvice
public class AccountControllerHandler {

    @ExceptionHandler(MyAccountNotFoundException.class)
    public ResponseEntity<ExceptionResponse> accountNotFound(
            MyAccountNotFoundException exception) {

        ExceptionResponse response =
                new ExceptionResponse(exception.getMessage(),
                                      "More detail");

        return new ResponseEntity<ExceptionResponse>(response,
                                                     HttpStatus.NOT_FOUND);

    }
}
```

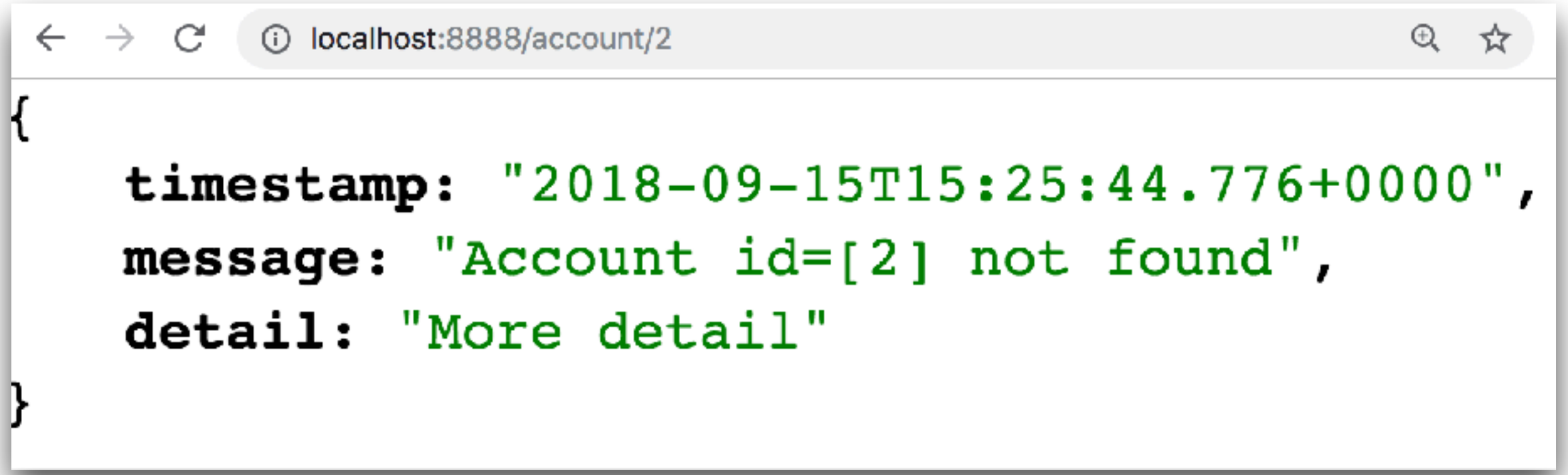
2

ExceptionReponse

Response format of error

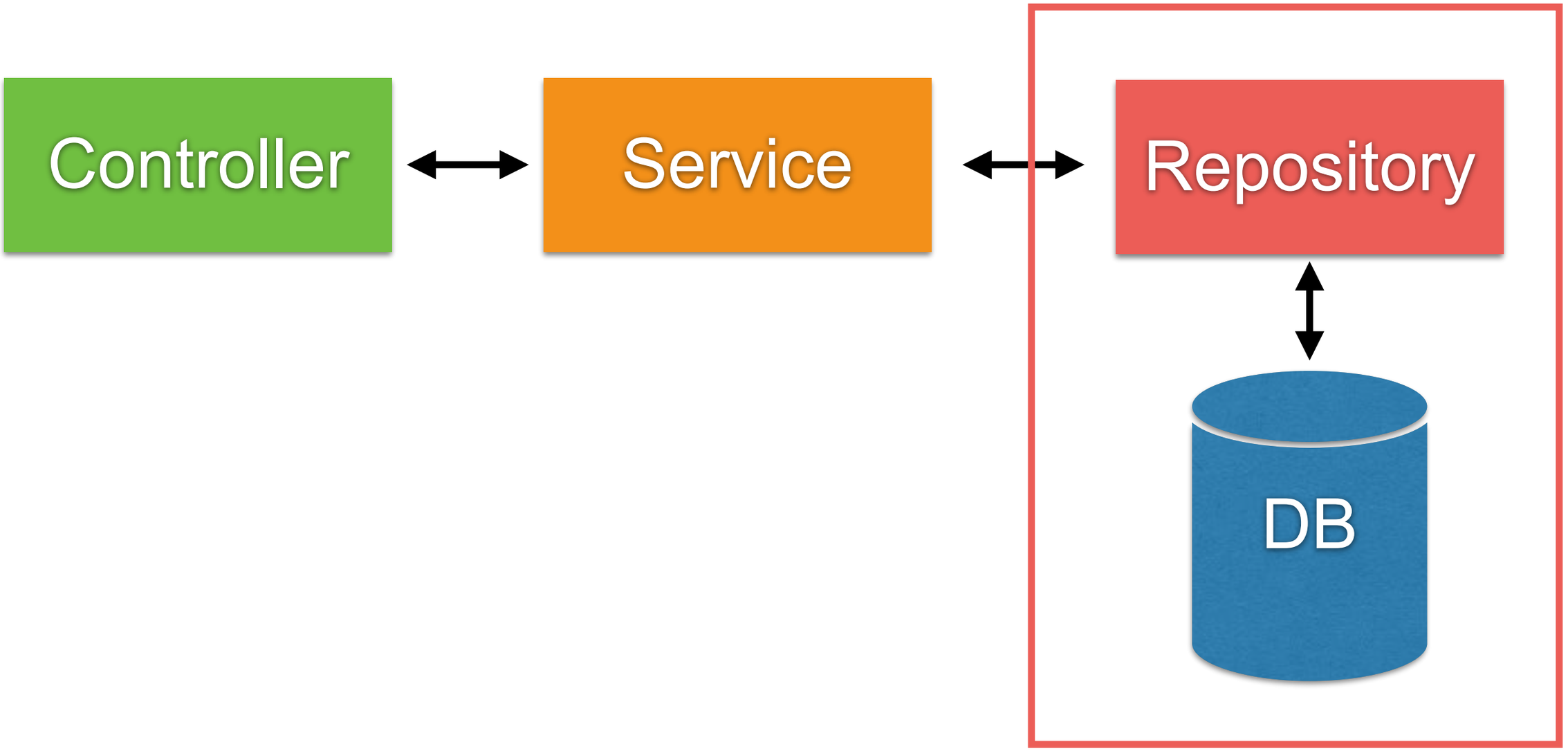
```
public class ExceptionResponse{

    private Date timestamp = new Date();
    private String message;
    private String detail;

public ExceptionResponse(String message, String detail) {
    this.message = message;
    this.detail = detail;
}
```

Result of API



Working with Database

Repository Layer in Spring



Basic of JDBC

```java
// 1. Load jdbc driver
Class.forName("postgresql");

// 2. Create connection
Connection connection = DriverManager.getConnection("", "", "");

// 3. Prepared Statement
String sql = "SELECT * FROM TABLE WHERE name=?";
PreparedStatement pStmt = connection.prepareStatement(sql);

// 4. Query
ResultSet resultSet = pStmt.executeQuery();
while(resultSet.next()) {

}

// 5. Release resource
if(resultSet != null) {
    resultSet.close();
    resultSet = null;
}
```

Framework !!

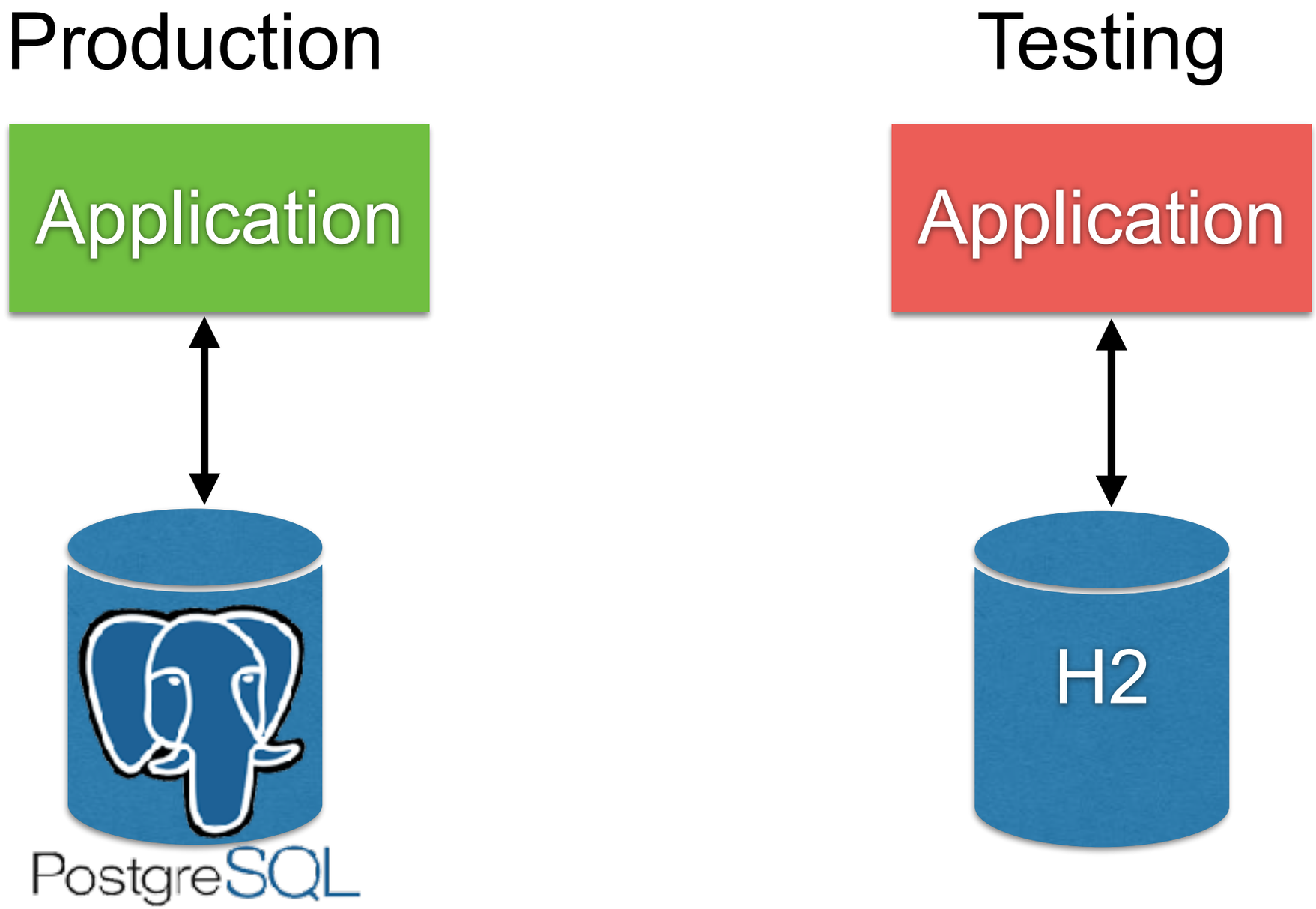
```
// 1. Load jdbc driver
Class.forName("postgresql");

// 2. Create connection
Connection connection = DriverManager.getConnection("", "", "");
```

Manage by Framework

```
// 3. Prepared Statement
String sql = "SELECT * FROM TABLE WHERE name=?";
PreparedStatement pStmt = connection.prepareStatement(sql);

// 4. Query
ResultSet resultSet = pStmt.executeQuery();
while(resultSet.next()) {

}

// 5. Release resource
if(resultSet != null) {
    resultSet.close();
    resultSet = null;
}
```

Manage by Framework

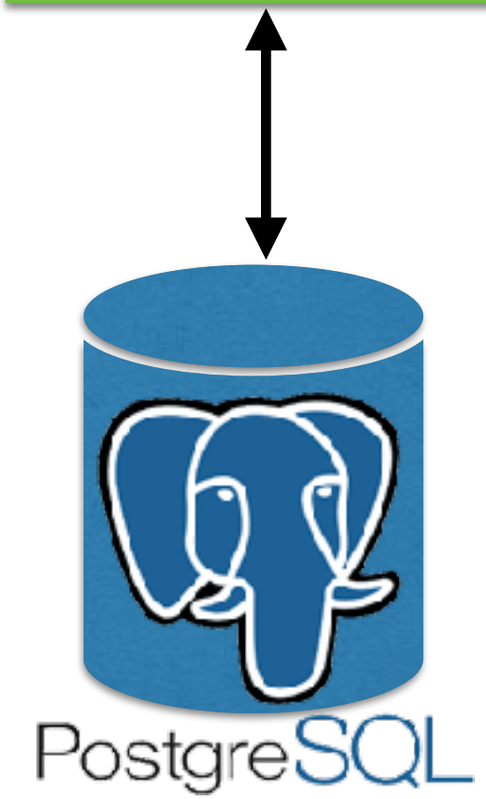
Working with Database ?



Working with Database ?

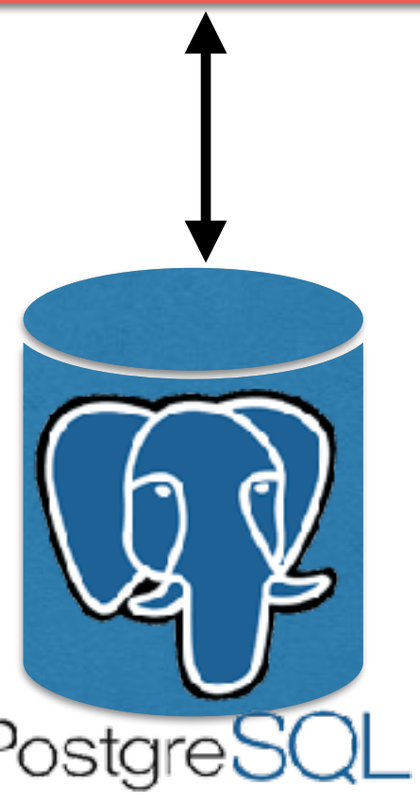
Production

Application



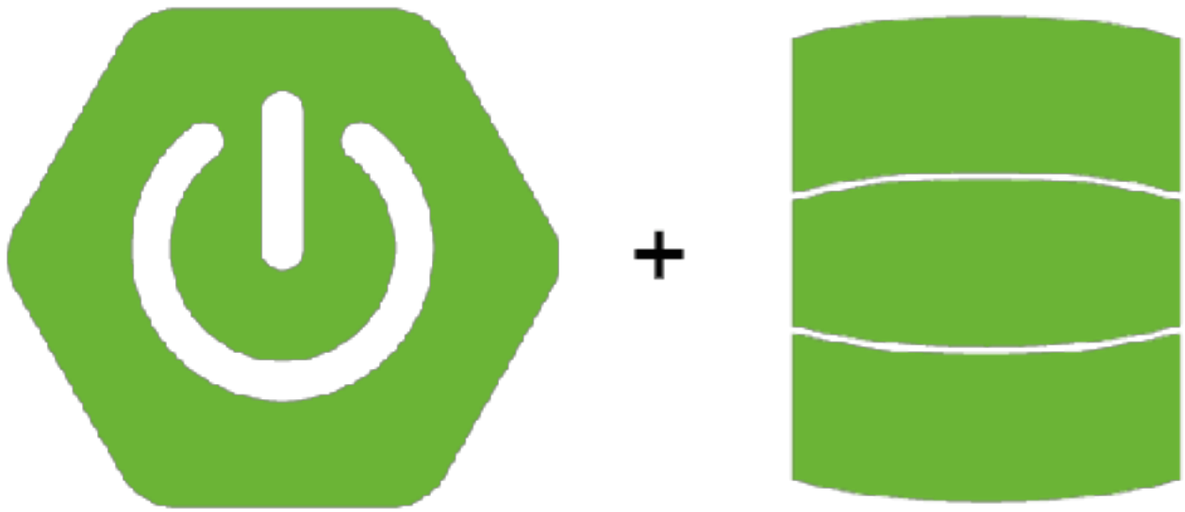
Testing

Application



Working with repository

We’re using Spring Data

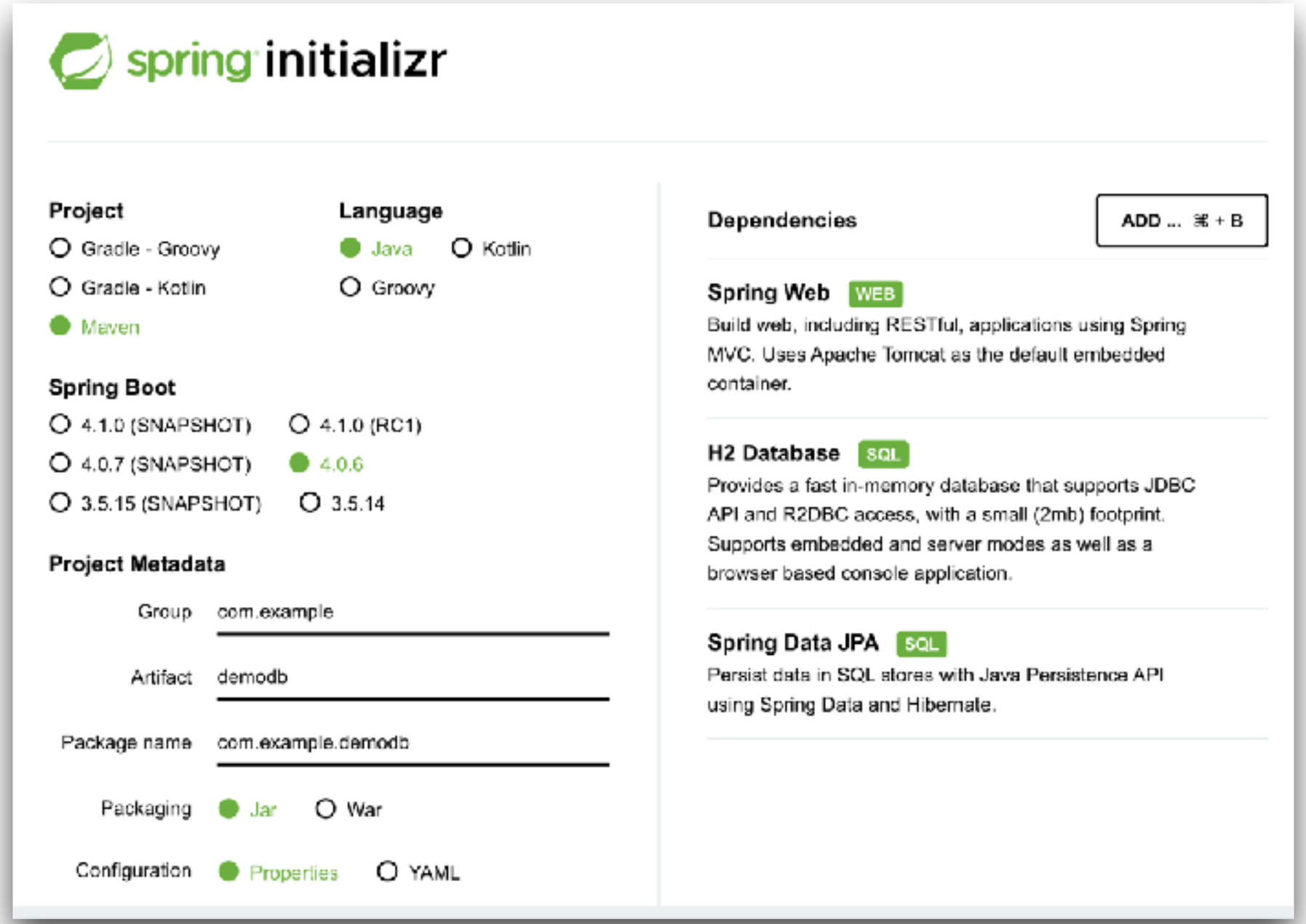


https://spring.io/projects/spring-data

Working with Database

Manage transactions with @Transactional
Manage multiple datasources

Create Project



https://start.spring.io/

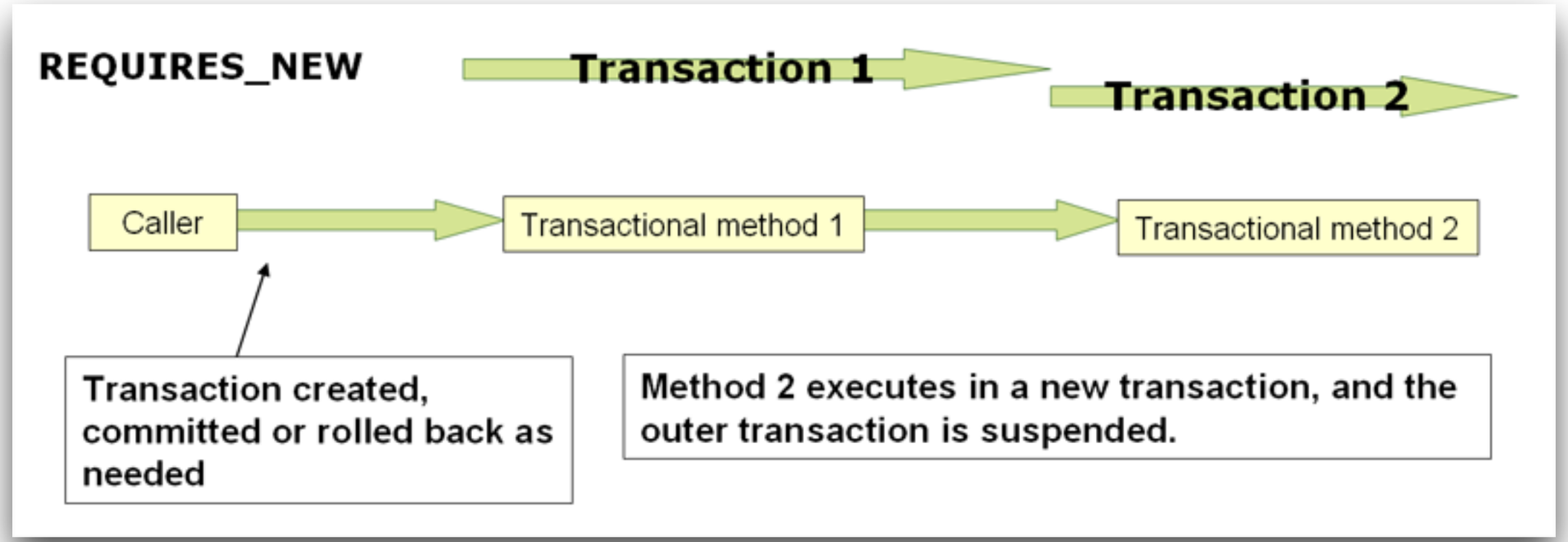
Transaction Propagation

Transaction Propagation (1)

Define how a transaction boundary with @Transaction

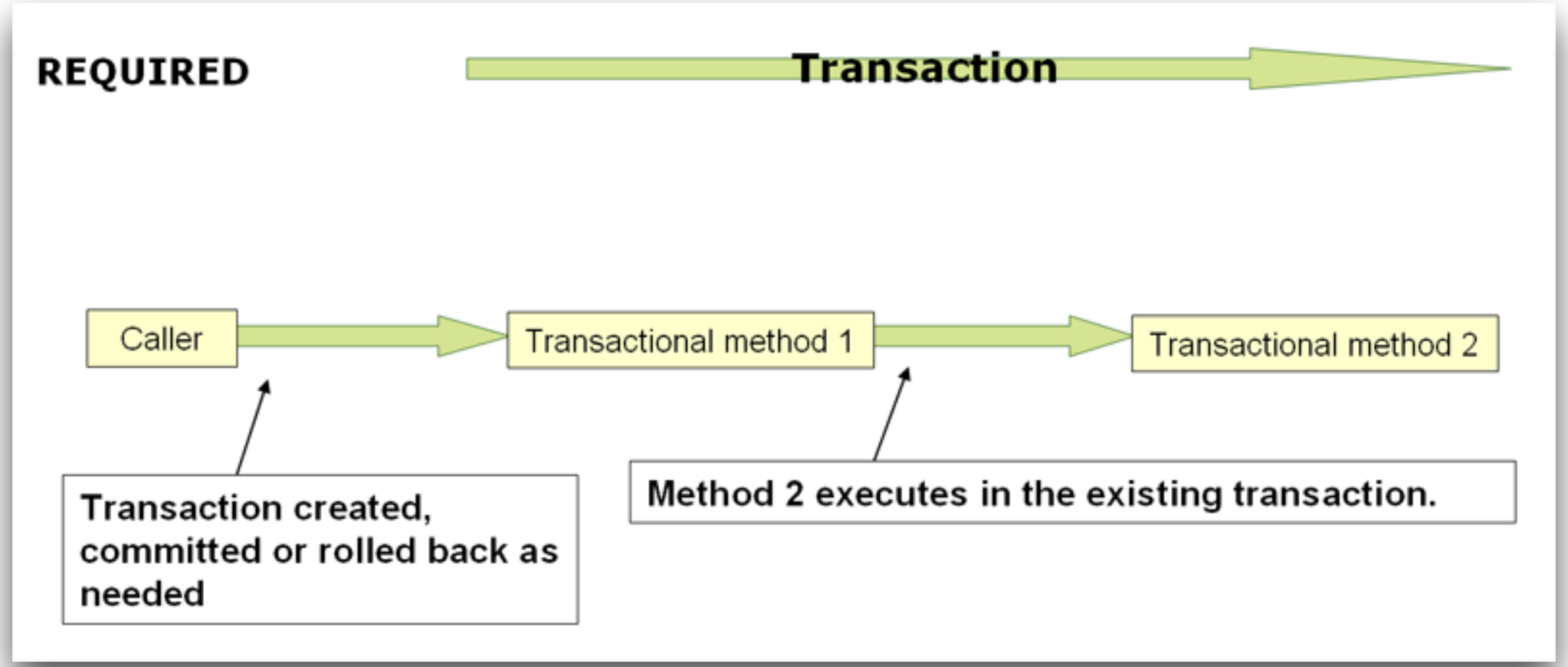
Propagation	Existing tx	No existing tx
REQUIRED	Joins the transaction	Start a new transaction
REQUIRES_NEW	Suspend current, start new	Start a new transaction
SUPPORTS	Joins the transaction	Run with non-transaction
MANDATORY	Joins the transaction	Throw exception
NOT_SUOORTED	Suspend current, run with non-transaction	Run with non-transaction
NEVER	Throw exception	Run with non-transaction

Transaction Propagation (2)



https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/tx-propagation.html

Transaction Propagation (3)



https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/tx-propagation.html

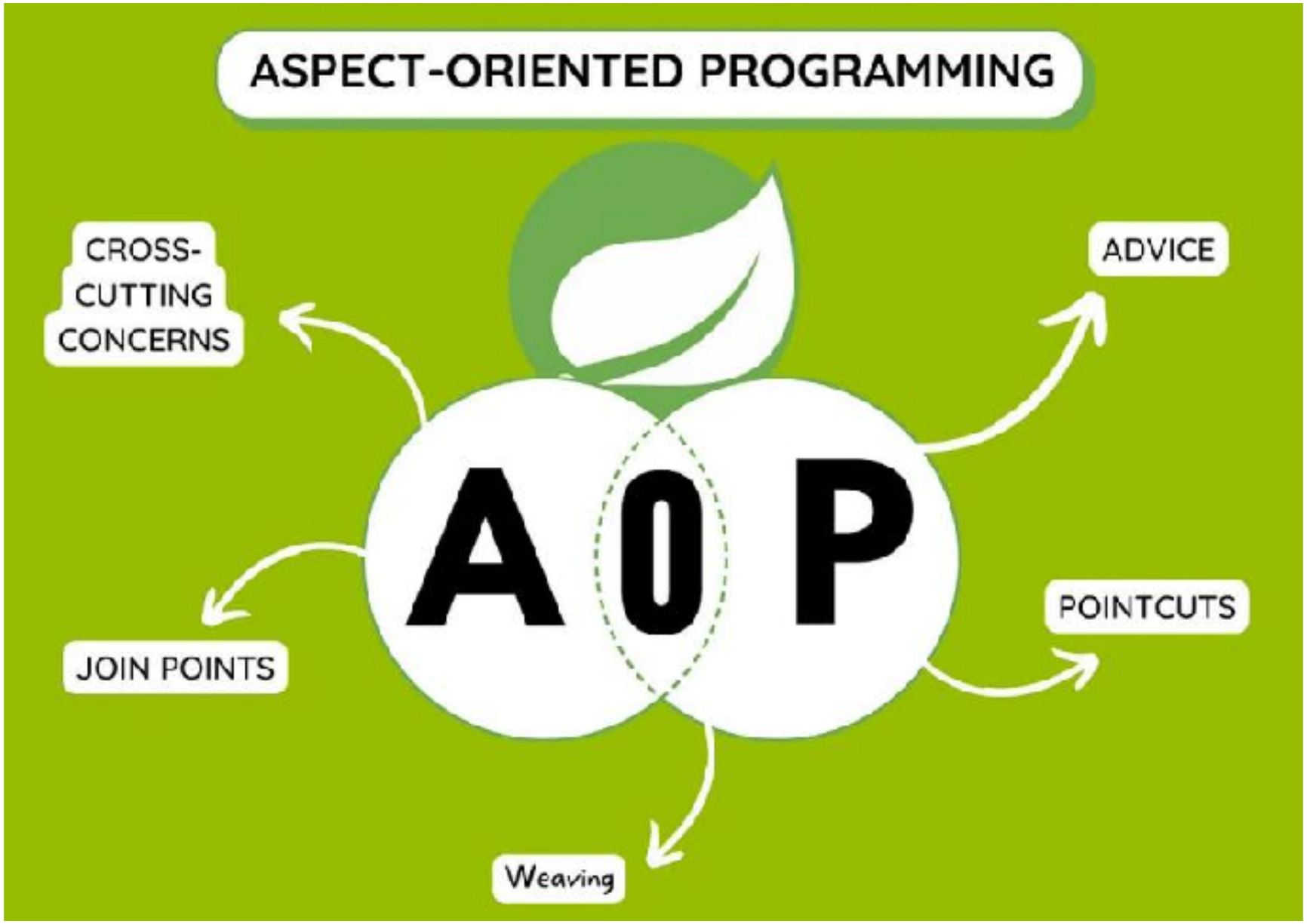
Isolation Levels

Performance (concurrency) vs Consistency (data integrity)

Isolation Level	Dirty read	Non-repeatable read	Phantom read	Performance
READ_UNCOMMITTED	Allowed	Allowed	Allowed	Fastest
READ_COMMITED	Prevented	Allowed	Allowed	Fast
REPEATABLE_READ	Prevented	Prevented	Allowed*	Moderate
SERIALIZABLE	Prevented	Prevented	Prevented	Slowest

Spring AOP !!

https://docs.spring.io/spring-framework/reference/core/aop.html



https://docs.spring.io/spring-framework/reference/core/aop.html

Spring AOP

Logging
Null-safety
Resilience

https://github.com/up1/course-springboot-2024/wiki/AOP-with-Logging

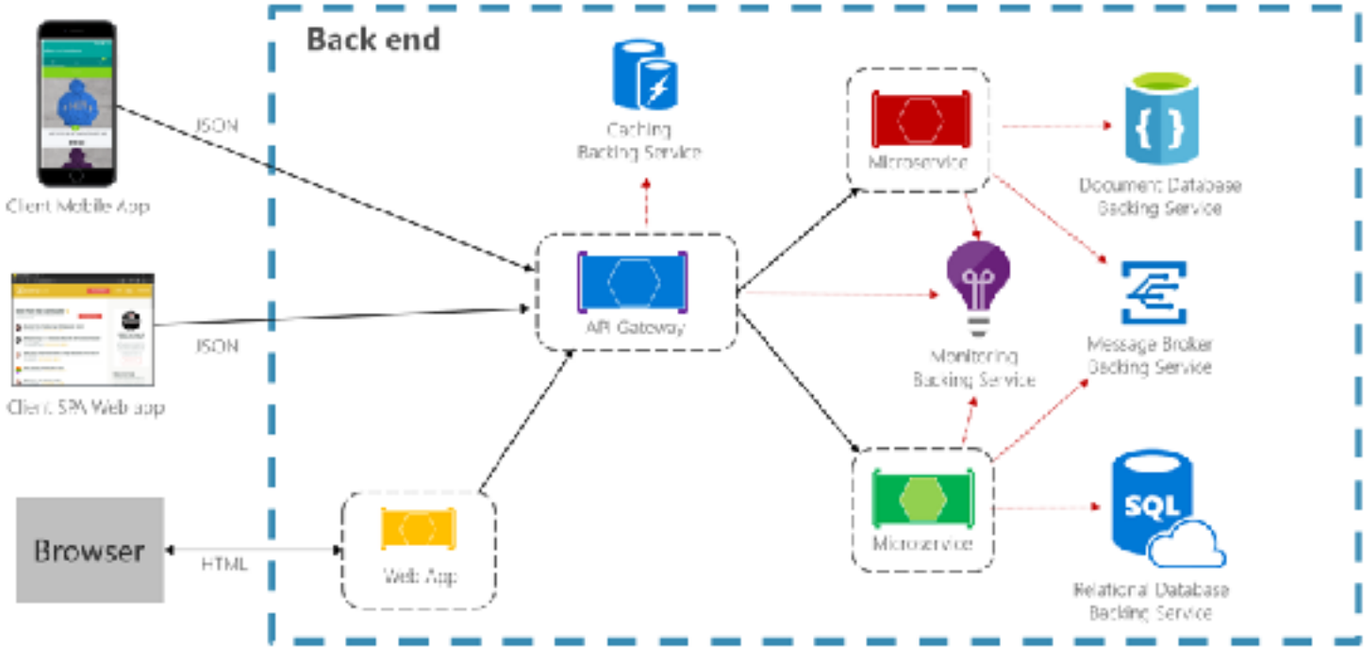
Resilience Patterns

Cascading effects of Failure !!

Resilience Patterns

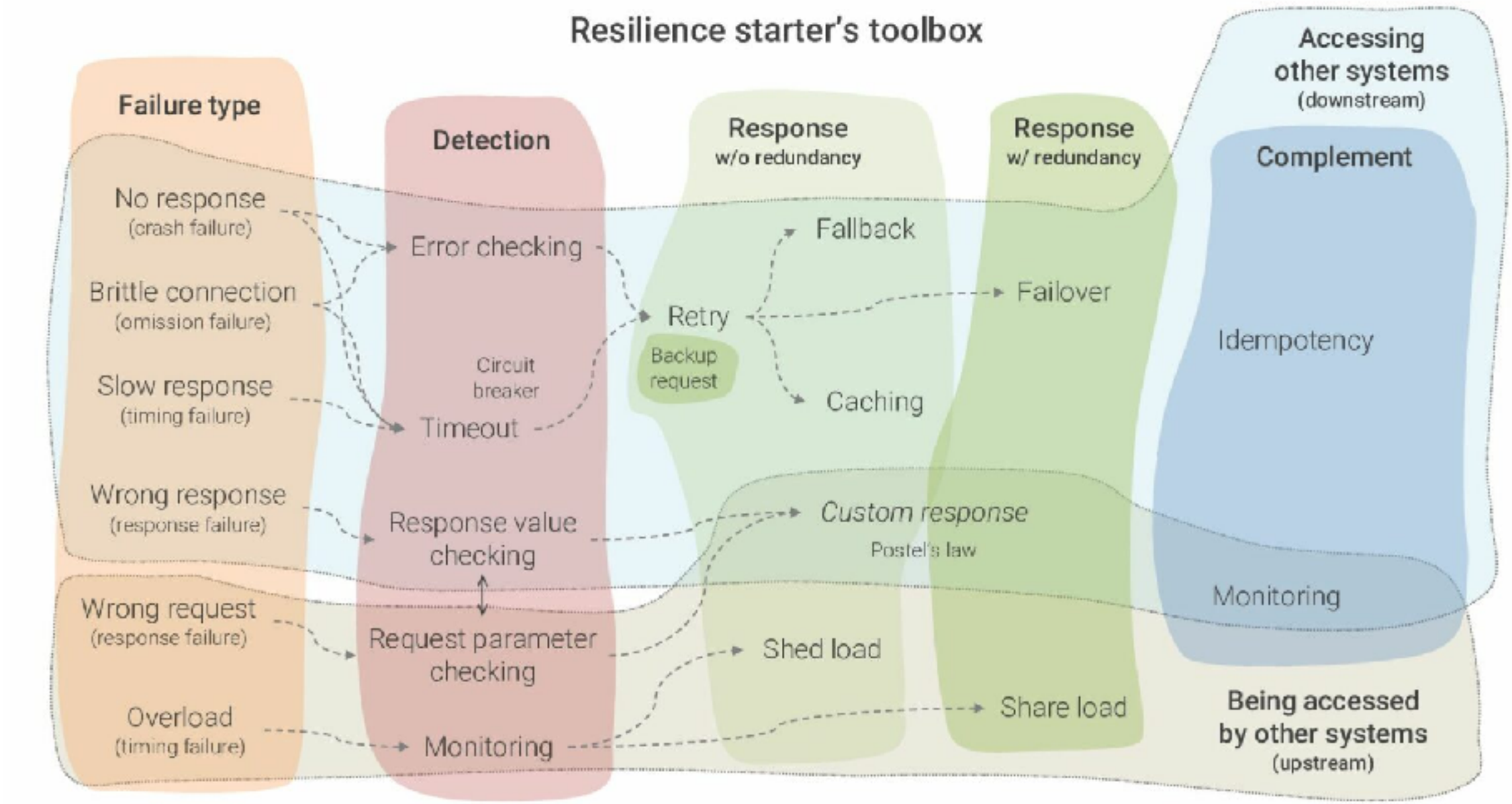
Ability of your system to react to failure and still remain functionality

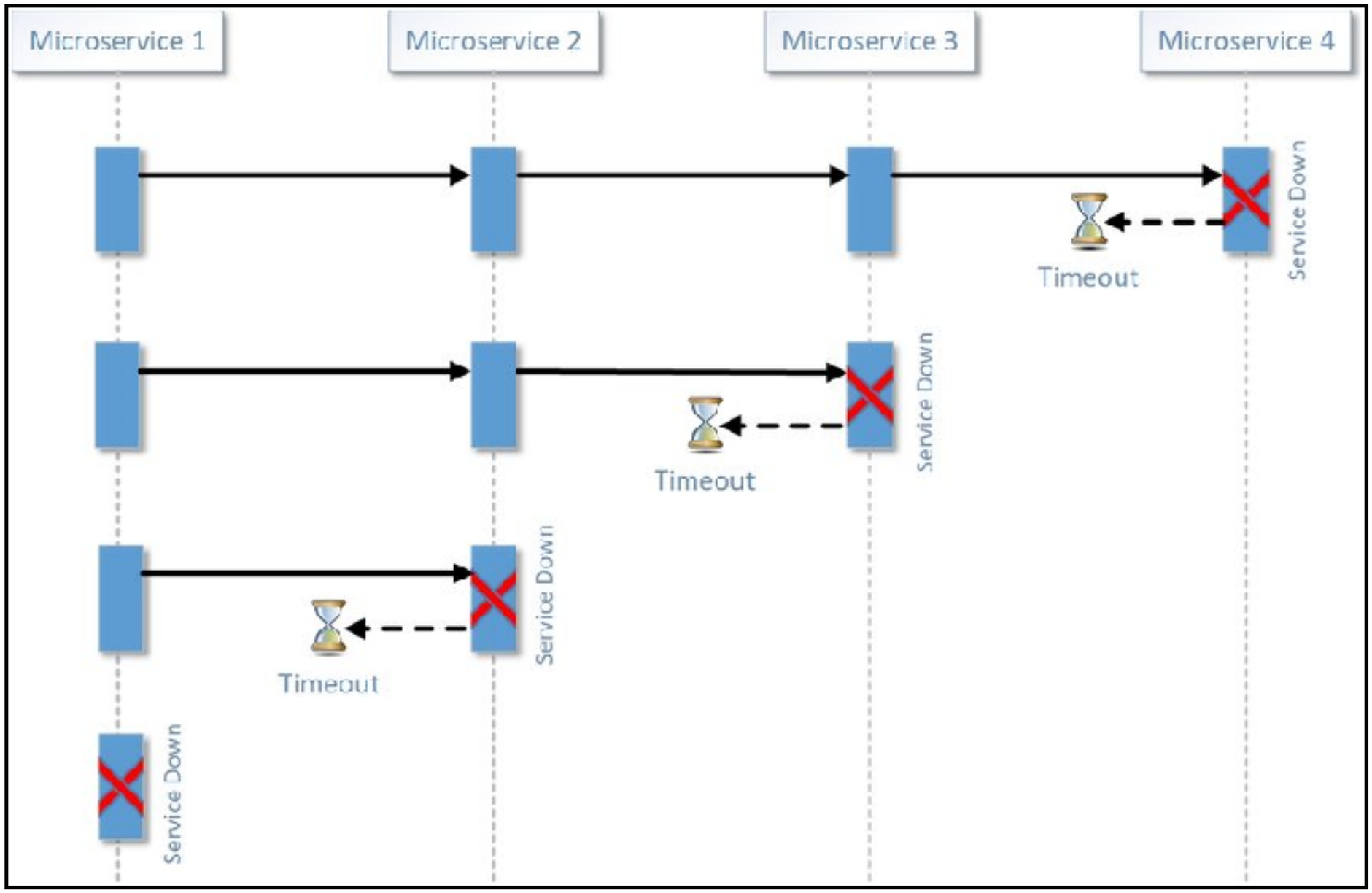
It’s not about avoiding failure, but accepting failure



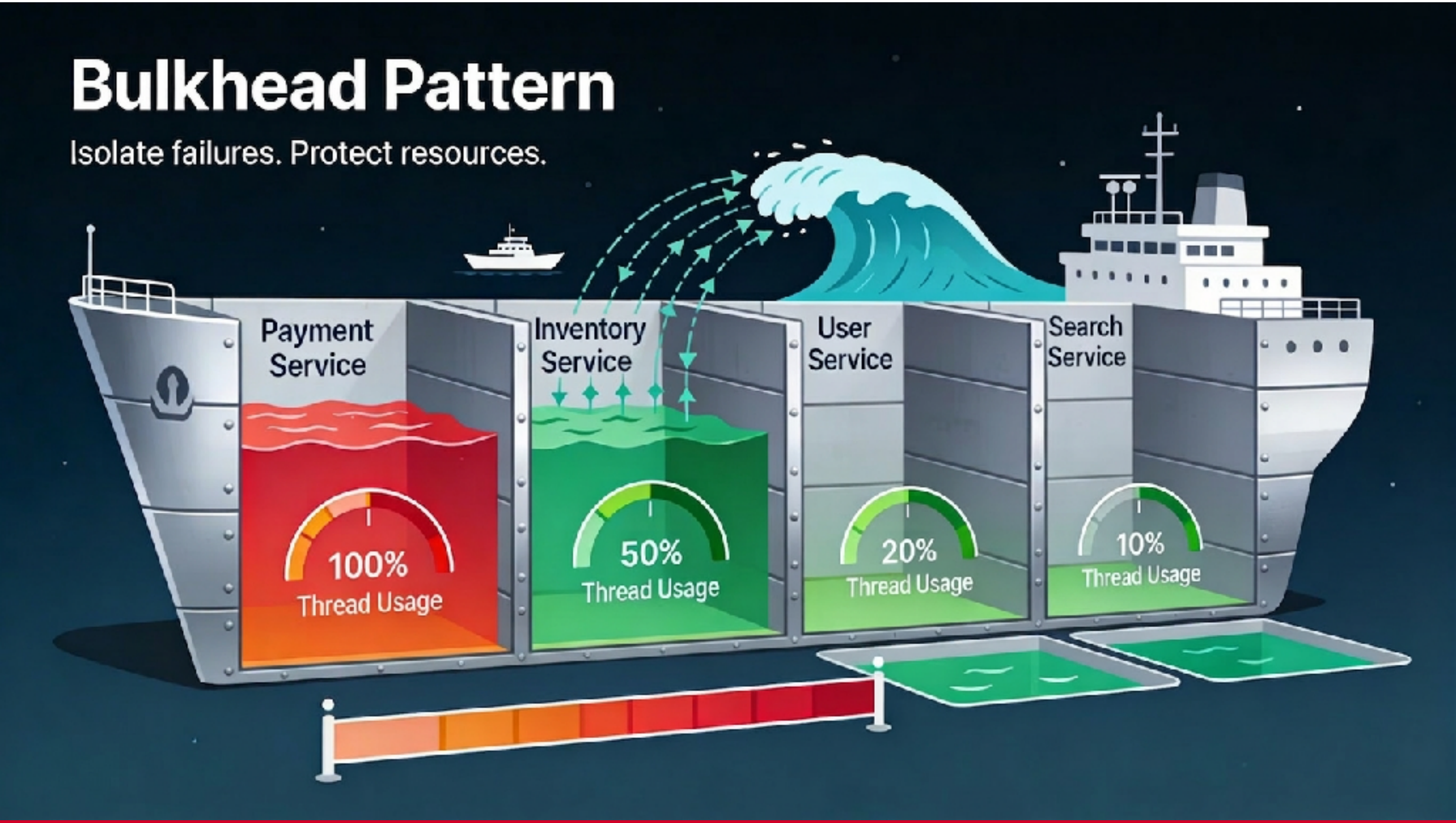
https://learn.microsoft.com/en-us/dotnet/architecture/cloud-native/resiliency

Resilience Patterns

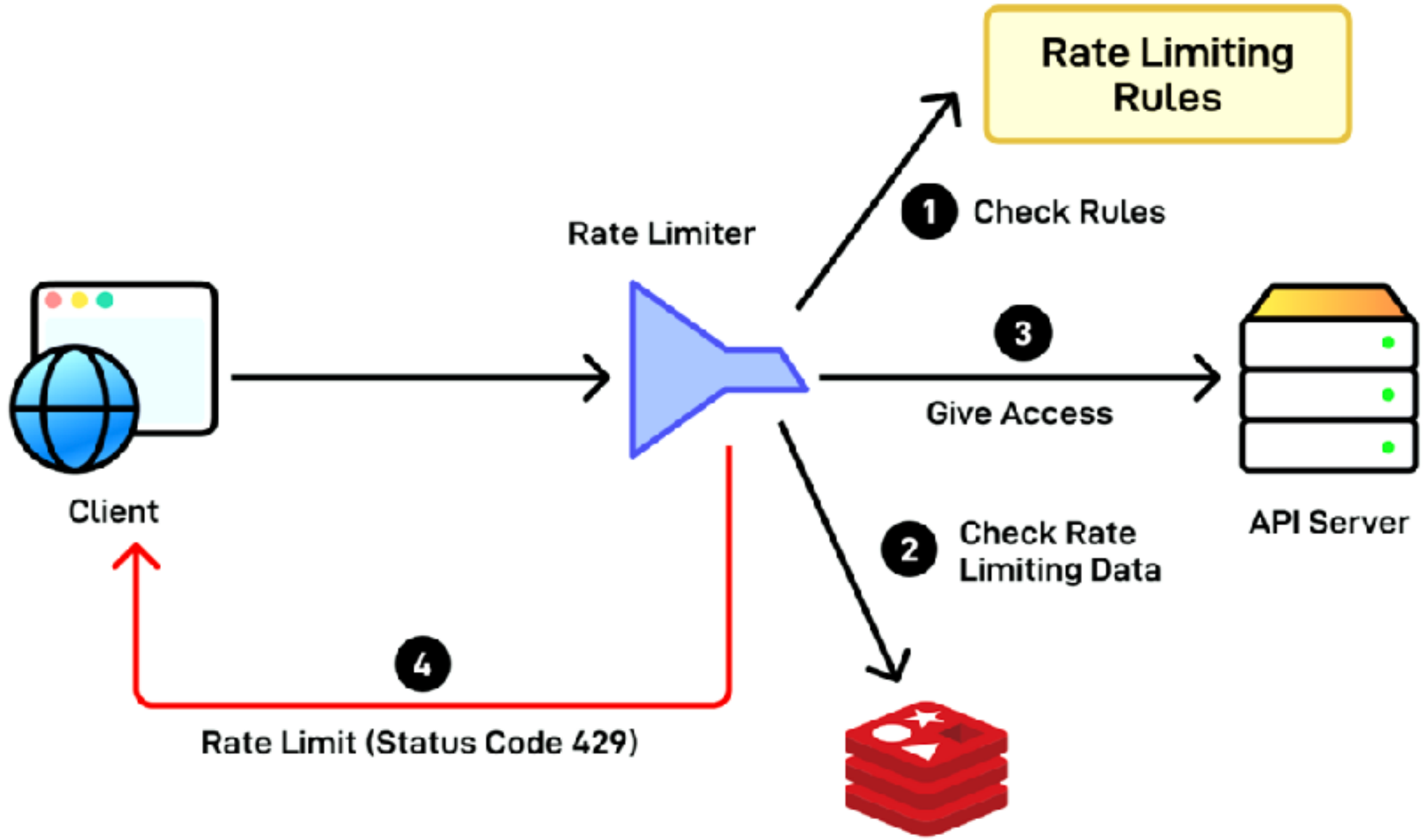




Bulk Head



Rate Limiting



Resilience Strategies (Reactive)

Strategy	Description
Retry	Many faults are transient and may self-correct after a short delay
Circuit breaker	When a system is seriously struggling, failing fast is better than making users/callers wait
Fallback	Things will still fail plan what you will do when that happens

Resilience Strategies (Proactive)

Strategy	Description
Timeout	Beyond a certain wait, a success result is unlikely
Rate limit	Limiting the rate a system handles requests is another way to control load
Queue/Messaging	Back pressure, wait to processing

Spring Framework 7

Resilience build-in features

@Retryable

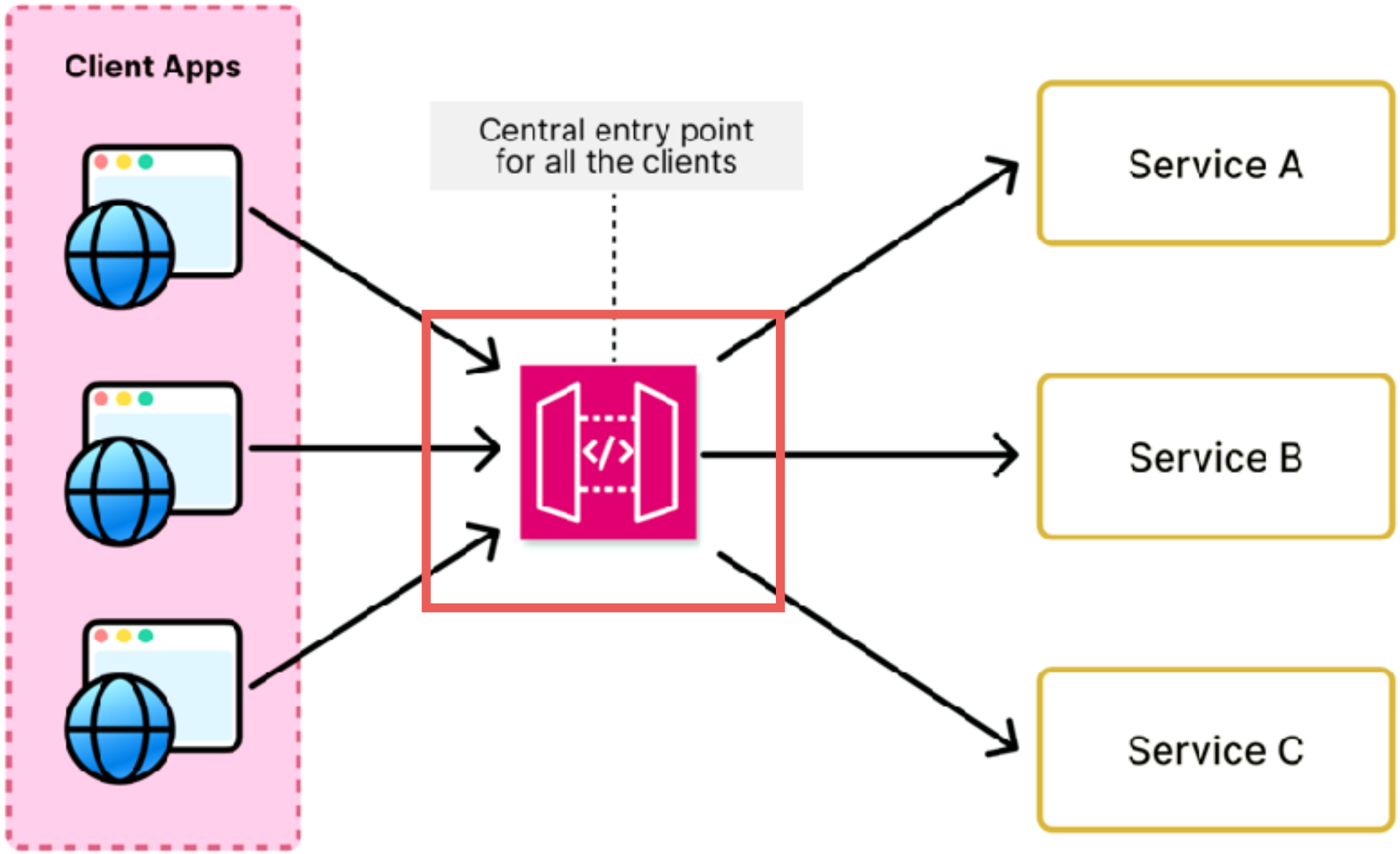
@ConcurrencyLimit

https://docs.spring.io/spring-framework/reference/core/resilience.html

Java Resilience Libraries

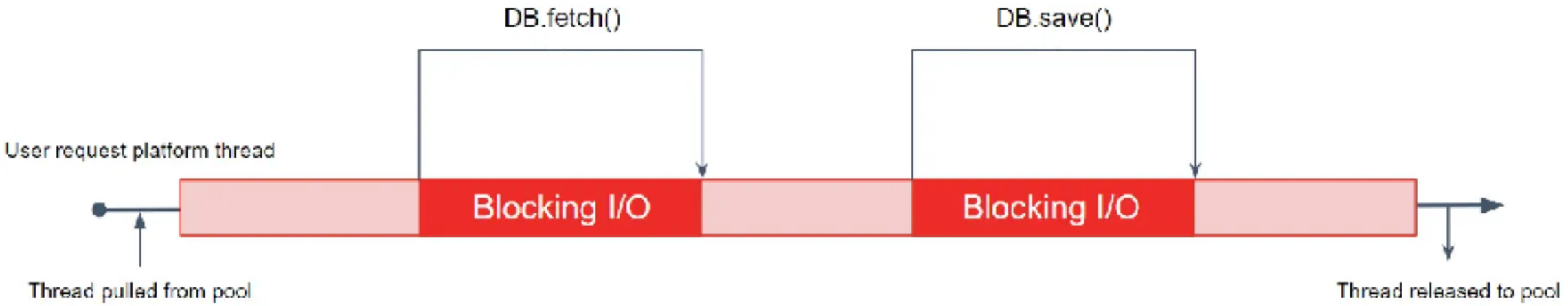
Resilience4j
Netflix Hystrix

Working with API Gateway



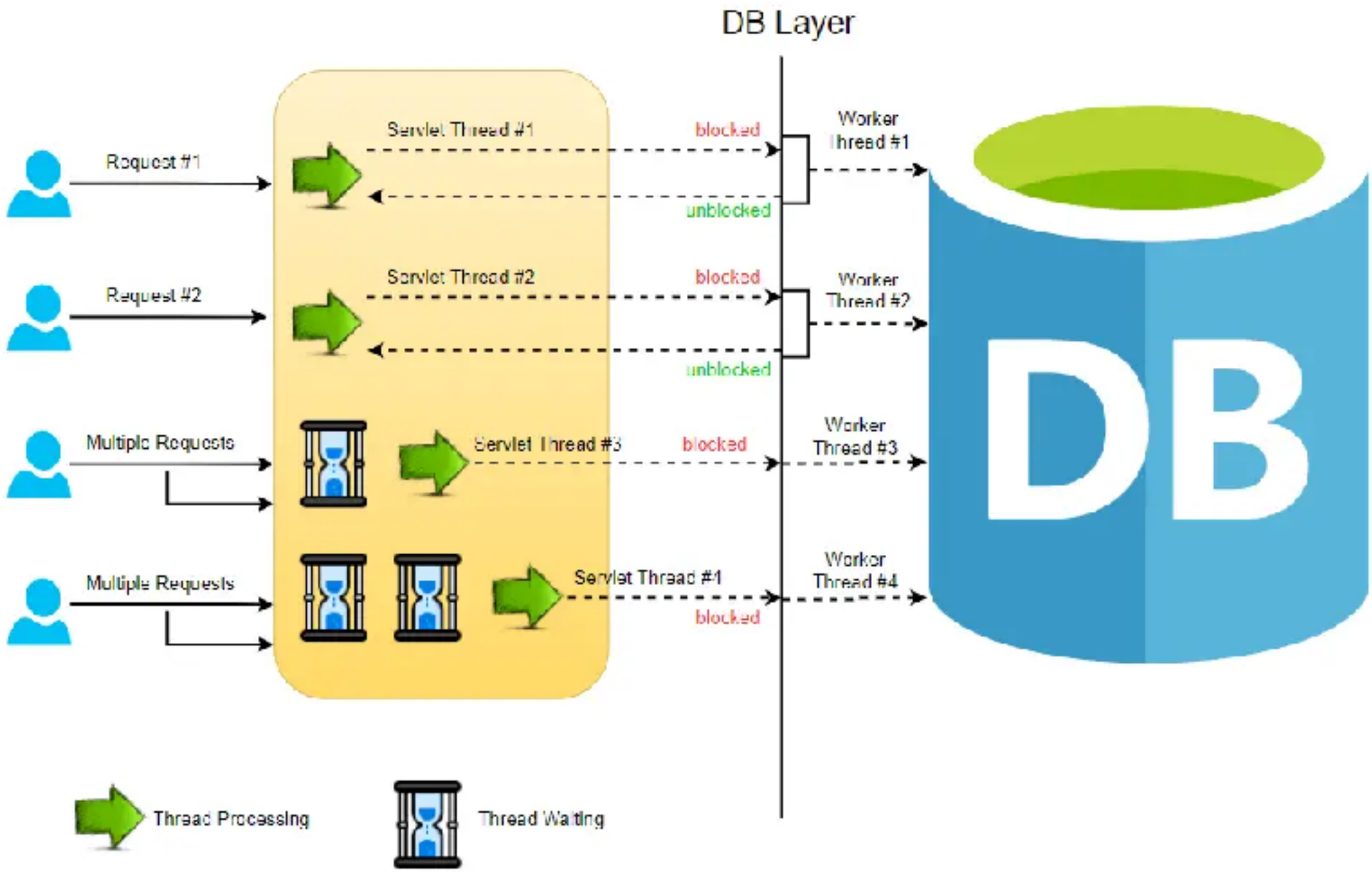
Non-Blocking I/O

Blocking I/O



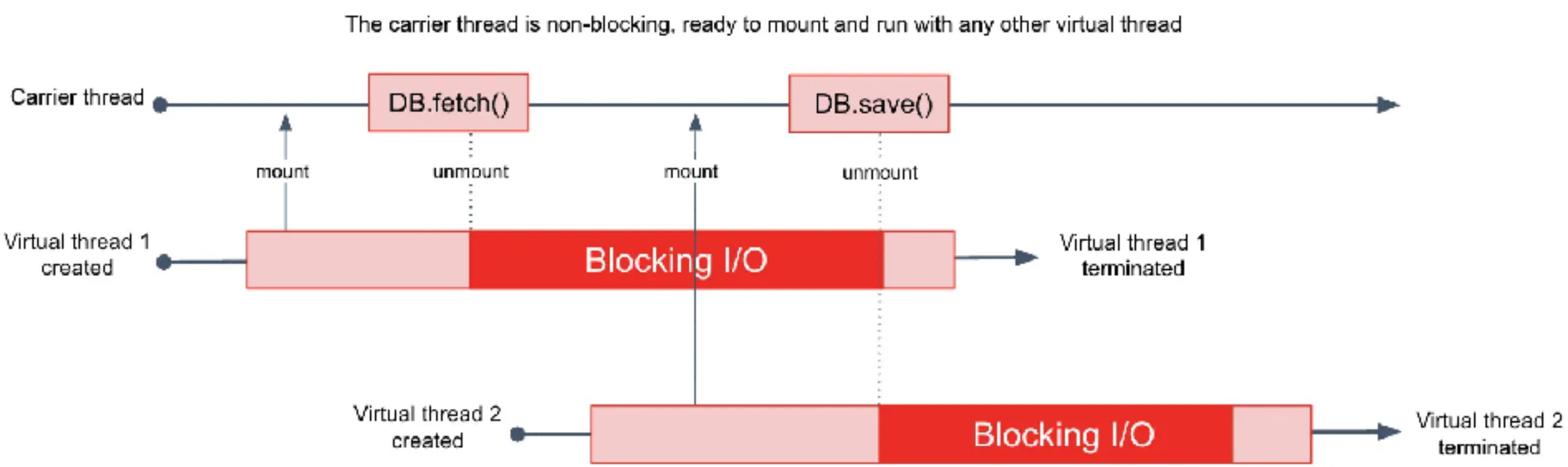
https://developers.ascendcorp.com/why-are-my-java-virtual-threads-slower-than-the-platform-threads-74612a1587f3

Blocking I/O



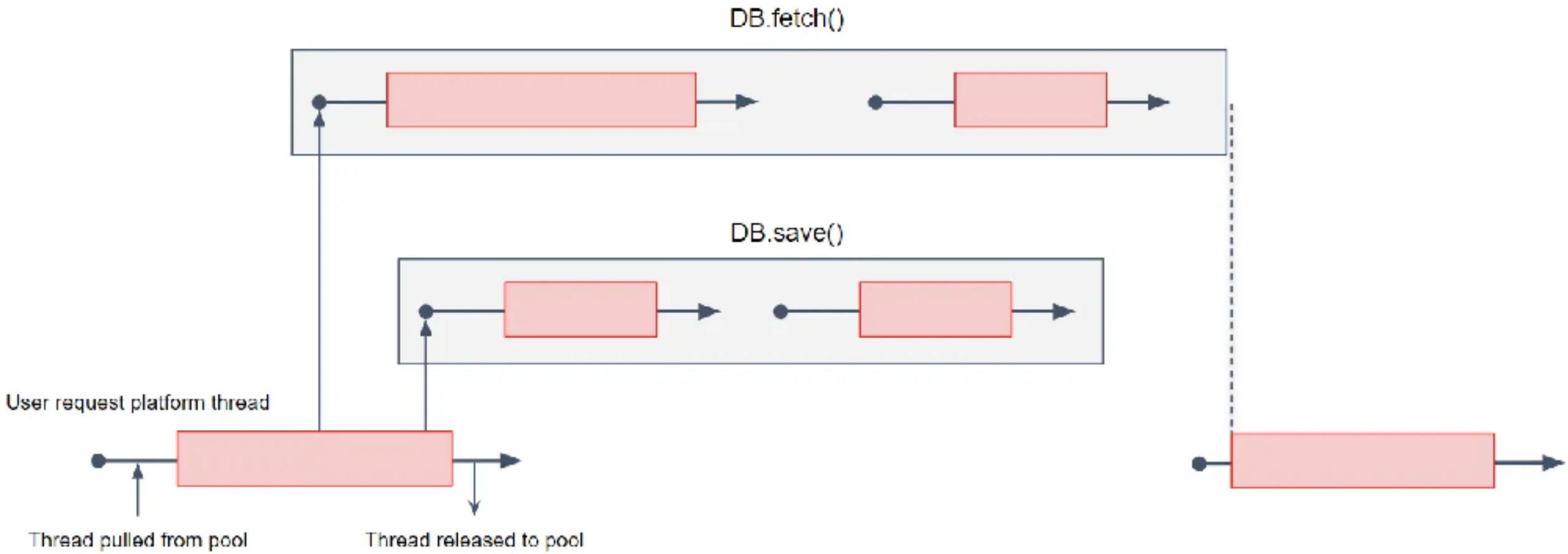
https://reflectoring.io/getting-started-with-spring-webflux/

Blocking I/O with Virtual Thread

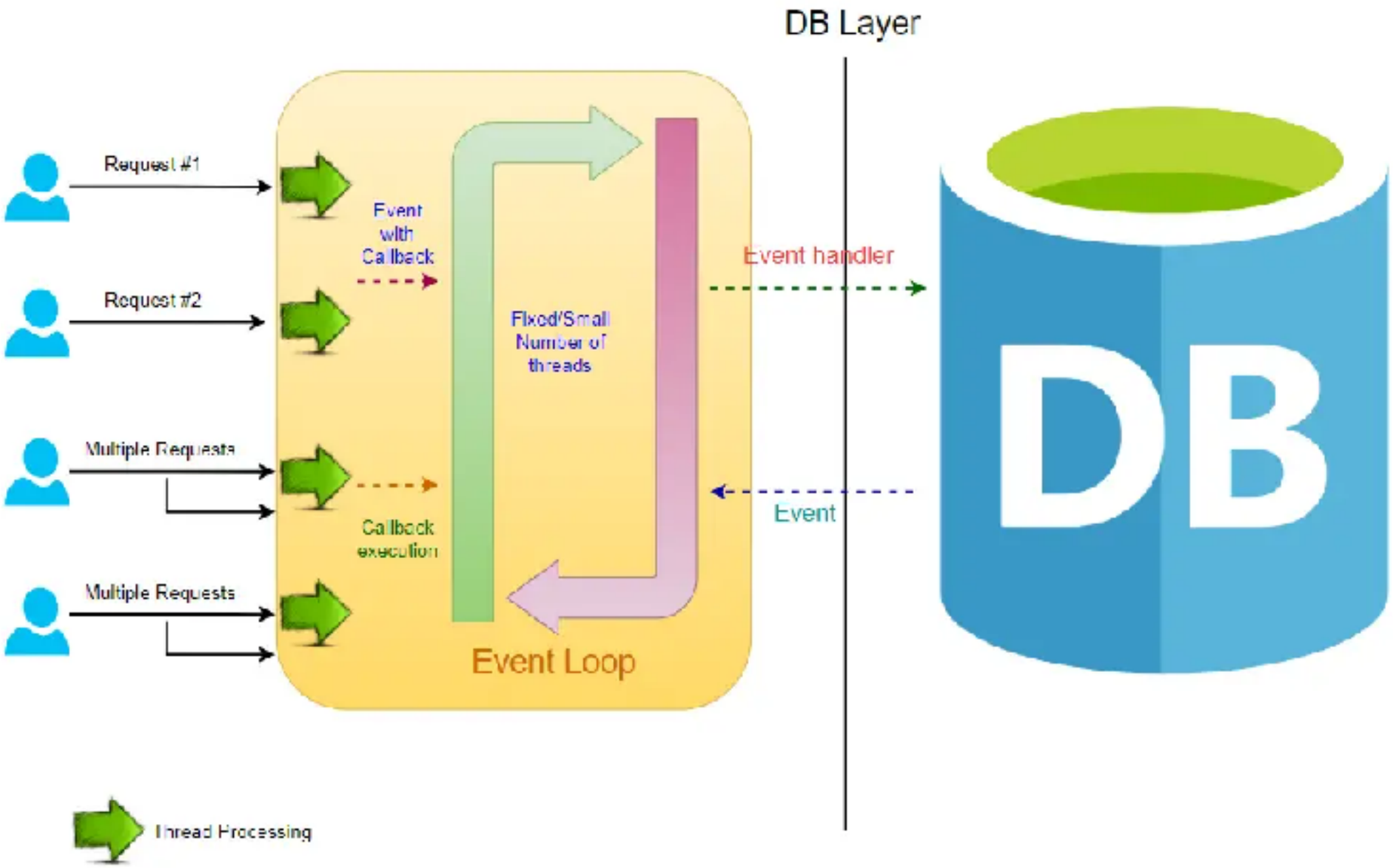


Non-Blocking I/O

Reactive framework

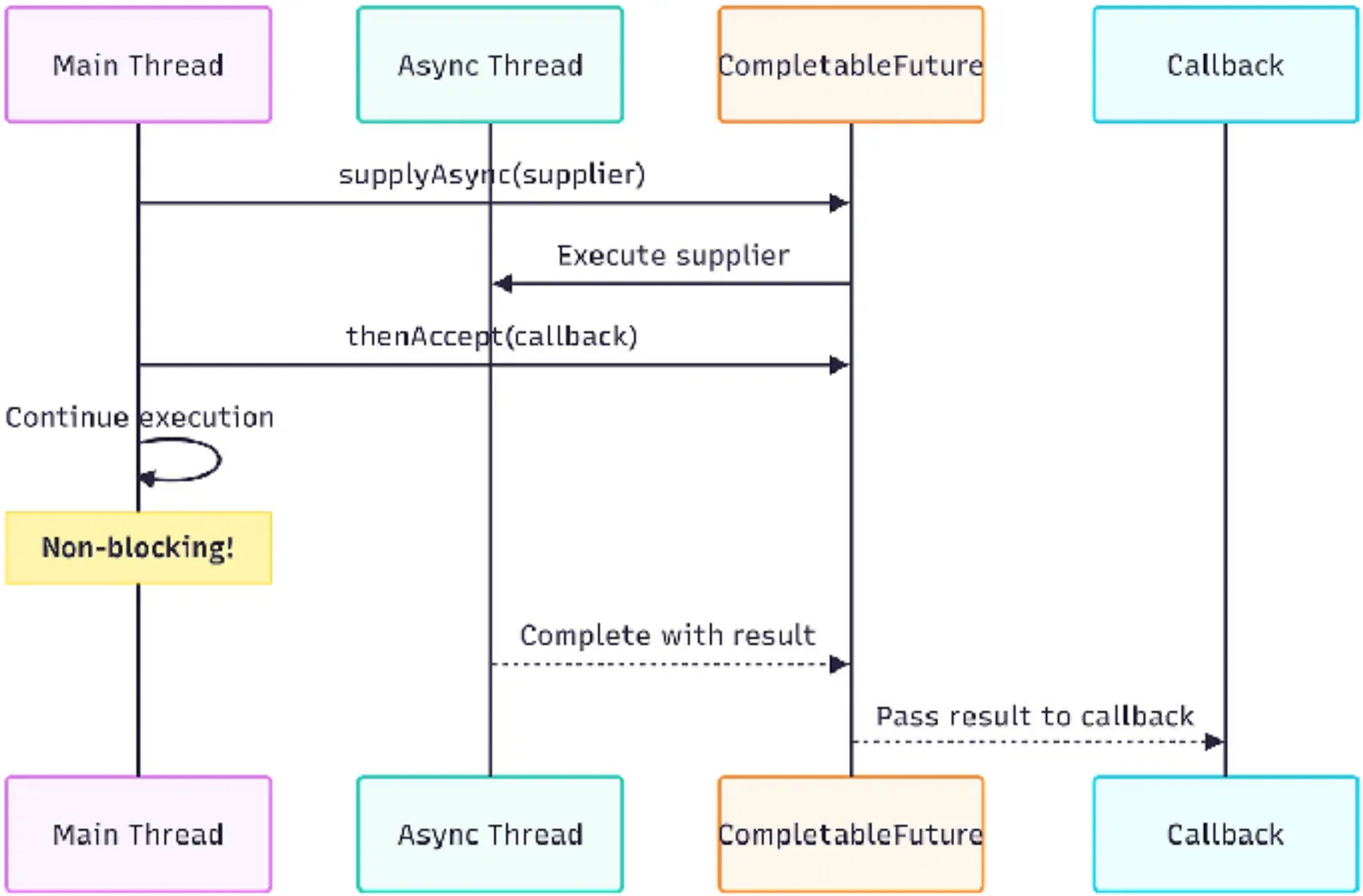


Non-Blocking I/O



https://reflectoring.io/getting-started-with-spring-webflux/

CompletableFuture



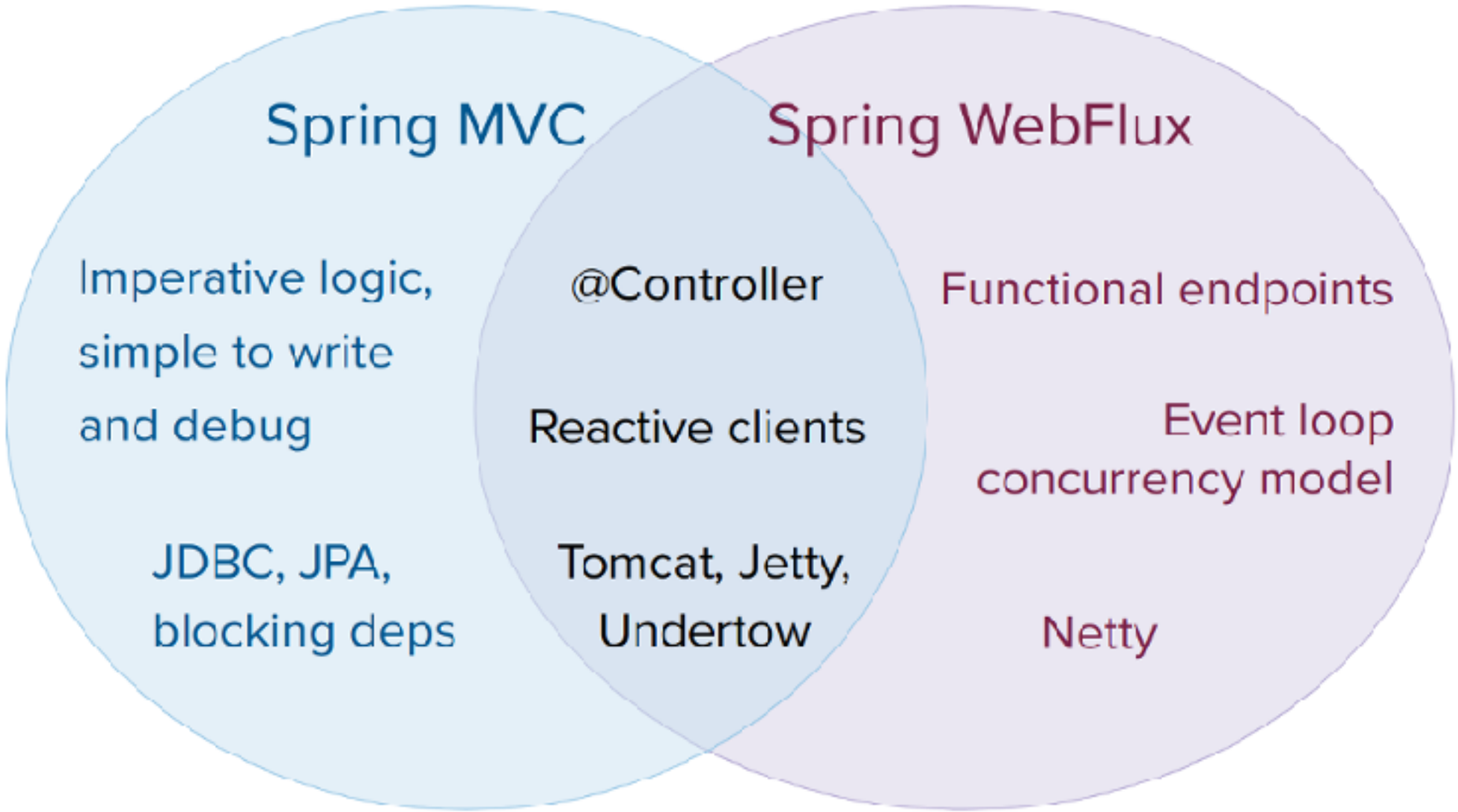
https://mobisoftinfotech.com/resources/blog/java-programming/java-async-programming-guide

Reactive in Java

QUARKUS

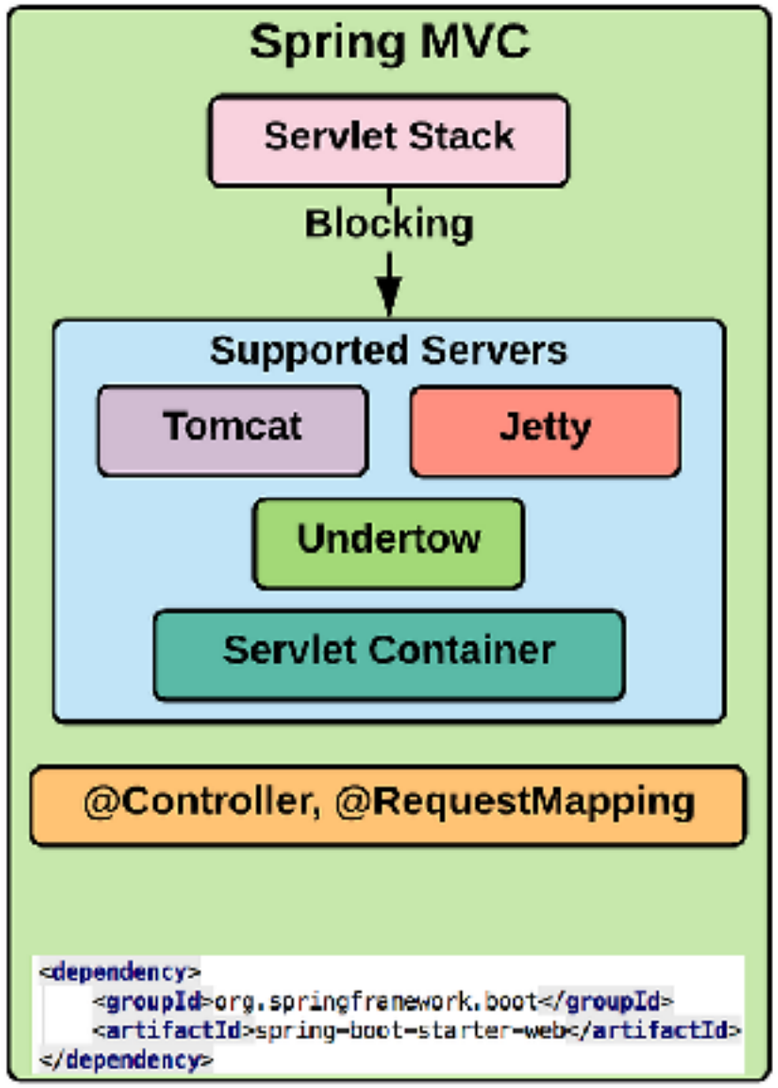
Spring Webflux

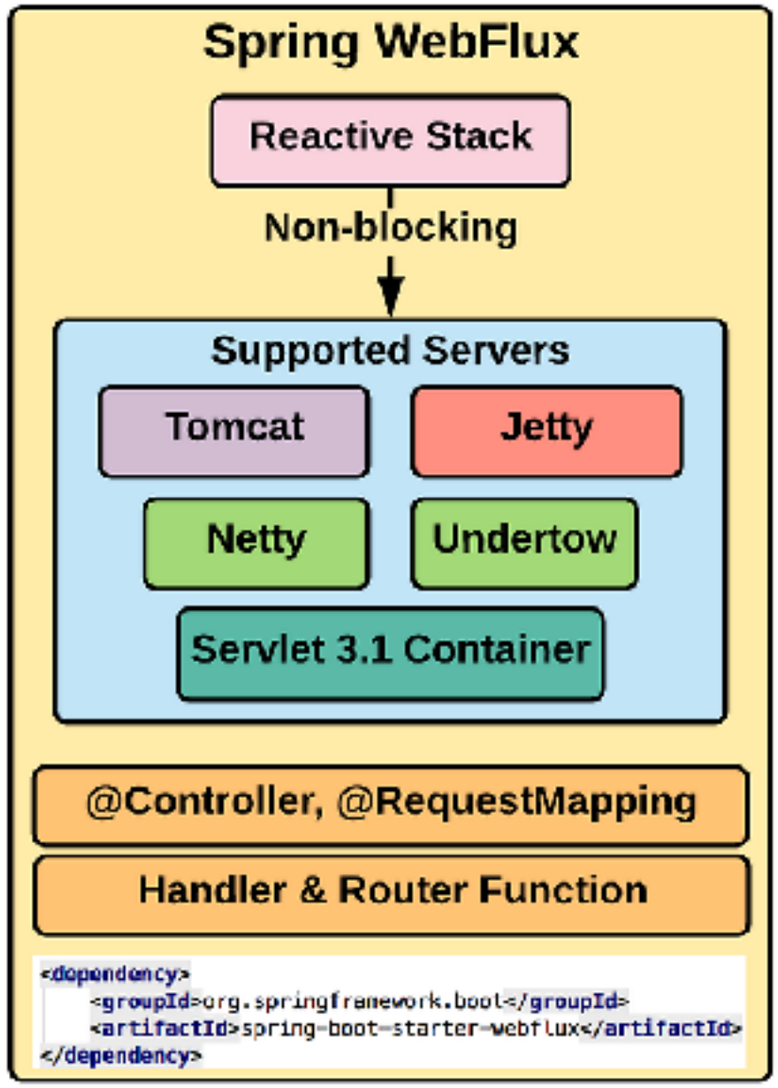
Spring WebFlux



https://docs.spring.io/spring-framework/reference/web/webflux.html

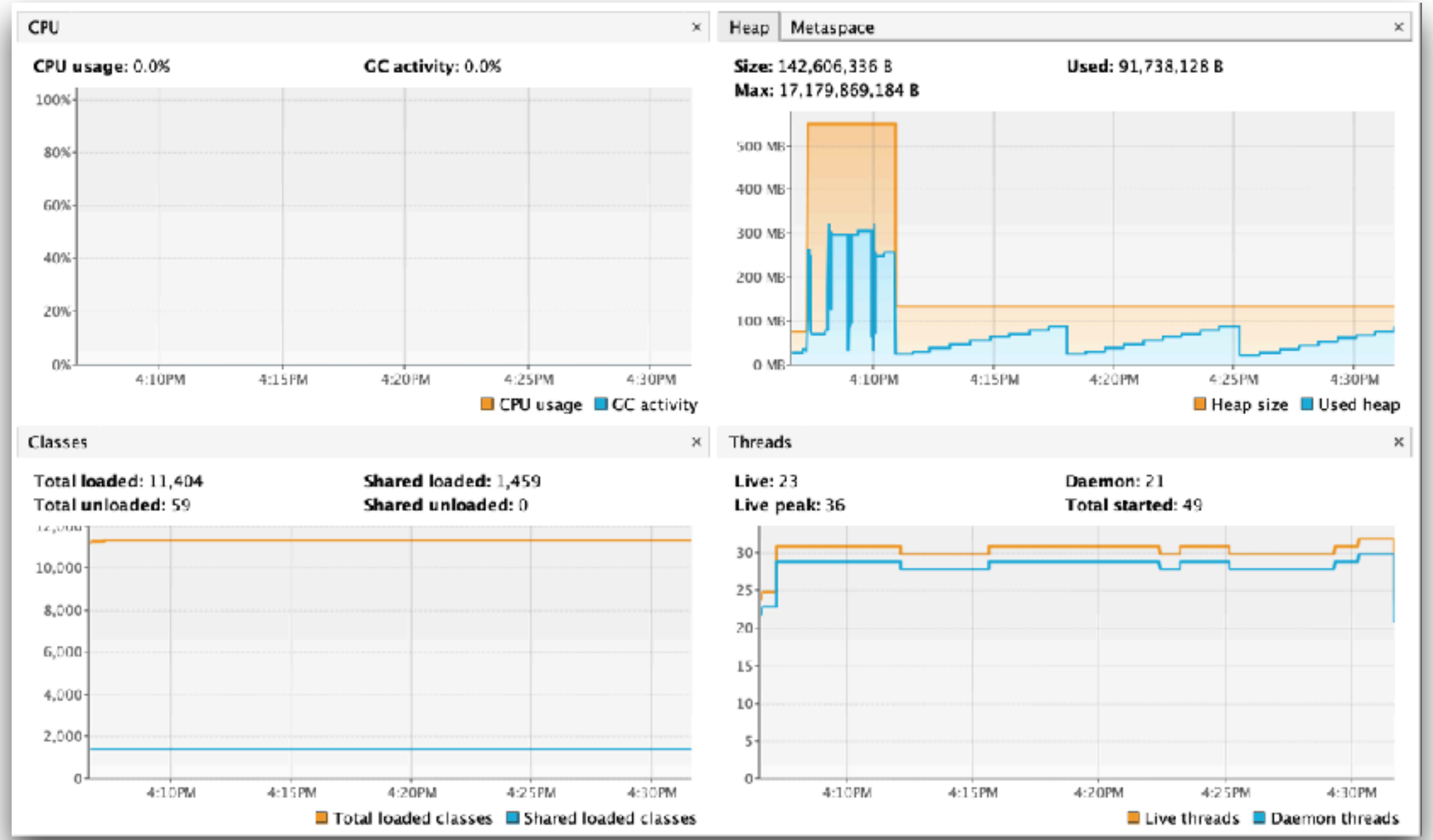
Spring MVC vs WebFlux





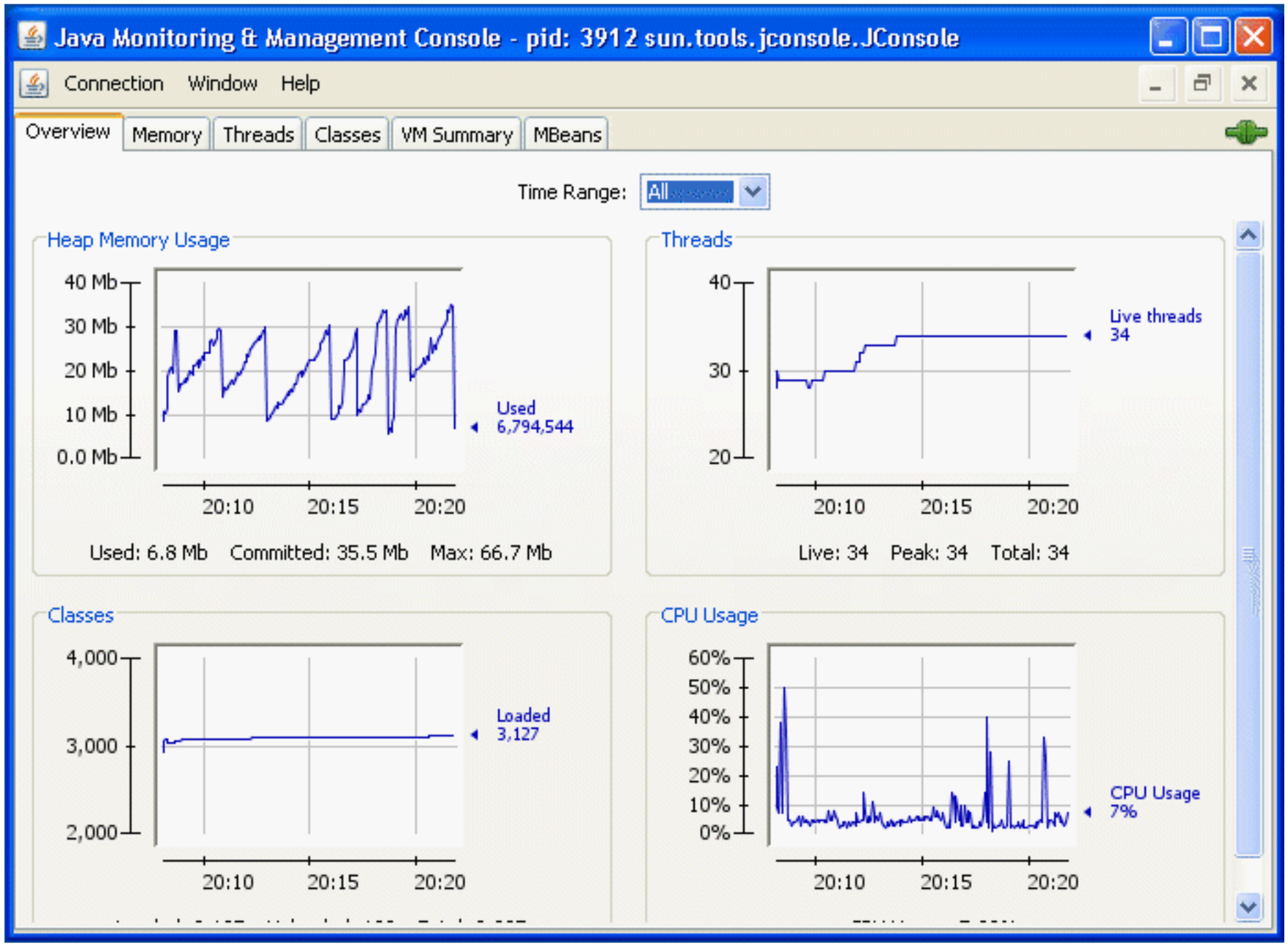
https://docs.spring.io/spring-framework/reference/web/webflux.html

Monitoring with VisualVM



https://visualvm.github.io/

Monitoring with JConsole



https://docs.oracle.com/en/java/javase/26/management/using-jconsole.html

JAVA 27

https://openjdk.org/projects/jdk/27/

Q/A